Contents

Table of Contents

- Publishing scripts
- Limitations
- General
- Alerts
- Data structures
- Functions
- Indicators
- Other data and timeframes
- Programming
- Strategies
- Strings and formatting
- Techniques
- Error messages
- Release notes
- Overview
- To Pine Script™ version 6
- To Pine Script™ version 5
- To Pine Script™ version 4
- To Pine Script™ version 3
- To Pine Script™ version 2
- Where can I get more information?

User Manual/Welcome to Pine Script™ v6

Welcome to Pine Script™ v6

Pine Script™ is TradingView's programming language. It allows traders to create their own trading tools and run them on our servers. We designed Pine Script™ as a lightweight, yet powerful, language for developing indicators and strategies that you can then backtest. Most of TradingView's built-in indicators are written in Pine Script™, and our thriving community of Pine Script™ programmers has published more than 150,000 Community Scripts, half of which are open-source.

Requirements

It's our explicit goal to keep Pine Script™ accessible and easy to understand for the broadest possible audience. Pine Script™ is cloud-based and therefore different from client-side programming languages. While we likely won't develop Pine Script™ into a full-fledged language, we do constantly improve it and are always happy to consider requests for new features.

Because each script uses computational resources in the cloud, we must impose limits in order to share these resources fairly among our users. We strive to set as few limits as possible, but will of course have to implement as many as needed for the platform to run smoothly. Limitations apply to the amount of data requested from additional symbols, execution time, memory usage and script size.

[Next

First steps](#first-steps) User Manual/Pine Script™ primer/First steps

First steps

Introduction

Welcome to the Pine Script™ v6 User Manual, which will accompany you in your journey to learn to program your own trading tools in Pine Script™. Welcome also to the very active community of Pine Script™ programmers on TradingView.

On this page, we present a step-by-step approach that you can follow to gradually become more familiar with indicators and strategies (also called *scripts*) written in the Pine Script™ programming language on TradingView. We will get you started on your journey to:

1. **Use** some of the tens of thousands of existing scripts on the platform.
2. **Read** the Pine Script™ code of existing scripts.
3. **Write** Pine scripts.

If you are already familiar with the use of Pine scripts on TradingView and are now ready to learn how to write your own, then jump to the Writing scripts section of this page.

If you are new to our platform, then please read on!

Using scripts

If you are interested in using technical indicators or strategies on TradingView, you can first start exploring the thousands of indicators already available on our platform. You can access existing indicators on the platform in two different ways:

- By using the chart's "Indicators, metrics, and strategies" button.
- By browsing TradingView's Community scripts, the largest repository of trading scripts in the world, with more than 150,000 scripts, half of which are free and *open-source*, which means you can see their Pine Script™ code.

If you can find the tools you need already written for you, it can be a good way to get started and gradually become proficient as a script user, until you are ready to start your programming journey in Pine Script™.

Loading scripts from the chart

To explore and load scripts from your chart, click the "Indicators, metrics, and strategies" button, or use the forward slash / keyboard shortcut:

Figure 1: image

The dialog box that appears presents different categories of scripts in its left pane:

- **"Favorites"** lists the scripts you have "favorited" by clicking on the star that appears to the left of the script name when you hover over it.
- **"Personal"** displays the scripts you have written and saved in the Pine Editor. They are saved on TradingView's servers.
- **"Technicals"** groups most TradingView built-in scripts, organized in four categories: "Indicators", "Strategies", "Profiles", and "Patterns". Most are written in Pine Script™ and available for free.
- **"Financials"** contains all built-in indicators that display financial metrics. The contents of that tab and the subcategories they are grouped into depend on the symbol currently open on the chart.
- **"Community"** is where you can search from the more than 150,000 published scripts written by TradingView users. The scripts can be sorted by one of the three different filters — "Editors' picks" only shows open-source scripts hand-picked by our script moderators, "Top" shows the most popular scripts of all time, and "Trending" displays the most popular scripts that were published recently.
- **"Invite-only"** contains the list of the invite-only scripts you have been granted access to by their authors.

Here, we selected the "Technicals" tab to see the TradingView built-in indicators:

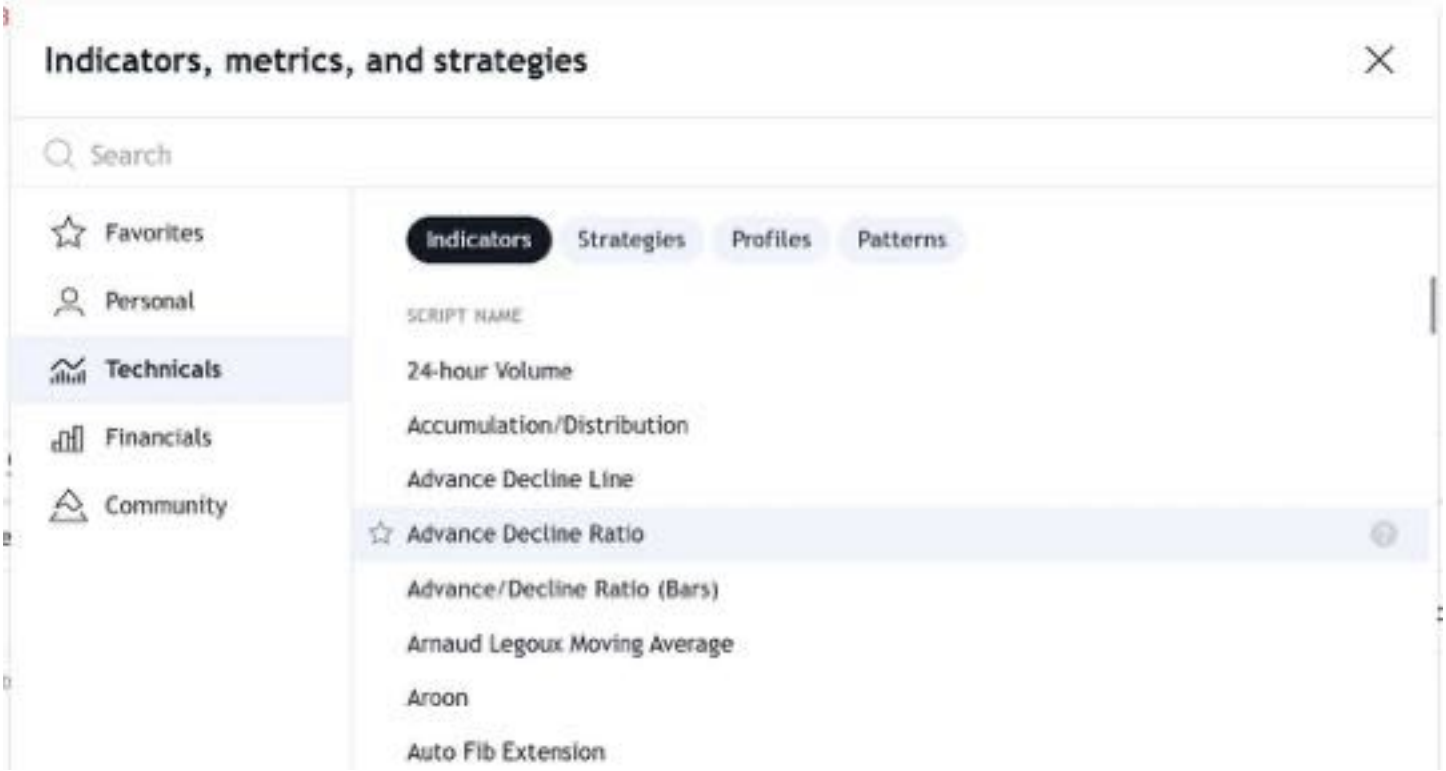

Figure 2: image

Clicking on one of the listed indicators or strategies loads the script on your chart. Strategy scripts are distinguished from indicators by a special symbol that appears to the right of the script name.

Browsing community scripts

To access the Community scripts feed from TradingView's homepage, select "Indicators and strategies" from the "Community" menu:

You can also search for scripts using the homepage's "Search" field, and filter scripts using different criteria. See this Help Center page explaining the different types of scripts that are available.

The scripts feed generates *script widgets*, which show the title and author of each publication with a preview of the published chart and description. Clicking on a widget opens the *script page*, which shows the publication's complete description, an enlarged chart, and any additional release notes. Users can boost, favorite, share, and comment on publications. If it is an open-source script, the source code is also available on the script page.

When you find an interesting script in the Community scripts, follow the instructions in the Help Center to load it on your chart.

Changing script settings

Once a script is loaded on the chart, you can double-click on its name or hover over the name and press the "Settings" button to bring up its "Settings/Inputs" tab:

The "Inputs" tab allows you to change the settings which the script's author has decided to make editable. You can configure some of the script's visuals using the "Style" tab of the same dialog box, and which timeframes the script should appear on using the "Visibility" tab.

Other settings are available to all scripts from the buttons that appear to the right of its name when you mouse over it, and from the "More" menu (the three dots):

Reading scripts

Reading code written by **good** programmers is the best way to develop your understanding of the language. This is as true for Pine Script™ as it is for all other programming languages. Finding good open-source Pine Script™ code is relatively easy.

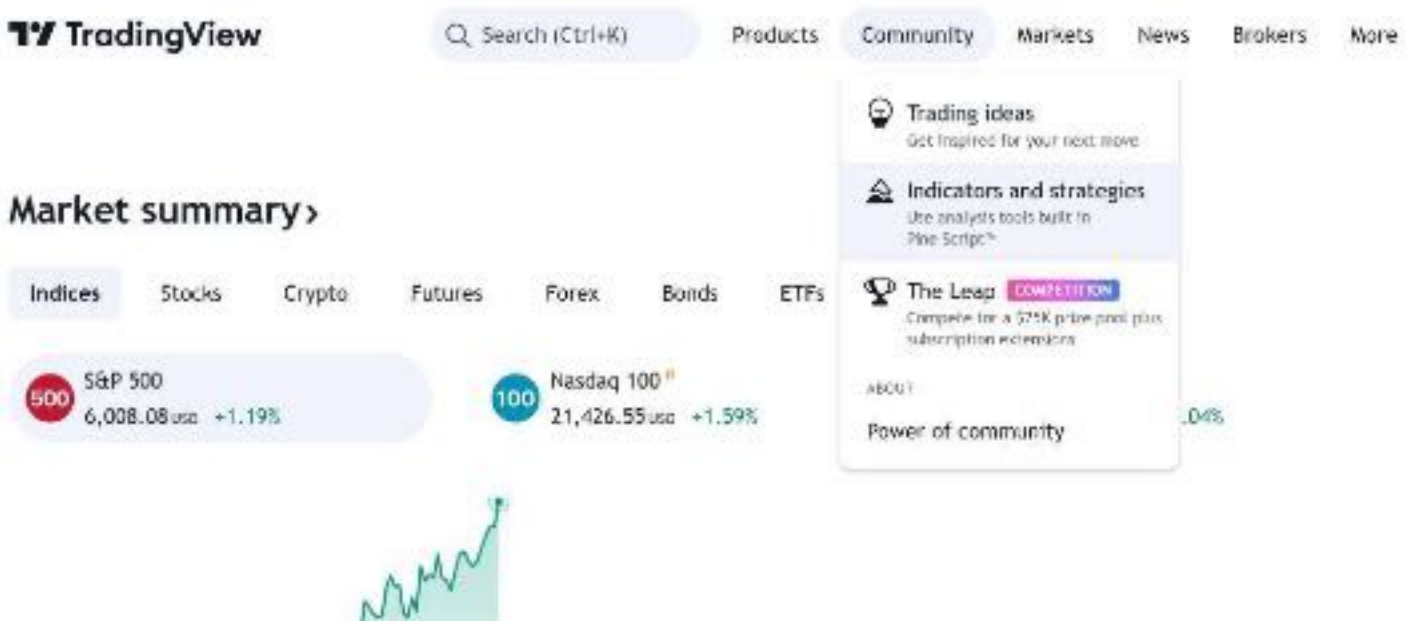

Figure 3: image

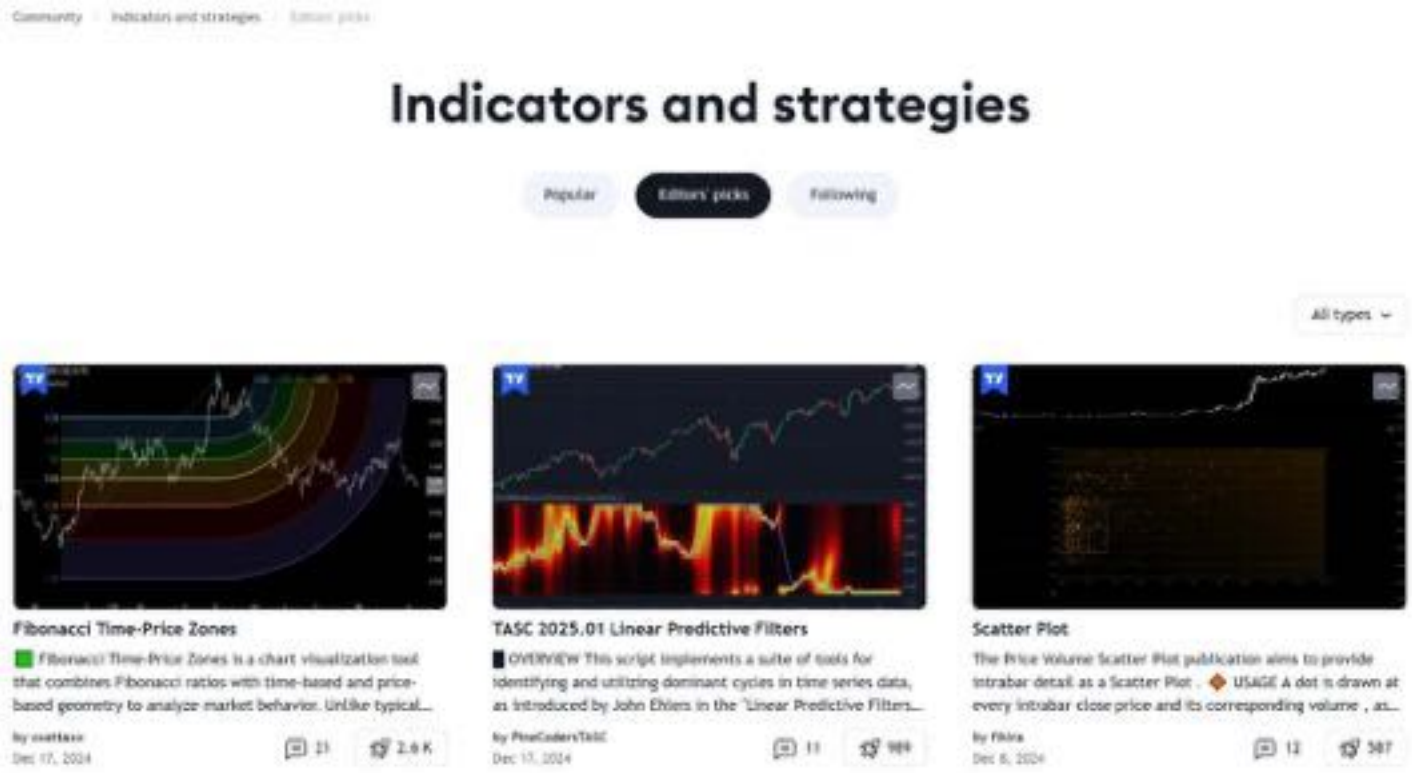

Figure 4: image

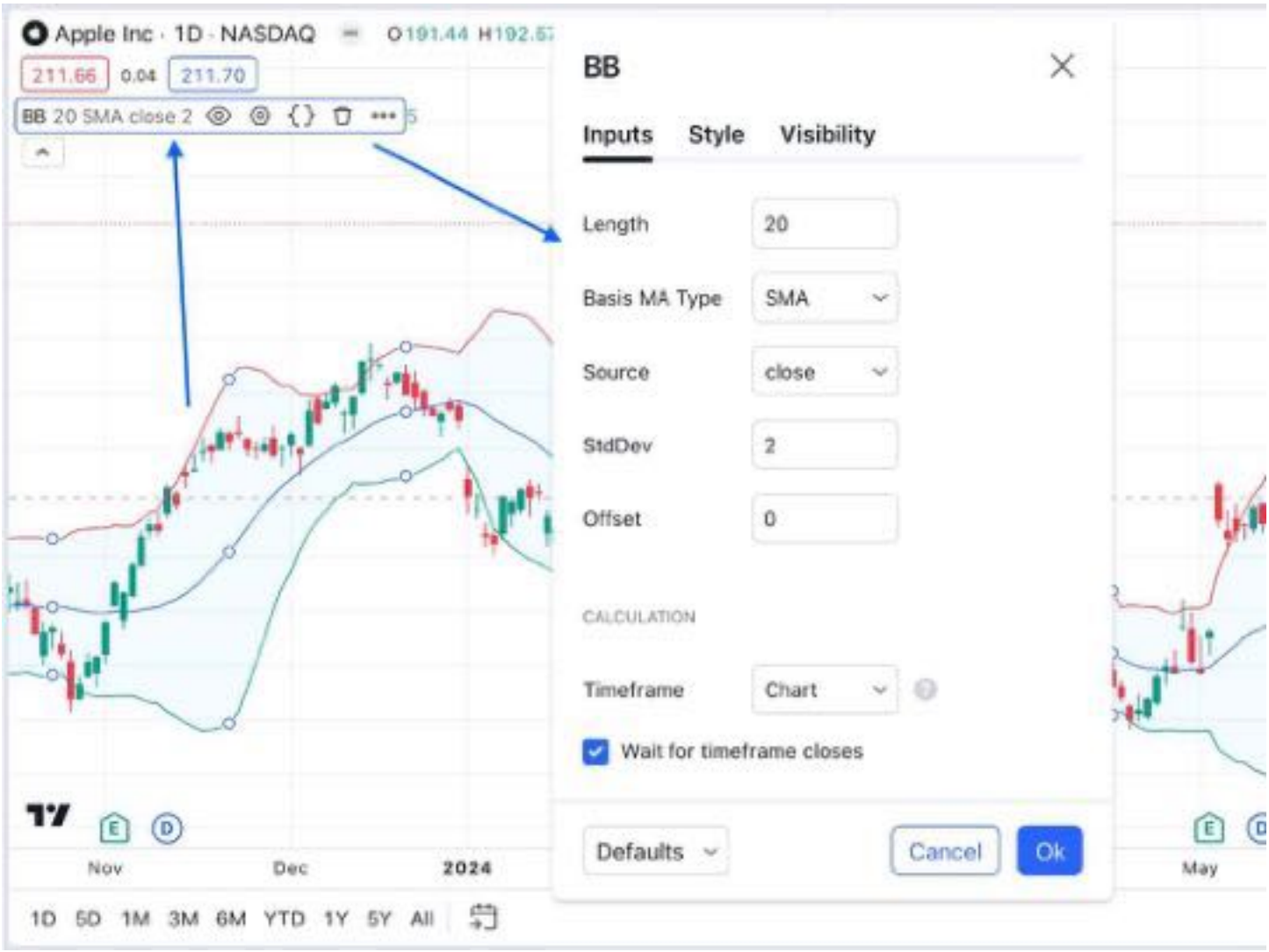

Figure 5: image

Figure 6: image

These are reliable sources of code written by good programmers on TradingView:

- The TradingView built-in indicators
- Scripts selected as Editors' Picks
- Scripts by the authors the PineCoders account follows
- Many scripts by authors with high reputations and open-source publications

Reading code from Community scripts is easy; if there is no grey or red "lock" icon in the upper-right corner of the script widget, then the script is open-source. By opening the script page, you can read its full source code.

To see the code of a TradingView built-in indicator, load the indicator on your chart, then hover over its name and select the "Source code" curly braces icon (if you don't see it, it's because the indicator's source is unavailable). When you click on the {} icon, the Pine Editor opens below the chart and displays the script's code. If you want to edit the script, you must first select the "Create a working copy" button. You will then be able to modify and save the code. Because the working copy is a different version of the script, you need to use the Editor's "Add to chart" button to add that new copy to the chart.

For example, this image shows the Pine Editor, where we selected to view the source code from the "Bollinger Bands" indicator on our chart. Initially, the script is *read-only*, as indicated by the orange warning text:

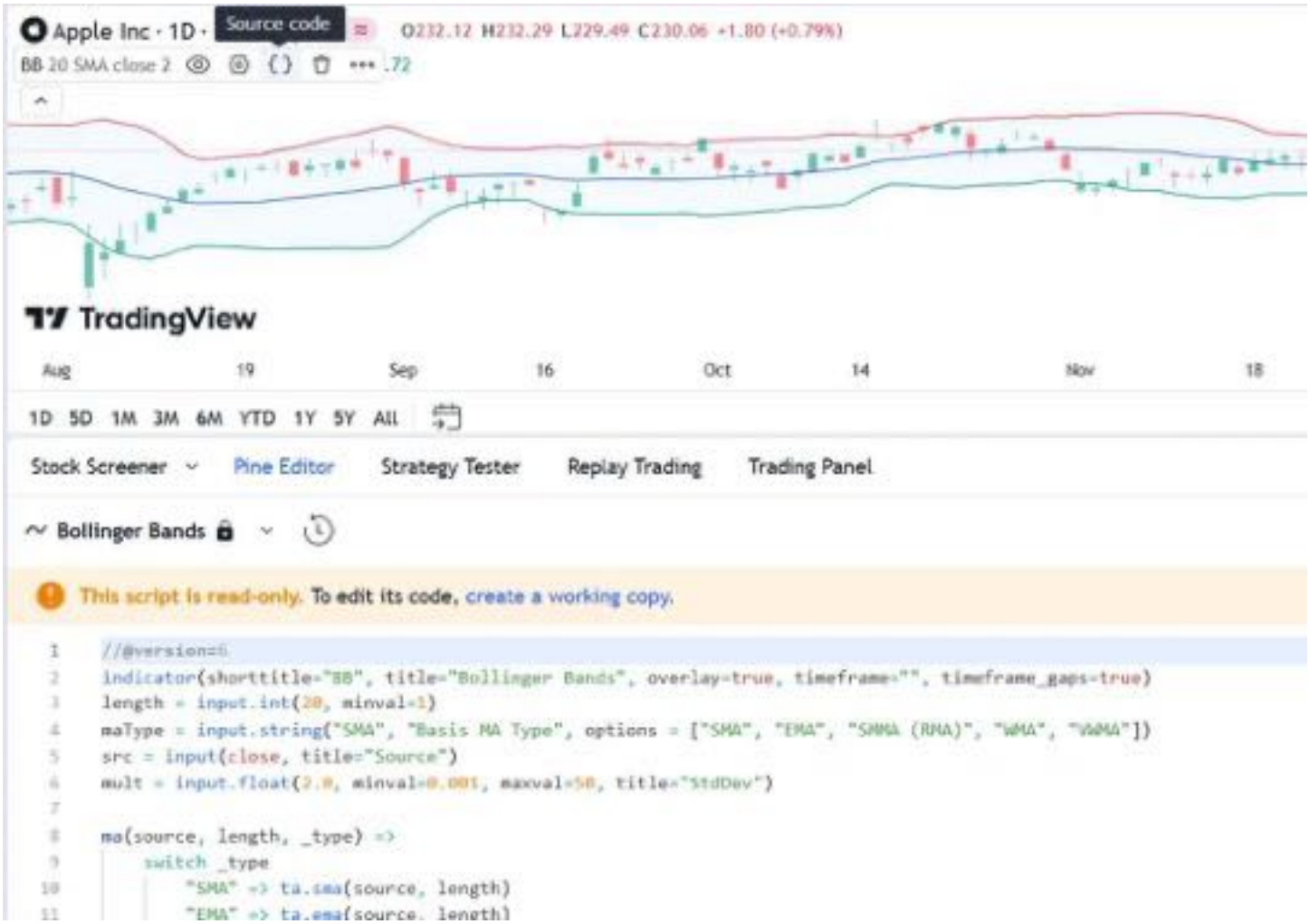

Figure 7: image

You can also open editable versions of the TradingView built-in scripts from the Pine Editor by using the "Create new" > "Built-in..." menu selection:

Writing scripts

We have built Pine Script™ to empower both new and experienced traders to create their own trading tools. Although learning a first programming language, like trading, is rarely **very** easy for anyone, we have designed Pine Script™ so it is relatively easy to learn for first-time programmers, yet powerful enough for knowledgeable programmers to build tools of moderate complexity.

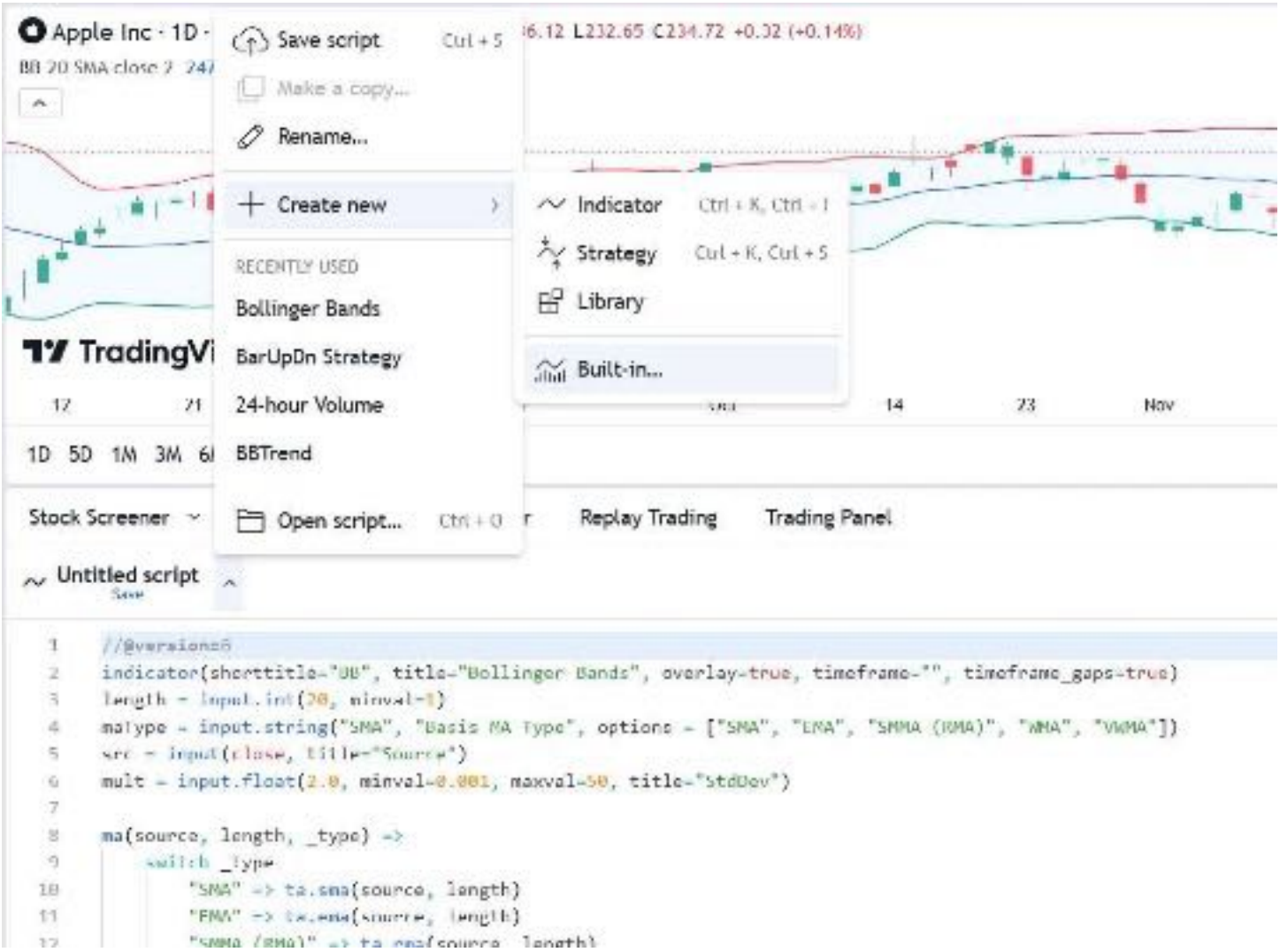

Figure 8: image

Pine Script™ allows you to write three types of scripts:

- **Indicators**, like RSI, MACD, etc.
- **Strategies**, which include logic to issue trading orders and can be backtested and forward-tested.
- **Libraries**, which are used by more advanced programmers to package often-used functions that can be reused by other scripts.

The next step we recommend is to write your first indicator.

[Next

First indicator](#first-indicator) User Manual/Pine Script™ primer/First indicator

First indicator

The Pine Editor

The Pine Editor is where you will be working on your scripts. While you can use any text editor you want to write your Pine scripts, using the Pine Editor has many advantages:

- It highlights your code following Pine Script™ syntax.
- It pops up syntax reminders when you hover over language constructs.
- It provides quick access to the Pine Script™ Reference Manual popup when you select `Ctrl` or `Cmd` and a built-in Pine Script™ construct, and opens the library publication page when doing the same with code imported from libraries.
- It provides an auto-complete feature that you can activate by selecting `Ctrl`+`Space` or `Cmd`+`I`, depending on your operating system.
- It makes the write/compile/run cycle more efficient because saving a new version of a script already loaded on the chart automatically compiles and executes it.

To open the Pine Editor, select the "Pine Editor" tab at the bottom of the TradingView chart.

First version

Let's create our first working Pine script, an implementation of the MACD indicator:

1. Open the Pine Editor's dropdown menu (the arrow at the top-left corner of the Pine Editor pane, beside the script name) and select "Create new/Indicator".
2. Copy the example script code below by clicking the button on the top-right of the code widget.
3. Select all the code already in the editor and replace it with the example code.
4. Save the script by selecting the script name or using the keyboard shortcut `Ctrl`+`S`. Choose a name for the script (e.g., "MACD #1"). The script is saved in TradingView's cloud servers, and is local to your account, meaning only you can see and use this version.
5. Select "Add to chart" in the Pine Editor's menu bar. The MACD indicator appears in a *separate pane* under the chart.

```
//@version=6
indicator("MACD #1")
fast = 12
slow = 26
fastMA = ta.ema(close, fast)
slowMA = ta.ema(close, slow)
macd = fastMA - slowMA
signal = ta.ema(macd, 9)
plot(macd, color = color.blue)
plot(signal, color = color.orange)
```

Our first Pine script is now running on the chart, which should look like this:

Let's look at our script's code, line by line:

Line 1: `//@version=6`

This is a compiler annotation telling the compiler the script uses version 6 of Pine Script™.

Line 2: `indicator("MACD #1")`

Declares this script as an indicator, and defines the title of the script that appears on the chart as "MACD #1".

Figure 9: image

Line 3: `fast = 12`

Defines an integer variable `fast` as the length of the fast moving average.

Line 4: `slow = 26`

Defines an integer variable `slow` as the length of the slow moving average.

Line 5: `fastMA = ta.ema(close, fast)`

Defines the variable `fastMA`, which holds the result of the *EMA* (Exponential Moving Average) calculated on the close series, i.e., the closing price of bars, with a length equal to `fast` (12).

Line 6: `slowMA = ta.ema(close, slow)`

Defines the variable `slowMA`, which holds the result of the EMA calculated on the close series with a length equal to `slow` (26).

Line 7: `macd = fastMA - slowMA`

Defines the variable `macd` as the difference between the two EMAs.

Line 8: `signal = ta.ema(macd, 9)`

Defines the variable `signal` as a smoothed value of `macd` using the EMA algorithm with a length of 9.

Line 9: `plot(macd, color = color.blue)`

Calls the plot() function to output the variable `macd` using a blue line.

Line 10: `plot(signal, color = color.orange)`

Calls the plot() function to output the variable `signal` using an orange line.

Second version

The first version of our script calculated the MACD using multiple steps, but because Pine Script™ is specially designed to write indicators and strategies, built-in functions exist for many common indicators, including one for MACD: ta.macd().

Therefore, we can write a second version of our script that takes advantage of Pine's available built-in functions:

```
//@version=6
indicator("MACD #2")
fastInput = input(12, "Fast length")
slowInput = input(26, "Slow length")
[macdLine, signalLine, histLine] = ta.macd(close, fastInput, slowInput, 9)
plot(macdLine, color = color.blue)
plot(signalLine, color = color.orange)
```

Note that:

- We add inputs so we can change the lengths of the moving averages from the script's settings.
- We now use the ta.macd() built-in function to calculate our MACD directly, which replaces three lines of calculations and makes our code easier to read.

Let's repeat the same process as before to create our new indicator:

1. Open the Pine Editor's dropdown menu (the arrow at the top-left corner of the Pine Editor pane, beside the script name) and select "Create new/Indicator".
2. Copy the example script code below. The button on the top-right of the code widget allows you to copy it with a single click.
3. Select all the code already in the editor and replace it with the example code.
4. Save the script by selecting the script name or using the keyboard shortcut `Ctrl+S`. Choose a name for your script that is different from the previous one (e.g., "MACD #2").
5. Select "Add to chart" in the Pine Editor's menu bar. The "MACD #2" indicator appears in a *separate pane* under the "MACD #1" indicator.

Our second Pine script is now running on the chart. If we double-click on the indicator's name on the chart, it displays the script's "Settings/Inputs" tab, where we can now change the fast and slow lengths used in the MACD calculation:

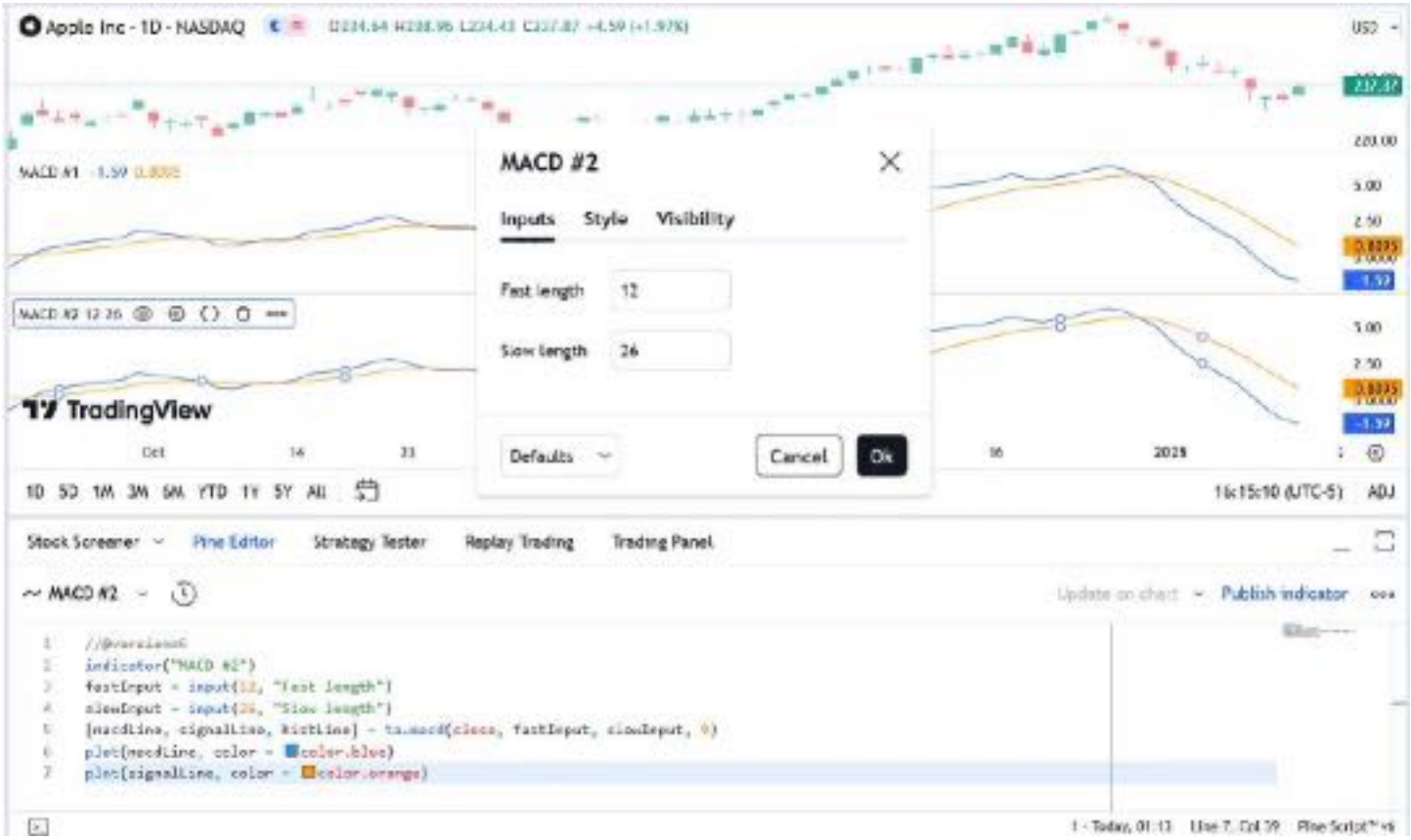

Figure 10: image

Let's look at the lines that have changed in the second version of our script:

Line 2: `indicator("MACD #2")`

We have changed `#1` to `#2` so the second version of our indicator displays a different name on the chart.

Line 3: `fastInput = input(12, "Fast length")`

Instead of assigning a constant value to the variable, we used the input() function so we can change the length value from the script's "Settings/Inputs" tab. The default value is `12` and the input field's label is `"Fast length"`. When we change the value in the "Inputs" tab, the `fastInput` variable updates to contain the new length and the script re-executes on the chart with that new value. Note that, as our Pine Script™ Style guide recommends, we add `Input` to the end of the variable's name to remind us, later in the script, that its value comes from a user input.

Line 4: `slowInput = input(26, "Slow length")`

As with `fastInput` in the previous line, we do the same for the slow length, taking care to use a different variable name, default value, and title string for the input field's label.

Line 5: `[macdLine, signalLine, histLine] = ta.macd(close, fastInput, slowInput, 9)`

This is where we call the ta.macd() built-in function to perform all the first version's calculations in only one line. The function requires four *parameters* (the values enclosed in parentheses after the function name). It returns *three* values, unlike the other functions we've used so far that only returned one. For this reason, we need to enclose the list of three variables receiving the function's result in square brackets (to form a tuple) to the left of the `=` sign. Note that two of the values we pass to the function are the "input" variables containing the fast and slow lengths (`fastInput` and `slowInput`).

Lines 6 and 7: `plot(macdLine, color = color.blue)` and `plot(signalLine, color = color.orange)`

The variable names we are plotting here have changed, but the lines still behave the same as in our first version.

Our second version of the script performs the same calculations as our first, but we've made the indicator more efficient as it now leverages Pine's built-in capabilities and easily supports variable lengths for the MACD calculation. Therefore, we have successfully improved our Pine script.

Next

We now recommend you go to the Next Steps page.

[Previous

First steps](#first-steps)[Next

Next steps](#next-steps) User Manual/Pine Script™ primer/Next steps

Next steps

After your first steps and your first indicator, let us explore a bit more of the Pine Script™ landscape by sharing some pointers to guide you in your journey to learn Pine Script™.

"indicators" vs "strategies"

Pine Script™ strategies are used to backtest on historical data and forward test on open markets. In addition to indicator calculations, they contain `strategy.*()` calls to send trade orders to Pine Script™'s broker emulator, which can then simulate their execution. Strategies display backtest results in the "Strategy Tester" tab at the bottom of the chart, next to the "Pine Editor" tab.

Pine Script™ indicators also contain calculations, but cannot be used in backtesting. Because they do not require the broker emulator, they use less resources and will run faster. It is thus advantageous to use indicators whenever you can.

Both indicators and strategies can run in either overlay mode (over the chart's bars) or pane mode (in a separate section below or above the chart). Both can also plot information in their respective space, and both can generate alert events.

How scripts are executed

A Pine script is **not** like programs in many programming languages that execute once and then stop. In the Pine Script™ *runtime* environment, a script runs in the equivalent of an invisible loop where it is executed once on each bar of whatever chart you are on, from left to right. Chart bars that have already closed when the script executes on them are called *historical bars*. When execution reaches the chart's last bar and the market is open, it is on the *realtime bar*. The script then executes

once every time a price or volume change is detected, and one last time for that realtime bar when it closes. That realtime bar then becomes an *elapsed realtime bar*. Note that when the script executes in realtime, it does not recalculate on all the chart's historical bars on every price/volume update. It has already calculated once on those bars, so it does not need to recalculate them on every chart tick. See the Execution model page for more information.

When a script executes on a historical bar, the close built-in variable holds the value of that bar's close. When a script executes on the realtime bar, close returns the **current** price of the symbol until the bar closes.

Contrary to indicators, strategies normally execute only once on realtime bars, when they close. They can also be configured to execute on each price/volume update if that is what you need. See the page on Strategies for more information, and to understand how strategies calculate differently than indicators.

Time series

The main data structure used in Pine Script™ is called a time series. Time series contain one value for each bar the script executes on, so they continuously expand as the script executes on more bars. Past values of the time series can be referenced using the history-referencing operator: []. `close[1]`, for example, refers to the value of close on the bar preceding the one where the script is executing.

While this indexing mechanism may remind many programmers of arrays, a time series is different and thinking in terms of arrays will be detrimental to understanding this key Pine Script™ concept. A good comprehension of both the execution model and time series is essential in understanding how Pine scripts work. If you have never worked with data organized in time series before, you will need practice to put them to work for you. Once you familiarize yourself with these key concepts, you will discover that by combining the use of time series with our built-in functions specifically designed to handle them efficiently, much can be accomplished in very few lines of code.

Publishing scripts

TradingView is home to a large community of Pine Script™ programmers and millions of traders from all around the world. Once you become proficient enough in Pine Script™, you can choose to share your scripts with other traders. Before doing so, please take the time to learn Pine Script™ well-enough to supply traders with an original and reliable tool. All publicly published scripts are analyzed by our team of moderators and must comply with our Script Publishing Rules, which require them to be original and well-documented.

If you want to use Pine scripts for your own use, simply write them in the Pine Editor and add them to your chart from there; you don't have to publish them to use them. If you want to share your scripts with just a few friends, you can publish them privately and send your friends the browser's link to your private publication. See the page on Publishing for more information.

Getting around the Pine Script™ documentation

While reading code from published scripts is no doubt useful, spending time in our documentation will be necessary to attain any degree of proficiency in Pine Script™. Our two main sources of documentation on Pine Script™ are:

- This Pine Script™ v6 User Manual
- Our Pine Script™ v6 Reference Manual

The Pine Script™ v6 User Manual, which is located on its separate page and in English only.

The Pine Script™ v6 Reference Manual documents what each language construct does. It is an essential tool for all Pine Script™ programmers; your life will be miserable if you try to write scripts of any reasonable complexity without consulting it. It exists in two formats: a separate page linked above, and the popup version. Access the popup version from the Pine Editor by either selecting `Ctrl` or `Cmd` and selecting a keyword, or by using the Editor's "More/Reference Manual…" menu. The Reference Manual is translated into multiple languages.

There are six different versions of Pine Script™ released. Ensure the documentation you use corresponds to the Pine Script™ version you are coding with.

Where to go from here?

This Pine Script™ v6 User Manual contains numerous examples of code used to illustrate the concepts we discuss. By going through it, you will be able to both learn the foundations of Pine Script™ and study the example scripts. Reading about key concepts and trying them out right away with real code is a productive way to learn any programming language. As you hopefully have already done in the First indicator page, copy this documentation's examples in the Editor and play with them. Explore! You won't break anything.

This is how the Pine Script™ v6 User Manual you are reading is organized:

- The Language section explains the main components of the Pine Script™ language and how scripts execute.
- The Concepts section is more task-oriented. It explains how to do things in Pine Script™.
- The Writing section explores tools and tricks that will help you write and publish scripts.
- The FAQ section answers common questions from Pine Script™ programmers.
- The Error messages page documents causes and fixes for the most common runtime and compiler errors.
- The Release Notes page is where you can follow the frequent updates to Pine Script™.
- The Migration guides section explains how to port between different versions of Pine Script™.
- The Where can I get more information page lists other useful Pine Script™-related content, including where to ask questions when you are stuck on code.

We wish you a successful journey with Pine Script™… and trading!

[Previous

First indicator](#first-indicator) User Manual/Language/Execution model

Execution model

The execution model of the Pine Script™ runtime is intimately linked to Pine Script™'s time series and type system. Understanding all three is key to making the most of the power of Pine Script™.

The execution model determines how your script is executed on charts, and thus how the code you write in scripts works. Your code would do nothing were it not for Pine Script™'s runtime, which kicks in after your code has compiled and it is executed on your chart because one of the events triggering the execution of a script has occurred.

When a Pine script is loaded on a chart it executes once on each historical bar using the available OHLCV (open, high, low, close, volume) values for each bar. Once the script's execution reaches the rightmost bar in the dataset, if trading is currently active on the chart's symbol, then Pine Script™ *indicators* will execute once every time an *update* occurs, i.e., price or volume changes. Pine Script™ *strategies* will by default only execute when the rightmost bar closes, but they can also be configured to execute on every update, like indicators do.

All symbol/timeframe pairs have a dataset comprising a limited number of bars. When you scroll a chart to the left to see the dataset's earlier bars, the corresponding bars are loaded on the chart. The loading process stops when there are no more bars for that particular symbol/timeframe pair or the maximum number of bars your account type permits has been loaded. You can scroll the chart to the left until the very first bar of the dataset, which has an index value of 0 (see bar_index).

When the script first runs on a chart, all bars in a dataset are *historical bars*, except the rightmost one if a trading session is active. When trading is active on the rightmost bar, it is called the *realtime bar*. The realtime bar updates when a price or volume change is detected. When the realtime bar closes, it becomes an *elapsed realtime bar* and a new realtime bar opens.

Calculation based on historical bars

Let's take a simple script and follow its execution on historical bars:

```
//@version=6
indicator("My Script", overlay = true)
src = close
a = ta.sma(src, 5)
b = ta.sma(src, 50)
c = ta.cross(a, b)
plot(a, color = color.blue)
plot(b, color = color.black)
plotshape(c, color = color.red)
```

On historical bars, a script executes at the equivalent of the bar's close, when the OHLCV values are all known for that bar. Prior to execution of the script on a bar, the built-in variables such as `open`, `high`, `low`, `close`, `volume` and `time` are set to values corresponding to those from that bar. A script executes **once per historical bar**.

Our example script is first executed on the very first bar of the dataset at index 0. Each statement is executed using the values for the current bar. Accordingly, on the first bar of the dataset, the following statement:

```
src = close
```

initializes the variable `src` with the `close` value for that first bar, and each of the next lines is executed in turn. Because the script only executes once for each historical bar, the script will always calculate using the same `close` value for a specific historical bar.

The execution of each line in the script produces calculations which in turn generate the indicator's output values, which can then be plotted on the chart. Our example uses the `plot` and `plotshape` calls at the end of the script to output some values. In the case of a strategy, the outcome of the calculations can be used to plot values or dictate the orders to be placed.

After execution and plotting on the first bar, the script is executed on the dataset's second bar, which has an index of 1. The process then repeats until all historical bars in the dataset are processed and the script reaches the rightmost bar on the chart.

Figure 11: image

Calculation based on realtime bars

The behavior of a Pine script on the realtime bar is very different than on historical bars. Recall that the realtime bar is the rightmost bar on the chart when trading is active on the chart's symbol. Also, recall that strategies can behave in two different ways in the realtime bar. By default, they only execute when the realtime bar closes, but the `calc_on_every_tick` parameter of the `strategy` declaration statement can be set to true to modify the strategy's behavior so that it executes each time the realtime bar updates, as indicators do. The behavior described here for indicators will thus only apply to strategies using `calc_on_every_tick=true`.

The most important difference between execution of scripts on historical and realtime bars is that while they execute only once on historical bars, scripts execute every time an update occurs during a realtime bar. This entails that built-in variables such as `high`, `low` and `close` which never change on a historical bar, **can** change at each of a script's iteration in the realtime bar. Changes in the built-in variables used in the script's calculations will, in turn, induce changes in the results of those calculations. This is required for the script to follow the realtime price action. As a result, the same script may produce different results every time it executes during the realtime bar.

Note: In the realtime bar, the `close` variable always represents the **current price**. Similarly, the `high` and `low` built-in variables represent the highest high and lowest low reached since the realtime bar's beginning. Pine Script™'s built-in variables will only represent the realtime bar's final values on the bar's last update.

Let's follow our script example in the realtime bar.

When the script arrives on the realtime bar it executes a first time. It uses the current values of the built-in variables to produce a set of results and plots them if required. Before the script executes another time when the next update happens, its user-defined variables are reset to a known state corresponding to that of the last *commit* at the close of the previous

bar. If no commit was made on the variables because they are initialized every bar, then they are reinitialized. In both cases their last calculated state is lost. The state of plotted labels and lines is also reset. This resetting of the script's user-defined variables and drawings prior to each new iteration of the script in the realtime bar is called *rollback*. Its effect is to reset the script to the same known state it was in when the realtime bar opened, so calculations in the realtime bar are always performed from a clean state.

The constant recalculation of a script's values as price or volume changes in the realtime bar can lead to a situation where variable `c` in our example becomes true because a cross has occurred, and so the red marker plotted by the script's last line would appear on the chart. If on the next price update the price has moved in such a way that the `close` value no longer produces calculations making `c` true because there is no longer a cross, then the marker previously plotted will disappear.

When the realtime bar closes, the script executes a last time. As usual, variables are rolled back prior to execution. However, since this iteration is the last one on the realtime bar, variables are committed to their final values for the bar when calculations are completed.

To summarize the realtime bar process:

- A script executes **at the open of the realtime bar and then once per update**.
- Variables are rolled back **before every realtime update**.
- Variables are committed **once at the closing bar update**.

Events triggering the execution of a script

A script executes across *all* available bars in a dataset when one of the following conditions occurs:

- The user adds the script to their chart from the Pine Editor or the "Indicators, Metrics & Strategies" menu.
- The user saves a new version of the source code while the script is on the chart.
- The chart uses a new *timeframe*.
- The chart uses a dataset with a new *ticker identifier*. Several user actions affect a chart's ticker ID, such as selecting a symbol from the "Symbol search" menu, switching between standard and non-standard chart types, activating Bar Replay mode, and toggling data modifications from the chart's settings.
- The user changes any input available in the script's "Settings/Inputs" tab.
- The user updates a strategy property from the "Settings/Properties" tab.
- The script uses the chart.left_visible_bar_time or chart.right_visible_bar_time variable in its calculations or logic, and the user changes the variable's value by scrolling or zooming on the chart.
- The system detects a browser refresh event.
- The user opens or closes the Pine Logs pane.
- The user activates or deactivates the Pine Profiler.

Either of the following triggers script executions on a *realtime bar* while the chart symbol's session is active:

- One of the conditions above occurs, causing the script to execute across the *entire* dataset up to the latest bar.
- The realtime bar's values change after new price or volume data becomes available. The script performs a *new execution* on that bar to use the latest information in its calculations.

Note that when a chart is left untouched when the market is active, a succession of realtime bars which have been opened and then closed will trail the current realtime bar. While these *elapsed realtime bars* will have been *confirmed* because their variables have all been committed, the script will not yet have executed on them in their *historical* state, since they did not exist when the script was last run on the chart's dataset.

When an event triggers the execution of the script on the chart and causes it to run on those bars which have now become historical bars, the script's calculation can sometimes vary from what they were when calculated on the last closing update of the same bars when they were realtime bars. This can be caused by slight variations between the OHLCV values saved at the close of realtime bars and those fetched from data feeds when the same bars have become historical bars. This behavior is one of the possible causes of *repainting*.

More information

- The built-in `barstate.*` variables provide information on the type of bar or the event where the script is executing. The page where they are documented also contains a script that allows you to visualize the difference between elapsed realtime and historical bars, for example.
- The Strategies page explains the details of strategy calculations, which are not identical to those of indicators.

Historical values of functions

Every function call in Pine leaves a trail of historical values that a script can access on subsequent bars using the [] operator. The historical series of functions depend on successive calls to record the output on every bar. When a script does not call functions on each bar, it can produce an inconsistent history that may impact calculations and results, namely when it depends on the continuity of their historical series to operate as expected. The compiler warns users in these cases to make them aware that the values from a function, whether built-in or user-defined, might be misleading.

To demonstrate, let's write a script that calculates the index of the current bar and outputs that value on every second bar. In the following script, we've defined a `calcBarIndex()` function that adds 1 to the previous value of its internal `index` variable on every bar. The script calls the function on each bar that the `condition` returns `true` on (every other bar) to update the `customIndex` value. It plots this value alongside the built-in `bar_index` to validate the output:

Figure 12: image

```
//@version=6
indicator("My script")

//@function Calculates the index of the current bar by adding 1 to its own value from the previous bar.
// The first bar will have an index of 0.
calcBarIndex() =>
    int index = na
    index := nz(index[1], replacement = -1) + 1

//@variable Returns `true` on every other bar.
condition = bar_index % 2 == 0

int customIndex = na

// Call `calcBarIndex()` when the `condition` is `true`. This prompts the compiler to raise a warning.
if condition
    customIndex := calcBarIndex()

plot(bar_index,   "Bar index",    color = color.green)
plot(customIndex, "Custom index", color = color.red, style = plot.style_cross)
```

Note that:

- The nz() function replaces na values with a specified `replacement` value (0 by default). On the first bar of the script,

when the `index` series has no history, the na value is replaced with -1 before adding 1 to return an initial value of 0.

Upon inspecting the chart, we see that the two plots differ wildly. The reason for this behavior is that the script called `calcBarIndex()` within the scope of an if structure on every other bar, resulting in a historical output inconsistent with the `bar_index` series. When calling the function once every two bars, internally referencing the previous value of `index` gets the value from two bars ago, i.e., the last bar the function executed on. This behavior results in a `customIndex` value of half that of the built-in `bar_index`.

To align the `calcBarIndex()` output with the `bar_index`, we can move the function call to the script's global scope. That way, the function will execute on every bar, allowing its entire history to be recorded and referenced rather than only the results from every other bar. In the code below, we've defined a `globalScopeBarIndex` variable in the global scope and assigned it to the return from `calcBarIndex()` rather than calling the function locally. The script sets the `customIndex` to the value of `globalScopeBarIndex` on the occurrence of the `condition`:

Figure 13: image

```
//@version=6
indicator("My script")

//@function Calculates the index of the current bar by adding 1 to its own value from the previous bar.
// The first bar will have an index of 0.
calcBarIndex() =>
    int index = na
    index := nz(index[1], replacement = -1) + 1

//@variable Returns `true` on every second bar.
condition = bar_index % 2 == 0

globalScopeBarIndex = calcBarIndex()
int customIndex = na

// Assign `customIndex` to `globalScopeBarIndex` when the `condition` is `true`. This won't produce a warning.
if condition
    customIndex := globalScopeBarIndex

plot(bar_index,   "Bar index",    color = color.green)
plot(customIndex, "Custom index", color = color.red, style = plot.style_cross)
```

This behavior can also radically impact built-in functions that reference history internally. For example, the ta.sma() function

references its past values "under the hood". If a script calls this function conditionally rather than on every bar, the values within the calculation can change significantly. We can ensure calculation consistency by assigning ta.sma() to a variable in the global scope and referencing that variable's history as needed.

The following example calculates three SMA series: `controlSMA`, `localSMA`, and `globalSMA`. The script calculates `controlSMA` in the global scope and `localSMA` within the local scope of an if structure. Within the if structure, it also updates the value of `globalSMA` using the `controlSMA` value. As we can see, the values from the `globalSMA` and `controlSMA` series align, whereas the `localSMA` series diverges from the other two because it uses an incomplete history, which affects its calculations:

Figure 14: image

```
//@version=6
indicator("My script")

//@variable Returns `true` on every second bar.
condition = bar_index % 2 == 0

controlSMA = ta.sma(close, 20)
float globalSMA = na
float localSMA  = na

// Update `globalSMA` and `localSMA` when `condition` is `true`.
if condition
    globalSMA := controlSMA        // No warning.
    localSMA  := ta.sma(close, 20) // Raises warning. This function depends on its history to work as intended

plot(controlSMA, "Control SMA", color = color.green)
plot(globalSMA,  "Global SMA",  color = color.blue, style = plot.style_cross)
plot(localSMA,   "Local SMA",   color = color.red,  style = plot.style_cross)
```

Why this behavior?

This behavior is required because forcing the execution of functions on each bar would lead to unexpected results in those functions that produce side effects, i.e., the ones that do something aside from returning the value. For example, the label.new() function creates a label on the chart, so forcing it to be called on every bar even when it is inside of an if structure would create labels where they should not logically appear.

Exceptions

Not all built-in functions use their previous values in their calculations, meaning not all require execution on every bar. For example, math.max() compares all arguments passed into it to return the highest value. Such functions that do not interact with their history in any way do not require special treatment.

If the usage of a function within a conditional block does not cause a compiler warning, it's safe to use without impacting calculations. Otherwise, move the function call to the global scope to force consistent execution. When keeping a function call within a conditional block despite the warning, ensure the output is correct at the very least to avoid unexpected results.

[Next

Time series](#time-series) User Manual/Language/Time series

Time series

Much of the power of Pine Script™ stems from the fact that it is designed to process *time series* efficiently. Time series are not a qualified type; they are the fundamental structure Pine Script™ uses to store the successive values of a variable over time, where each value is tethered to a point in time. Since charts are composed of bars, each representing a particular point in time, time series are the ideal data structure to work with values that may change with time.

The notion of time series is intimately linked to Pine's execution model and type system concepts. Understanding all three is key to making the most of the power of Pine Script™.

Take the built-in open variable, which contains the "open" price of each bar in the dataset, the *dataset* being all the bars on any given chart. If your script is running on a 5min chart, then each value in the open time series is the "open" price of the consecutive 5min chart bars. When your script refers to open, it is referring to the "open" price of the bar the script is executing on. To refer to past values in a time series, we use the [] history-referencing operator. When a script is executing on a given bar, `open[1]` refers to the value of the open time series on the previous bar.

While time series may remind programmers of arrays, they are totally different. Pine Script™ does have an array data structure, but it is a completely different concept than a time series.

Time series in Pine Script™, combined with its special runtime engine and built-in functions, make it easy to compute calculations across bars without needing a for loop. For example, we can calculate the cumulative total of close values by using the ta.cum() function. Although `ta.cum(close)` appears rather static in a script, it is in fact executed on *each* bar, so its value becomes increasingly larger as it continues adding the close values of each new bar. When the script reaches the rightmost bar of the chart, `ta.cum(close)` returns the sum of the close values from *all* bars on the chart.

Similarly, Pine scripts can calculate the mean of the difference between the last 14 high and low values using the ta.sma() function (`ta.sma(high - low, 14)`), or the distance in bars since the last time the chart made five consecutive higher highs by combining ta.barssince() and ta.rising() calls (`ta.barssince(ta.rising(high, 5))`).

The result of function calls on successive bars leaves a succession of values in a time series that scripts can reference using the [] history-referencing operator. This can be useful, for example, when testing the close of the current bar for a breach of the highest high in the last 10 bars, but excluding the current bar, which we could write as `breach = close > ta.highest(close, 10)[1]`. The same statement could also be written as `breach = close > ta.highest(close[1], 10)`.

The same logic of executing a single code statement on all bars applies to function calls such as plot(open), which will repeat on each bar to successively plot the values of the open time series on the chart.

Do not confuse "time series" with the "series" qualifier. The *time series* concept explains how consecutive values of variables are stored in Pine Script™; the "series" qualifier denotes variables whose values can change from bar to bar. Consider, for example, the timeframe.period built-in variable, which has the "simple" qualifier and "string" type, meaning it is of the "simple string" qualified type. The "simple" qualifier entails that the variable's value is established on bar zero (the first bar where the script executes) and does not change during the script's execution on any of the chart's bars. The variable's value is the chart's timeframe in "string" format, so `"1D"` for a daily chart, for example. Even though its value cannot change during the script's execution, it would be syntactically correct in Pine Script™ (though not very useful) to refer to its value 10 bars ago using `timeframe.period[10]`. This is possible because the successive values of timeframe.period for each bar are stored in a time series, even though all the values in that particular time series are the same. Note, however, that when the [] operator is used to access past values of a variable, it yields a "series" qualified value, even when the variable without an offset uses a different qualifier, such as "simple" in the case of timeframe.period.

Programmers can define complex calculations using little code by using Pine's syntax and execution model to efficiently handle time series.

[Previous

Execution model](#execution-model)[Next

Script structure](#script-structure) User Manual/Language/Script structure

Script structure

A Pine script follows this general structure:

```
<version><declaration_statement><code>
```

Version

A compiler annotation in the following form tells the compiler which of the versions of Pine Script™ the script is written in:

```
//@version=6
```

- The version number is a number from 1 to 6.
- The compiler annotation is not mandatory. When omitted, version 1 is assumed. It is strongly recommended to always use the latest version of the language.
- While it is synctactically correct to place the version compiler annotation anywhere in the script, it is much more useful to readers when it appears at the top of the script.

Notable changes to the current version of Pine Script™ are documented in the Release notes.

Declaration statement

All Pine scripts must contain one declaration statement, which is a call to one of these functions:

- indicator()
- strategy()
- library()

The declaration statement:

- Identifies the type of the script, which in turn dictates which content is allowed in it, and how it can be used and executed.
- Sets key properties of the script such as its name, where it will appear when it is added to a chart, the precision and format of the values it displays, and certain values that govern its runtime behavior, such as the maximum number of drawing objects it will display on the chart. With strategies, the properties include parameters that control backtesting, such as initial capital, commission, slippage, etc.

Each script type has distinct basic requirements. Scripts that do not meet these criteria cause a compilation error:

- Indicators must call at least one function that creates a script output, such as plot(), plotshape(), barcolor(), line.new(), log.info(), alert(), etc.
- Strategies must call at least one order placement command or other output function.
- Libraries must export at least one user-defined function, method, type, or enum.

Code

Lines in a script that are not comments or compiler annotations are *statements*, which implement the script's algorithm. A statement can be one of these:

- variable declaration
- variable reassignment
- function declaration
- built-in function call, user-defined function call or a library function call
- if, for, while, switch, type, or enum*structure*.

Statements can be arranged in multiple ways:

- Some statements can be expressed in one line, like most variable declarations, lines containing only a function call or single-line function declarations. Lines can also be wrapped (continued on multiple lines). Multiple one-line statements can be concatenated on a single line by using the comma as a separator.
- Others statements such as structures or multi-line function declarations always require multiple lines because they require a *local block*. A local block must be indented by a tab or four spaces. Each local block defines a distinct *local scope*.
- Statements in the *global scope* of the script (i.e., which are not part of local blocks) cannot begin with white space (a space or a tab). Their first character must also be the line's first character. Lines beginning in a line's first position become by definition part of the script's *global scope*.

A simple valid Pine Script™ indicator can be generated in the Pine Script™ Editor by using the "Open" button and choosing "New blank indicator":

```
//@version=6
indicator("My Script")
plot(close)
```

This indicator includes three local blocks, one in the `barIsUp()` function declaration, and two in the variable declaration using an if structure:

```
//@version=6

indicator("", "", true)    // Declaration statement (global scope)

barIsUp() =>    // Function declaration (global scope)
    close > open    // Local block (local scope)

plotColor = if barIsUp()  // Variable declaration (global scope)
    color.green     // Local block (local scope)
else
    color.red       // Local block (local scope)

bgcolor(color.new(plotColor, 70))   // Call to a built-in function  (global scope)
```

You can bring up a simple Pine Script™ strategy by selecting "New blank strategy" instead:

```
//@version=6
strategy("My Strategy", overlay=true, margin_long=100, margin_short=100)

longCondition = ta.crossover(ta.sma(close, 14), ta.sma(close, 28))
if (longCondition)
    strategy.entry("My Long Entry Id", strategy.long)

shortCondition = ta.crossunder(ta.sma(close, 14), ta.sma(close, 28))
if (shortCondition)
    strategy.entry("My Short Entry Id", strategy.short)
```

Comments

Double slashes (`//`) define comments in Pine Script™. Comments can begin anywhere on the line. They can also follow Pine Script™ code on the same line:

```
//@version=6
indicator("")
// This line is a comment
a = close // This is also a comment
plot(a)
```

The Pine Editor has a keyboard shortcut to comment/uncomment lines: `ctrl` + `/`. You can use it on multiple lines by highlighting them first.

Line wrapping

Long lines can be split on multiple lines, or "wrapped". Wrapped lines must be indented with any number of spaces, provided it's not a multiple of four (those boundaries are used to indent local blocks):

```
a = open + high + low + close
```

may be wrapped as:

```
a = open +
      high +
          low +
             close
```

A long plot() call may be wrapped as:

```
plot(ta.correlation(src, ovr, length),
   color = color.new(color.purple, 40),
   style = plot.style_area,
   trackprice = true)
```

Expressions inside *local* code blocks can also use line wrapping. However, because the code in a local block requires indentation by four spaces or a tab, we recommend using a *larger* indentation that is not a multiple of four spaces for the wrapped lines inside the block. For example:

```
upDown(float s) =>
    var int ud = 0
    bool isEqual   = s == s[1]
    bool isGrowing = s > s[1]
    ud := isEqual ?
           0 :
           isGrowing ?
               (ud <= 0 ?
                     1 :
                   ud + 1) :
               (ud >= 0 ?
                   -1 :
                   ud - 1)
```

Wrapped lines can also include comments:

```
//@version=6
indicator("")
c = open > close ? color.red :
  high > high[1] ? color.lime : // A comment
  low < low[1] ? color.blue : color.black
bgcolor(c)
```

Compiler annotations

Compiler annotations are comments that issue special instructions for a script:

- `//@version=` specifies the PineScript™ version that the compiler will use. The number in this annotation should not be confused with the script's version number, which updates on every saved change to the code.
- `//@description` sets a custom description for scripts that use the library() declaration statement.
- `//@function`, `//@param` and `//@returns` add custom descriptions for a user-defined function or method, its parameters, and its result when placed above the function declaration.
- `//@type` adds a custom description for a user-defined type (UDT) when placed above the type declaration.
- `//@enum` adds a custom description for an enum types when placed above the enum declaration.
- `//@field` adds a custom description for the field of a user-defined type (UDT) or an enum types when placed above the type or enum declaration.
- `//@variable` adds a custom description for a variable when placed above its declaration.
- `//@strategy_alert_message` provides a default message for strategy scripts to pre-fill the "Message" field in the alert creation dialog.

The Pine Editor also features two specialized annotations, `//#region` and `//#endregion`, that create *collapsible* code regions. Clicking the dropdown arrow next to a `//#region` line collapses all the code between that line and the nearest `//#endregion` annotation below it.

This example draws a triangle using three interactively selected points on the chart. The script illustrates how one can use compiler and Editor annotations to document code and make it easier to navigate:

Figure 15: image

```
//@version=6
indicator("Triangle", "", true)

//#region -------------------- Constants and inputs

int   TIME_DEFAULT  = 0
float PRICE_DEFAULT = 0.0

x1Input = input.time(TIME_DEFAULT,   "Point 1", inline = "1", confirm = true)
y1Input = input.price(PRICE_DEFAULT, "",        inline = "1", tooltip = "Pick point 1", confirm = true)
x2Input = input.time(TIME_DEFAULT,   "Point 2", inline = "2", confirm = true)
y2Input = input.price(PRICE_DEFAULT, "",        inline = "2", tooltip = "Pick point 2", confirm = true)
x3Input = input.time(TIME_DEFAULT,   "Point 3", inline = "3", confirm = true)
y3Input = input.price(PRICE_DEFAULT, "",        inline = "3", tooltip = "Pick point 3", confirm = true)
//#endregion

//#region -------------------- Types and functions

// @type            Used to represent the coordinates and color to draw a triangle.
// @field time1     Time of first point.
// @field time2     Time of second point.
// @field time3     Time of third point.
// @field price1    Price of first point.
// @field price2    Price of second point.
// @field price3    Price of third point.
// @field lineColor Color to be used to draw the triangle lines.
type Triangle
    int   time1
    int   time2
    int   time3
    float price1
```

```
    float price2
    float price3
    color lineColor

//@function Draws a triangle using the coordinates of the `t` object.
//@param t  (Triangle) Object representing the triangle to be drawn.
//@returns  The ID of the last line drawn.
drawTriangle(Triangle t) =>
    line.new(t.time1, t.price1, t.time2, t.price2, xloc = xloc.bar_time, color = t.lineColor)
    line.new(t.time2, t.price2, t.time3, t.price3, xloc = xloc.bar_time, color = t.lineColor)
    line.new(t.time1, t.price1, t.time3, t.price3, xloc = xloc.bar_time, color = t.lineColor)
//#endregion

//#region -------------------- Calculations

// Draw the triangle only once on the last historical bar.
if barstate.islastconfirmedhistory
    //@variable Used to hold the Triangle object to be drawn.
    Triangle triangle = Triangle.new()

    triangle.time1  := x1Input
    triangle.time2  := x2Input
    triangle.time3  := x3Input
    triangle.price1 := y1Input
    triangle.price2 := y2Input
    triangle.price3 := y3Input
    triangle.lineColor := color.purple

    drawTriangle(triangle)
//#endregion
```

[Previous

Time series](#time-series)[Next

Identifiers](#identifiers) User Manual/Language/Identifiers

Identifiers

Identifiers are names used for user-defined variables and functions:

- They must begin with an uppercase (`A-Z`) or lowercase (`a-z`) letter, or an underscore (`_`).
- The next characters can be letters, underscores or digits (`0-9`).
- They are case-sensitive.

Here are some examples:

```
myVar
_myVar
my123Var
functionName
MAX_LEN
max_len
maxLen
3barsDown  // NOT VALID!
```

The Pine Script™ Style Guide recommends using uppercase SNAKE_CASE for constants, and camelCase for other identifiers:

```
GREEN_COLOR = #4CAF50
MAX_LOOKBACK = 100
int fastLength = 7
// Returns 1 if the argument is `true`, 0 if it is `false` or `na`.
```

```
zeroOne(boolValue) => boolValue ? 1 : 0
```

[Previous

Script structure](#script-structure)[Next

Operators](#operators) User Manual/Language/Operators

Operators

Introduction

Some operators are used to build *expressions* returning a result:

- Arithmetic operators
- Comparison operators
- Logical operators
- The ?: ternary operator
- The [] history-referencing operator

Other operators are used to assign values to variables:

- `=` is used to assign a value to a variable, **but only when you declare the variable** (the first time you use it)
- `:=` is used to assign a value to a **previously declared variable**. The following operators can also be used in such a way: `+=`, `-=`, `*=`, `/=`, `%=`

As is explained in the Type system page, *qualifiers* and *types* play a critical role in determining the type of results that expressions yield. This, in turn, has an impact on how and with what functions you will be allowed to use those results. Expressions always return a value with the strongest qualifier used in the expression, e.g., if you multiply an "input int" with a "series int", the expression will produce a "series int" result, which you will not be able to use as the argument to `length` in ta.ema().

This script will produce a compilation error:

```
//@version=6
indicator("")
lenInput = input.int(14, "Length")
factor = year > 2020 ? 3 : 1
adjustedLength = lenInput * factor
ma = ta.ema(close, adjustedLength)  // Compilation error!
plot(ma)
```

The compiler will complain: *Cannot call 'ta.ema' with argument 'length'='adjustedLength'. An argument of 'series int' type was used but a 'simple int' is expected;.* This is happening because `lenInput` is an "input int" but `factor` is a "series int" (it can only be determined by looking at the value of year on each bar). The `adjustedLength` variable is thus assigned a "series int" value. Our problem is that the Reference Manual entry for ta.ema() tells us that its `length` parameter requires a "simple" value, which is a weaker qualifier than "series", so a "series int" value is not allowed.

The solution to our conundrum requires:

- Using another moving average function that supports a "series int" length, such as ta.sma(), or
- Not using a calculation producing a "series int" value for our length.

Arithmetic operators

There are five arithmetic operators in Pine Script™:

OperatorMeaning+Addition and string concatenation-Subtraction*Multiplication/Division%Modulo (remainder after division)The arithmetic operators above are all *binary*, meaning they need two *operands* — or values — to work on, as in the example operation `1 + 2`. The `+` and `-` can also be *unary* operators, which means they work on one operand, as in the example values `-1` or `+1`.

If both operands are numbers but at least one of these is of float type, the result will also be a float. If both operands are of int type, the result will also be an int. If at least one operand is na, the result is also na.

Note that when using the division operator with "int" operands, if the two "int" values are not evenly divisible, the result of the division is always a number with a fractional value, e.g., `5/2 = 2.5`. To discard the fractional remainder, wrap the division with the int() function, or round the result using math.round(), math.floor(), or math.ceil().

The `+` operator also serves as the concatenation operator for strings. `"EUR"+"USD"` yields the `"EURUSD"` string.

The `%` operator calculates the modulo by rounding down the quotient to the lowest possible value. Here is an easy example that helps illustrate how the modulo is calculated behind the scenes:

```
//@version=6
indicator("Modulo function")
modulo(series int a, series int b) =>
    a - b * math.floor(nz(a/b))
plot(modulo(-1, 100))
```

Comparison operators

There are six comparison operators in Pine Script™:

OperatorMeaning<Less Than<=Less Than or Equal To!=Not Equal==Equal>Greater Than>=Greater Than or Equal ToComparison operations are binary, and return a result of type "bool", i.e., true or false. The `==` equal and `!=` not equal operators can work with operands of any fundamental type, such as colors and strings, while the other comparison operators are only applicable to numerical values. Therefore, `"a" != "b"` is a valid comparison, but `"a" > "b"` is invalid.

Examples:

```
1 > 2  // false
1 != 1 // false
close >= open  // Depends on values of `close` and `open`
```

Logical operators

There are three logical operators in Pine Script™:

OperatorMeaning`not`Negation`and`Logical Conjunction`or`Logical DisjunctionThe operator `not` is unary. When applied to a `true`, operand the result will be `false`, and vice versa.

`and` operator truth table:

aba and btruetruetruetruefalsefalsefalsetruefalsefalsefalsefalse`or` operator truth table:

aba or btruetruetruetruefalsetruefalsetruetruefalsefalsefalse## `?:` ternary operator

The ?: ternary operator is used to create expressions of the form:

```
condition ? valueWhenConditionIsTrue : valueWhenConditionIsFalse
```

The ternary operator returns a result that depends on the value of `condition`. If it is `true`, then it returns `valueWhenConditionIsTrue`. Otherwise, if `condition` is `false`, then it returns `valueWhenConditionIsFalse`.

A combination of ternary expressions can be used to achieve the same effect as a switch structure, e.g.:

```
timeframe.isintraday ? color.red : timeframe.isdaily ? color.green : timeframe.ismonthly ? color.blue : na
```

The example is calculated from left to right:

- If timeframe.isintraday is `true`, then `color.red` is returned. If it is `false`, then timeframe.isdaily is evaluated.
- If timeframe.isdaily is `true`, then `color.green` is returned. If it is `false`, then timeframe.ismonthly is evaluated.
- If timeframe.ismonthly is `true`, then `color.blue` is returned, otherwise na is returned.

Note that the return values on each side of the `:` are expressions — not local blocks, so they will not affect the limit of 500 local blocks per scope.

`[ ]` history-referencing operator

It is possible to refer to past values of time series using the [] history-referencing operator. Past values are values a variable had on bars preceding the bar where the script is currently executing — the *current bar*. See the Execution model page for more information about the way scripts are executed on bars.

The [] operator is used after a variable, expression or function call. The value used inside the square brackets of the operator is the offset in the past we want to refer to. To refer to the value of the volume built-in variable two bars away from the current bar, one would use `volume[2]`.

Because series grow dynamically, as the script calculates on successive bars, a constant historical offset refers to different bars. Let's see how the value returned by the same offset is dynamic, and why series are very different from arrays. In Pine Script™, the close variable, or `close[0]` which is equivalent, holds the value of the current bar's "close". If your code is now executing on the **third** bar of the *dataset* (the set of all bars on your chart), `close` will contain the price at the close of that bar, `close[1]` will contain the price at the close of the preceding bar (the dataset's second bar), and `close[2]`, the first bar. `close[3]` will return na because no bar exists in that position, and thus its value is *not available*.

When the same code is executed on the next bar, the **fourth** in the dataset, `close` will now contain the closing price of that bar, and the same `close[1]` used in your code will now refer to the "close" of the third bar in the dataset. The close of the first bar in the dataset will now be `close[3]`, and this time `close[4]` will return na.

In the Pine Script™ runtime environment, as your code is executed once for each historical bar in the dataset, starting from the left of the chart, Pine Script™ is adding a new element in the series at index 0 and pushing the pre-existing elements in the series one index further away. Arrays, in comparison, can have constant or variable sizes, and their content or indexing structure is not modified by the runtime environment. Pine Script™ series are thus very different from arrays and only share familiarity with them through their indexing syntax.

When the market for the chart's symbol is open and the script is executing on the chart's last bar, the *realtime bar*, close returns the value of the current price. It will only contain the actual closing price of the realtime bar the last time the script is executed on that bar, when it closes.

Pine Script™ has a variable that contains the number of the bar the script is executing on: bar_index. On the first bar, bar_index is equal to 0 and it increases by 1 on each successive bar the script executes on. On the last bar, bar_index is equal to the number of bars in the dataset minus one.

There is another important consideration to keep in mind when using the `[]` operator in Pine Script™. We have seen cases when a history reference may return the na value. na represents a value which is not a number and using it in any expression will produce a result that is also na (similar to NaN). Such cases often happen during the script's calculations in the early bars of the dataset, but can also occur in later bars under certain conditions. If your code does not explicitly handle these special cases using the na() and nz() functions, na values can introduce invalid results in your script's calculations that can affect calculations all the way to the realtime bar.

These are all valid uses of the [] operator:

```
high[10]
ta.sma(close, 10)[1]
ta.highest(high, 10)[20]
close > nz(close[1], open)
```

Note that the [] operator can only be used once on the same value. This is not allowed:

```
close[1][2] // Error: incorrect use of [] operator
```

Operator precedence

The order of calculations is determined by the operators' precedence. Operators with greater precedence are calculated first. Below is a list of operators sorted by decreasing precedence:

PrecedenceOperator9`[]`8unary `+`, unary `-`, `not`7`*`, `/`, `%`6`+`, `-`5`>`, `<`, `>=`, `<=`4`==`, `!=`3`and`2`or`1`?:`If in one expression there are several operators with the same precedence, then they are calculated left to right.

If the expression must be calculated in a different order than precedence would dictate, then parts of the expression can be grouped together with parentheses.

= assignment operator

The `=` operator is used to assign a variable when it is initialized — or declared —, i.e., the first time you use it. It says *this is a new variable that I will be using, and I want it to start on each bar with this value.*

These are all valid variable declarations:

```
i = 1
MS_IN_ONE_MINUTE = 1000 * 60
```

```
showPlotInput = input.bool(true, "Show plots")
pHi = ta.pivothigh(5, 5)
plotColor = color.green
```

See the Variable declarations page for more information on how to declare variables.

:= reassignment operator

The `:=` is used to *reassign* a value to an existing variable. It says *use this variable that was declared earlier in my script, and give it a new value.*

Variables which have been first declared, then reassigned using `:=`, are called *mutable* variables. All the following examples are valid variable reassignments. You will find more information on how var works in the section on the `var` declaration mode:

```
//@version=6
indicator("", "", true)
// Declare `pHi` and initilize it on the first bar only.
var float pHi = na
// Reassign a value to `pHi`
pHi := nz(ta.pivothigh(5, 5), pHi)
plot(pHi)
```

Note that:

- We declare `pHi` with this code: `var float pHi = na`. The var keyword tells Pine Script™ that we only want that variable initialized with na on the dataset's first bar. The `float` keyword tells the compiler we are declaring a variable of type "float". This is necessary because, contrary to most cases, the compiler cannot automatically determine the type of the value on the right side of the `=` sign.
- While the variable declaration will only be executed on the first bar because it uses var, the `pHi := nz(ta.pivothigh(5, 5), pHi)` line will be executed on all the chart's bars. On each bar, it evaluates if the ta.pivothigh() call returns na because that is what the function does when it hasn't found a new pivot. The nz() function is the one doing the "checking for na" part. When its first argument (`ta.pivothigh(5, 5)`) is na, it returns the second argument (`pHi`) instead of the first. When ta.pivothigh() returns the price point of a newly found pivot, that value is assigned to `pHi`. When it returns na because no new pivot was found, we assign the previous value of `pHi` to itself, in effect preserving its previous value.

The output of our script looks like this:

Figure 16: image

Note that:

- The line preserves its previous value until a new pivot is found.
- Pivots are detected five bars after the pivot actually occurs because our `ta.pivothigh(5, 5)` call says that we require five lower highs on both sides of a high point for it to be detected as a pivot.

See the Variable reassignment section for more information on how to reassign values to variables.

[Previous

Identifiers](#identifiers)[Next

Variable declarations](#variable-declarations) User Manual/Language/Variable declarations

Variable declarations

Introduction

Variables are identifiers that hold values. They must be *declared* in your code before you use them. The syntax of variable declarations is:

```
[<declaration_mode>] [<type>] <identifier> = <expression> | <structure>
```

or

```
<tuple_declaration> = <function_call> | <structure>
```

where:

- `|` means "or", and parts enclosed in square brackets (`[]`) can appear zero or one time.
- is the variable's declaration mode. It can be var or varip, or nothing.
- is a valid *type keyword* with an optional *qualifier prefix*. Specifying a variable's type is optional in most cases. See the Type system page to learn more.
- is the variable's name.
- can be a literal, a variable, an expression or a function call.
- can be an if, for, while or switch*structure*.
- is a comma-separated list of variable names enclosed in square brackets (`[]`), e.g., `[ma, upperBand, lowerBand]`.

These are all valid variable declarations. Note that the last declaration requires four lines of code because it uses the returned value from an if statement:

```
BULL_COLOR = color.lime
i = 1
len = input(20, "Length")
float f = 10.5
closeRoundedToTick = math.round_to_mintick(close)
sma = ta.sma(close, 14)
var barRange = float(na)
var firstBarOpen = open
varip float lastClose = na
[macdLine, signalLine, histLine] = ta.macd(close, 12, 26, 9)
plotColor = if close > open
    color.green
else
    color.red
```

The formal syntax of a variable declaration is:

```
<variable_declaration>    [<declaration_mode>] [<type>] <identifier> = <expression> | <structure>    |    <tup
<declaration_mode>    var | varip
<type>    int | float | bool | color | string | line | linefill | label | box | table | polyline | chart.point
```

Initialization with `na`

In most cases, an explicit type declaration is redundant because type is automatically inferred from the value on the right of the `=` at compile time, so the decision to use them is often a matter of preference. For example:

```
baseLine0 = na          // compile time error!
float baseLine1 = na    // OK
baseLine2 = float(na)   // OK
```

In the first line of the example, the compiler cannot determine the type of the `baseLine0` variable because na is a generic value of no particular type. The declaration of the `baseLine1` variable is correct because its float type is declared explicitly.

The declaration of the `baseLine2` variable is also correct because its type can be derived from the expression `float(na)`, which is an explicit cast of the na value to the float type. The declarations of `baseLine1` and `baseLine2` are equivalent.

Tuple declarations

Function calls or structures are allowed to return multiple values. When we call them and want to store the values they return, a *tuple declaration* must be used, which is a comma-separated set of one or more values enclosed in brackets. This allows us to declare multiple variables simultaneously. As an example, the ta.bb() built-in function for Bollinger bands returns three values:

```
[bbMiddle, bbUpper, bbLower] = ta.bb(close, 5, 4)
```

Using an underscore (_) as an identifier

When declaring a variable, it is possible to use a single underscore (_) as its identifier. A value assigned to such a variable cannot be accessed. You can assign any number of values to a _ identifier anywhere in the script, even if the current scope already has such an assignment.

This is particularly useful when a tuple returns unneeded values. Let's write another Bollinger Bands script. Here, we only need the bands themselves, without the center line:

```
//@version=6
indicator("Underscore demo")

// We do not need the middle Bollinger Bands value, and do not use it.
// To make this clear, we assign it to the `_` identifier.
[_, bbUpper, bbLower] = ta.bb(close, 5, 4)

// We can continue to use `_` in the same code without causing compilation errors:
[bbMiddleLong, _, _] = ta.bb(close, 20, 2)

plot(bbUpper)
```

Variable reassignment

A variable reassignment is done using the := reassignment operator. It can only be done after a variable has been first declared and given an initial value. Reassigning a new value to a variable is often necessary in calculations, and it is always necessary when a variable from the global scope must be assigned a new value from within a structure's local block, e.g.:

```
//@version=6
indicator("", "", true)
sensitivityInput = input.int(2, "Sensitivity", minval = 1, tooltip = "Higher values make color changes less se
ma = ta.sma(close, 20)
maUp = ta.rising(ma, sensitivityInput)
maDn = ta.falling(ma, sensitivityInput)

// On first bar only, initialize color to gray
var maColor = color.gray
if maUp
    // MA has risen for two bars in a row; make it lime.
    maColor := color.lime
else if maDn
    // MA has fallen for two bars in a row; make it fuchsia.
    maColor := color.fuchsia

plot(ma, "MA", maColor, 2)
```

Note that:

- We initialize `maColor` on the first bar only, so it preserves its value across bars.
- On every bar, the if statement checks if the MA has been rising or falling for the user-specified number of bars (the default is 2). When that happens, the value of `maColor` must be reassigned a new value from within the if local blocks. To do this, we use the := reassignment operator.

- If we did not use the := reassignment operator, the effect would be to initialize a new `maColor` local variable which would have the same name as that of the global scope, but actually be a very confusing independent entity that would persist only for the length of the local block, and then disappear without a trace.

All user-defined variables in Pine Script™ are *mutable*, which means their value can be changed using the := reassignment operator. Assigning a new value to a variable may change its *type qualifier* (see the page on Pine Script™'s type system for more information). A variable can be assigned a new value as many times as needed during the script's execution on one bar, so a script can contain any number of reassignments of one variable. A variable's declaration mode determines how new values assigned to a variable will be saved.

Declaration modes

Understanding the impact that declaration modes have on the behavior of variables requires prior knowledge of Pine Script™'s execution model.

When you declare a variable, if a declaration mode is specified, it must come first. Three modes can be used:

- "On each bar", when none is specified
- var
- varip

On each bar

When no explicit declaration mode is specified, i.e. no var or varip keyword is used, the variable is declared and initialized on each bar, e.g., the following declarations from our first set of examples in this page's introduction:

```
BULL_COLOR = color.lime
i = 1
len = input(20, "Length")
float f = 10.5
closeRoundedToTick = math.round_to_mintick(close)
st = ta.supertrend(4, 14)
[macdLine, signalLine, histLine] = ta.macd(close, 12, 26, 9)
plotColor = if close > open
    color.green
else
    color.red
```

var

When the var keyword is used, the variable is only initialized once, on the first bar if the declaration is in the global scope, or the first time the local block is executed if the declaration is inside a local block. After that, it will preserve its last value on successive bars, until we reassign a new value to it. This behavior is very useful in many cases where a variable's value must persist through the iterations of a script across successive bars. For example, suppose we'd like to count the number of green bars on the chart:

```
//@version=6
indicator("Green Bars Count")
var count = 0
isGreen = close >= open
if isGreen
    count := count + 1
plot(count)
```

Without the `var` modifier, variable `count` would be reset to zero (thus losing its value) every time a new bar update triggered a script recalculation.

Declaring variables on the first bar only is often useful to manage drawings more efficiently. Suppose we want to extend the last bar's close line to the right of the right chart. We could write:

```
//@version=6
indicator("Inefficient version", "", true)
closeLine = line.new(bar_index - 1, close, bar_index, close, extend = extend.right, width = 3)
line.delete(closeLine[1])
```

Figure 17: image

but this is inefficient because we are creating and deleting the line on each historical bar and on each update in the realtime bar. It is more efficient to use:

```
//@version=6
indicator("Efficient version", "", true)
var closeLine = line.new(bar_index - 1, close, bar_index, close, extend = extend.right, width = 3)
if barstate.islast
    line.set_xy1(closeLine, bar_index - 1, close)
    line.set_xy2(closeLine, bar_index, close)
```

Note that:

- We initialize `closeLine` on the first bar only, using the var declaration mode
- We restrict the execution of the rest of our code to the chart's last bar by enclosing our code that updates the line in an ifbarstate.islast structure.

There is a very slight penalty performance for using the var declaration mode. For that reason, when declaring constants, it is preferable not to use var if performance is a concern, unless the initialization involves calculations that take longer than the maintenance penalty, e.g., functions with complex code or string manipulations.

`varip`

Understanding the behavior of variables using the varip declaration mode requires prior knowledge of Pine Script™'s execution model and bar states.

The varip keyword can be used to declare variables that escape the *rollback process*, which is explained in the page on Pine Script™'s execution model.

Whereas scripts only execute once at the close of historical bars, when a script is running in realtime, it executes every time the chart's feed detects a price or volume update. At every realtime update, Pine Script™'s runtime normally resets the values of a script's variables to their last committed value, i.e., the value they held when the previous bar closed. This is generally handy, as each realtime script execution starts from a known state, which simplifies script logic.

Sometimes, however, script logic requires code to be able to save variable values **between different executions** in the realtime bar. Declaring variables with varip makes that possible. The "ip" in varip stands for *intrabar persist*.

Let's look at the following code, which does not use varip:

```
//@version=6
indicator("")
int updateNo = na
if barstate.isnew
    updateNo := 1
else
    updateNo := updateNo + 1

plot(updateNo, style = plot.style_circles)
```

On historical bars, barstate.isnew is always true, so the plot shows a value of "1" because the `else` part of the if structure is never executed. On realtime bars, barstate.isnew is only true when the script first executes on the bar's "open". The plot will then briefly display "1" until subsequent executions occur. On the next executions during the realtime bar, the second branch of the if statement is executed because barstate.isnew is no longer true. Since `updateNo` is initialized to na at each execution, the `updateNo + 1` expression yields na, so nothing is plotted on further realtime executions of the script.

If we now use varip to declare the `updateNo` variable, the script behaves very differently:

```
//@version=6
indicator("")
varip int updateNo = na
if barstate.isnew
    updateNo := 1
else
    updateNo := updateNo + 1

plot(updateNo, style = plot.style_circles)
```

The difference now is that `updateNo` tracks the number of realtime updates that occur on each realtime bar. This can happen because the varip declaration allows the value of `updateNo` to be preserved between realtime updates; it is no longer rolled back at each realtime execution of the script. The test on barstate.isnew allows us to reset the update count when a new realtime bar comes in.

Because varip only affects the behavior of your code in the realtime bar, it follows that backtest results on strategies designed using logic based on varip variables will not be able to reproduce that behavior on historical bars, which will invalidate test results on them. This also entails that plots on historical bars will not be able to reproduce the script's behavior in realtime.

[Previous

Operators](#operators)[Next

Conditional structures](#conditional-structures) User Manual/Language/Conditional structures

Conditional structures

Introduction

The conditional structures in Pine Script™ are if and switch. They can be used:

- For their side effects, i.e., when they don't return a value but do things, like reassign values to variables or call functions.
- To return a value or a tuple which can then be assigned to one (or more, in the case of tuples) variable.

Conditional structures, like the for and while structures, can be embedded; you can use an if or switch inside another structure.

Some Pine Script™ built-in functions are **not** callable from within the local blocks of conditional structures, including barcolor(), bgcolor(), plot(), plotshape(), plotchar(), plotarrow(), plotcandle(), plotbar(), hline(), fill(), alertcondition(), indicator(), strategy(), and library().

This restriction does not entail their functionality cannot be controlled by conditions evaluated by your script — only that it cannot be done by including them in conditional structures. Note that while `input*.()` function calls are allowed in local blocks, their functionality is the same as if they were in the script's *global scope*.

The local blocks in conditional structures must be indented by four spaces or a tab.

if structure

if used for its side effects

An if structure used for its side effects has the following syntax:

```
if <expression>    <local_block>{else if <expression>    <local_block>}[else    <local_block>]
```

where:

- Parts enclosed in square brackets (`[]`) can appear zero or one time, and those enclosed in curly braces (`{}`) can appear zero or more times.
- must be of "bool" type or be auto-castable to that type, which is only possible for "int" or "float" values (see the Type system page).
- consists of zero or more statements followed by a return value, which can be a tuple of values. It must be indented by four spaces or a tab.
- There can be zero or more `else if` clauses.
- There can be zero or one `else` clause.

When the following the if evaluates to true, the first local block is executed, the if structure's execution ends, and the value(s) evaluated at the end of the local block are returned.

When the following the if evaluates to false, the successive `else if` clauses are evaluated, if there are any. When the of one evaluates to true, its local block is executed, the if structure's execution ends, and the value(s) evaluated at the end of the local block are returned.

When no has evaluated to true and an `else` clause exists, its local block is executed, the if structure's execution ends, and the value(s) evaluated at the end of the local block are returned.

When no has evaluated to true and no `else` clause exists, na is returned. The only exception to this is if the structure returns "bool" values — in that case, false is returned instead.

Using if structures for their side effects can be useful to manage the order flow in strategies, for example. While the same functionality can often be achieved using the `when` parameter in `strategy.*()` calls, code using if structures is easier to read:

```
if (ta.crossover(source, lower))
    strategy.entry("BBandLE", strategy.long, stop=lower,
                   oca_name="BollingerBands",
                   oca_type=strategy.oca.cancel, comment="BBandLE")
else
    strategy.cancel(id="BBandLE")
```

Restricting the execution of your code to specific bars ican be done using if structures, as we do here to restrict updates to our label to the chart's last bar:

```
//@version=6
indicator("", "", true)
var ourLabel = label.new(bar_index, na, na, color = color(na), textcolor = color.orange)
if barstate.islast
    label.set_xy(ourLabel, bar_index + 2, hl2[1])
    label.set_text(ourLabel, str.tostring(bar_index + 1, "# bars in chart"))
```

Note that:

- We initialize the `ourLabel` variable on the script's first bar only, as we use the var declaration mode. The value used to initialize the variable is provided by the label.new() function call, which returns a label ID pointing to the label it creates. We use that call to set the label's properties because once set, they will persist until we change them.
- What happens next is that on each successive bar the Pine Script™ runtime will skip the initialization of `ourLabel`, and the if structure's condition (barstate.islast) is evaluated. It returns `false` on all bars until the last one, so the script does nothing on most historical bars after bar zero.
- On the last bar, barstate.islast becomes true and the structure's local block executes, modifying on each chart update the properties of our label, which displays the number of bars in the dataset.
- We want to display the label's text without a background, so we make the label's background na in the label.new() function call, and we use `hl2[1]` for the label's *y* position because we don't want it to move all the time. By using the average of the **previous** bar's high and low values, the label doesn't move until the moment when the next realtime bar opens.
- We use `bar_index + 2` in our label.set_xy() call to offset the label to the right by two bars.

`if` used to return a value

An if structure used to return one or more values has the following syntax:

```
[<declaration_mode>] [<type>] <identifier> = if <expression>    <local_block>{else if <expression>    <local_b
```

where:

- Parts enclosed in square brackets (`[]`) can appear zero or one time, and those enclosed in curly braces (`{}`) can appear zero or more times.
- is the variable's declaration mode
- is optional, as in almost all Pine Script™ variable declarations (see types)
- is the variable's name
- can be a literal, a variable, an expression or a function call.
- consists of zero or more statements followed by a return value, which can be a tuple of values. It must be indented by four spaces or a tab.
- The value assigned to the variable is the return value of the , or na if no local block is executed. If other local blocks return "bool" values, false will be returned instead.

This is an example:

```
//@version=6
indicator("", "", true)
string barState = if barstate.islastconfirmedhistory
    "islastconfirmedhistory"
else if barstate.isnew
    "isnew"
else if barstate.isrealtime
    "isrealtime"
else
    "other"

f_print(_text) =>
    var table _t = table.new(position.middle_right, 1, 1)
    table.cell(_t, 0, 0, _text, bgcolor = color.yellow)
f_print(barState)
```

It is possible to omit the *else* block. In this case, if the `condition` is false, an *empty* value (`na`, `false`, or `""`) will be assigned to the `var_declarationX` variable.

This is an example showing how na is returned when no local block is executed. If `close > open` is `false` in here, na is returned:

```
x = if close > open
    close
```

Scripts can contain `if` structures with nested `if` and other conditional structures. For example:

```
if condition1
    if condition2
        if condition3
            expression
```

However, nesting these structures is not recommended from a performance perspective. When possible, it is typically more optimal to compose a single `if` statement with multiple logical operators rather than several nested `if` blocks:

```
if condition1 and condition2 and condition3
    expression
```

`switch` structure

The switch structure exists in two forms. One switches on the different values of a key expression:

```
[[<declaration_mode>] [<type>] <identifier> = ]switch <expression>    {<expression> => <local_block>}    => <l
```

The other form does not use an expression as a key; it switches on the evaluation of different expressions:

```
[[<declaration_mode>] [<type>] <identifier> = ]switch    {<expression> => <local_block>}    => <local_block>
```

where:

- Parts enclosed in square brackets (`[]`) can appear zero or one time, and those enclosed in curly braces (`{}`) can appear zero or more times.
- is the variable's declaration mode
- is optional, as in almost all Pine Script™ variable declarations (see types)
- is the variable's name
- can be a literal, a variable, an expression or a function call.
- consists of zero or more statements followed by a return value, which can be a tuple of values. It must be indented by four spaces or a tab.
- The value assigned to the variable is the return value of the , or na if no local block is executed.
- The `=> <local_block>` at the end allows you to specify a return value which acts as a default to be used when no other case in the structure is executed.

Only one local block of a switch structure is executed. It is thus a *structured switch* that doesn't *fall through* cases. Consequently, `break` statements are unnecessary.

Both forms are allowed as the value used to initialize a variable.

As with the if structure, if no local block is exectuted, the expression returns either false (when other local blocks return a "bool" value) or na (in all other cases).

`switch` with an expression

Let's look at an example of a switch using an expression:

```
//@version=6
indicator("Switch using an expression", "", true)

string maType = input.string("EMA", "MA type", options = ["EMA", "SMA", "RMA", "WMA"])
int maLength = input.int(10, "MA length", minval = 2)

float ma = switch maType
    "EMA" => ta.ema(close, maLength)
    "SMA" => ta.sma(close, maLength)
    "RMA" => ta.rma(close, maLength)
    "WMA" => ta.wma(close, maLength)
    =>
        runtime.error("No matching MA type found.")
        float(na)

plot(ma)
```

Note that:

- The expression we are switching on is the variable `maType`, which is of "input int" type (see here for an explanation of what the "input" qualifier is). Since it cannot change during the execution of the script, this guarantees that whichever MA type the user selects will be executing on each bar, which is a requirement for functions like ta.ema() which require a "simple int" argument for their `length` parameter.
- If no matching value is found for `maType`, the switch executes the last local block introduced by `=>`, which acts as a catch-all. We generate a runtime error in that block. We also end it with `float(na)` so the local block returns a value whose type is compatible with that of the other local blocks in the structure, to avoid a compilation error.

`switch` without an expression

This is an example of a switch structure which does not use an expression:

```
//@version=6
strategy("Switch without an expression", "", true)

bool longCondition  = ta.crossover( ta.sma(close, 14), ta.sma(close, 28))
bool shortCondition = ta.crossunder(ta.sma(close, 14), ta.sma(close, 28))

switch
    longCondition  => strategy.entry("Long ID", strategy.long)
```

```
    shortCondition => strategy.entry("Short ID", strategy.short)
```

Note that:

- We are using the switch to select the appropriate strategy order to emit, depending on whether the `longCondition` or `shortCondition` "bool" variables are `true`.
- The building conditions of `longCondition` and `shortCondition` are exclusive. While they can both be `false` simultaneously, they cannot be `true` at the same time. The fact that only **one** local block of the switch structure is ever executed is thus not an issue for us.
- We evaluate the calls to ta.crossover() and ta.crossunder()**prior** to entry in the switch structure. Not doing so, as in the following example, would prevent the functions to be executed on each bar, which would result in a compiler warning and erratic behavior:

```
//@version=6
strategy("Switch without an expression", "", true)

switch
    // Compiler warning! Will not calculate correctly!
    ta.crossover( ta.sma(close, 14), ta.sma(close, 28)) => strategy.entry("Long ID", strategy.long)
    ta.crossunder(ta.sma(close, 14), ta.sma(close, 28)) => strategy.entry("Short ID", strategy.short)
```

Matching local block type requirement

When multiple local blocks are used in structures, the type of the return value of all its local blocks must match. This applies only if the structure is used to assign a value to a variable in a declaration, because a variable can only have one type, and if the statement returns two incompatible types in its branches, the variable type cannot be properly determined. If the structure is not assigned anywhere, its branches can return different values.

This code compiles fine because close and open are both of the `float` type:

```
x = if close > open
    close
else
    open
```

This code does not compile because the first local block returns a `float` value, while the second one returns a `string`, and the result of the `if`-statement is assigned to the `x` variable:

```
// Compilation error!
x = if close > open
    close
else
    "open"
```

[Previous

Variable declarations](#variable-declarations)[Next

Loops](#loops) User Manual/Language/Loops

Loops

Introduction

Loops are structures that repeatedly execute a block of statements based on specified criteria. They allow scripts to perform repetitive tasks without requiring duplicated lines of code. Pine Script™ features three distinct loop types: for, while, and for…in.

Every loop structure in Pine Script consists of two main parts: a *loop header* and a *loop body*. The loop header determines the criteria under which the loop executes. The loop body is the indented block of code (local block) that the script executes on each loop cycle (*iteration*) as long as the header's conditions remain valid. See the Common characteristics section to learn more.

Understanding when and how to use loops is essential for making the most of the power of Pine Script. Inefficient or unnecessary usage of loops can lead to suboptimal runtime performance. However, effectively using loops when necessary enables scripts to perform a wide range of calculations that would otherwise be impractical or impossible without them.

When loops are unnecessary

Pine's execution model and time series structure make loops *unnecessary* in many situations.

When a user adds a Pine script to a chart, it runs within the equivalent of a *large loop*, executing its code once on *every* historical bar and realtime tick in the available data. Scripts can access the values from the executions on previous bars with the history-referencing operator, and calculated values can *persist* across executions when assigned to variables declared with the var or varip keywords. These capabilities enable scripts to utilize bar-by-bar calculations to accomplish various tasks instead of relying on explicit loops.

In addition, several built-ins, such as those in the `ta.*` namespace, are internally optimized to eliminate the need to use loops for various calculations.

Let's consider a simple example demonstrating unnecessary loop usage in Pine Script. To calculate the average close over a specified number of bars, newcomers to Pine may write a code like the following, which uses a for loop to calculate the sum of historical values over `lengthInput` bars and divides the result by the `lengthInput`:

Figure 18: image

```
//@version=6
indicator("Unnecessary loops demo", overlay = true)

//@variable The number of bars in the calculation window.
int lengthInput = input.int(defval = 20, title = "Length")

//@variable The sum of `close` values over `lengthInput` bars.
float closeSum = 0

// Loop over the most recent `lengthInput` bars, adding each bar's `close` to the `closeSum`.
for i = 0 to lengthInput - 1
    closeSum += close[i]

//@variable The average `close` value over `lengthInput` bars.
float avgClose = closeSum / lengthInput

// Plot the `avgClose`.
plot(avgClose, "Average close", color.orange, 2)
```

Using a for loop is an **unnecessary**, inefficient way to accomplish tasks like this in Pine. There are several ways to utilize the execution model and the available built-ins to eliminate this loop. Below, we replaced these calculations with a simple

call to the ta.sma() function. This code is shorter, and it achieves the same result much more efficiently:

Figure 19: image

```
//@version=6
indicator("Unnecessary loops corrected demo", overlay = true)

//@variable The number of bars in the calculation window.
int lengthInput = input.int(defval = 20, title = "Length")

//@variable The average `close` value over `lengthInput` bars.
float avgClose = ta.sma(close, lengthInput)

// Plot the `avgClose`.
plot(avgClose, "Average close", color.blue, 2)
```

Note that:

- Users can see the substantial difference in efficiency between these two example scripts by analyzing their performance with the Pine Profiler.

When loops are necessary

Although Pine's execution model, time series, and available built-ins often eliminate the need for loops in many cases, not all iterative tasks have loop-free alternatives. Loops *are necessary* for several types of tasks, including:

- Iterating through or manipulating collections (arrays, matrices, and maps)
- Performing calculations that one **cannot** accomplish with loop-free expressions or the available built-ins
- Looking back through history to analyze past bars with a reference value only available on the *current bar*

For example, a loop is *necessary* to identify which past bars' high values are above the current bar's high because the current value is **not** obtainable during a script's executions on previous bars. The script can only access the current bar's value while it executes on that bar, and it must *look back* through the historical series during that execution to compare the previous values.

The script below uses a for loop to compare the high values of `lengthInput` previous bars with the last historical bar's high. Within the loop, it calls label.new() to draw a circular label above each past bar that has a high value exceeding that of the last historical bar:

```
//@version=6
indicator("Necessary loop demo", overlay = true, max_labels_count = 500)

//@variable The number of previous `high` values to compare to the last historical bar's `high`.
int lengthInput = input.int(20, "Length", 1, 500)
```

Figure 20: image

```
if barstate.islastconfirmedhistory
    // Draw a horizontal line segment at the last historical bar's `high` to visualize the level.
    line.new(bar_index - lengthInput, high, bar_index, high, color = color.gray, style = line.style_dashed, wi
    // Create a `for` loop that counts from 1 to `lengthInput`.
    for i = 1 to lengthInput
        // Draw a circular `label` above the bar from `i` bars ago if that bar's `high` is above the current `
        if high[i] > high
            label.new(
                 bar_index - i, na, "", yloc = yloc.abovebar, color = color.purple,
                 style = label.style_circle, size = size.tiny
             )

// Highlight the last historical bar.
barcolor(barstate.islastconfirmedhistory ? color.orange : na, title = "Last historical bar highlight")
```

Note that:

- Each *iteration* of the for loop retrieves a previous bar's high with the history-referencing operator [], using the loop's *counter* (`i`) as the historical offset. The label.new() call also uses the counter to determine each label's x-coordinate.
- The indicator declaration statement includes `max_labels_count = 500`, meaning the script can show up to 500 labels on the chart.
- The script calls barcolor() to highlight the last historical chart bar, and it draws a horizontal line at that bar's high for visual reference.

Common characteristics

The for, while, and for…in loop statements all have similarities in their structure, syntax, and general behavior. Before we explore each specific loop type, let's familiarize ourselves with these characteristics.

Structure and syntax

In any loop statement, programmers define the criteria under which a script remains in a loop and performs *iterations*, where an iteration refers to *one execution* of the code within the loop's local block (*body*). These criteria are part of the *loop header*. A script evaluates the header's criteria *before* each iteration, only allowing new iterations to occur while they remain valid. When the header's criteria are no longer valid, the script *exits* the loop and skips over its body.

The specific header syntax varies with each loop statement (for, while, or for…in) because each uses *distinct* criteria to control its iterations. Effective use of loops entails choosing the structure with control criteria best suited for a script's required tasks. See the `for` loops, `while` loops, and `for…in` loops sections below for more information on each loop statement and its control criteria.

All loop statements in Pine Script follow the same general syntax:

```
[variables = | :=] loop_header    statements | continue | break    return_expression
```

Where:

- `loop_header` represents the loop structure's header statement, which defines the criteria that control the loop's iterations.
- `statements` represents the code statements and expressions within the loop's body, i.e., the *indented* block of code beneath the loop header. All code within the body belongs to the loop's local scope.
- `continue` and `break` are loop-specific *keywords* that control the flow of a loop's iterations. The `continue` keyword instructs the script to *skip* the remainder of the current loop iteration and *continue* to the next iteration. The `break` keyword prompts the script to *stop* the current iteration and *exit* the loop entirely. See this section below for more information.
- `return_expression` refers to the *last* code line or block within the loop's body. The loop returns the results from this code after the final iteration. If the loop skips parts of some iterations or stops prematurely due to a `continue` or `break` statement, the returned values or references are those of the latest iteration that evaluated this code. To use the loop's returned results, assign them to a variable or tuple.
- `variables` represents an optional variable or tuple to hold the values or references from the last evaluation of the `return_expression`. The script can assign the loop's returned results to variables only if the results are not void. If the loop's conditions prevent iteration, or if no iterations evaluate the `return_expression`, the variables' assigned values and references are na.

Scope

All code lines that a script executes within a loop must have an indentation of *four spaces* or a *tab* relative to the loop's header. The indented lines following the header define the loop's *body*. This code represents a *local block*, meaning that all the definitions within the body are accessible only during the loop's execution. In other words, the code within the loop's body is part of its *local scope*.

Scripts can modify and reassign most variables from *outer* scopes inside a loop. However, any variables declared within the loop's body strictly belong to that loop's local scope. A script **cannot** access a loop's declared variables *outside* its local block.

Note that:

- Variables declared within a loop's *header* are also part of the local scope. For instance, a script cannot use the *counter variable* in a for loop anywhere but within the loop's local block.

The body of any Pine loop statement can include conditional structures and *nested* loop statements. When a loop includes nested structures, each structure within the body maintains a *distinct* local scope. For example, variables declared within an *outer* loop's scope are accessible to an *inner* loop. However, any variables declared within the inner loop's scope are **not** accessible to the outer loop.

The simple example below demonstrates how a loop's local scope works. This script calls label.new() within a for loop on the last historical bar to draw labels above `lengthInput` past bars. The color of each label depends on the `labelColor` variable declared *within* the loop's local block, and each label's location depends on the loop counter (`i`):

```
//@version=6
indicator("Loop scope demo", overlay = true)

//@variable The number of bars in the calculation.
int lengthInput = input.int(20, "Lookback length", 1)

if barstate.islastconfirmedhistory
    for i = 1 to lengthInput
        //@variable Has a value of `color.blue` if `close[i]` is above the current `close`, `color.orange` oth
        //          This variable is LOCAL to the `for` loop's scope.
        color labelColor = close[i] > close ? color.blue : color.orange
        // Display a colored `label` on the historical `high` from `i` bars back, using `labelColor` to set th
        label.new(bar_index - i, high[i], "", color = labelColor, size = size.normal)
```

In the above code, the `i` and `labelColor` variables are only accessible to the for loop's local scope. They are **not** usable within any outer scopes. Here, we added a label.new() call *after* the loop with `bar_index - i` as the `x` argument and `labelColor`

Figure 21: image

as the `color` argument. This code causes a *compilation error* because neither `i` nor `labelColor` are valid variables in the outer scope:

```
//@version=6
indicator("Loop scope demo", overlay = true)

//@variable The number of bars in the calculation.
int lengthInput = input.int(20, "Lookback length", 1)

if barstate.islastconfirmedhistory
    for i = 1 to lengthInput
        //@variable Has a value of `color.blue` if `close[i]` is above the current `close`, `color.orange` oth
        //          This variable is LOCAL to the `for` loop's scope.
        color labelColor = close[i] > close ? color.blue : color.orange
        // Display a colored `label` on the historical `high` from `i` bars back, using `labelColor` to set th
        label.new(bar_index - i, high[i], "", color = labelColor, size = size.normal)

    // Call `label.new()` to using the `i` and `labelColor` variables outside the loop's local scope.
    // This code causes a compilation error because these variables are not accessible in this location.
    label.new(
         bar_index - i, low, "Scope test", textcolor = color.white, color = labelColor, style = label.style_la
     )
```

Keywords and return expressions

Every loop in Pine Script implicitly *returns* values, references, or void. A loop's returned results come from the *latest* execution of the *last* expression or nested structure within its body as of the final iteration. The results are usable only if they are not of the void type. Loops return na results for values or references when no iterations occur. Scripts can add a variable or tuple assignment to a loop statement to hold the returned results for use in additional calculations outside the loop's local scope.

The values or references that a loop returns usually come from evaluating the last written expression or nested code block on the *final* iteration. However, a loop's body can include `continue` and `break` keywords to control the flow of iterations beyond the criteria the loop header specifies, which can also affect the returned results. Programmers often include these keywords within conditional structures to control how iterations behave when certain conditions occur.

The `continue` keyword instructs a script to *skip* the remaining statements and expressions in the current loop iteration, re-evaluate the loop header's criteria, and proceed to the *next* iteration. The script *exits* the loop if the header's criteria do

not allow another iteration.

The `break` keyword instructs a script to *stop* the loop entirely and immediately *exit* at that point without allowing any subsequent iterations. After breaking the loop, the script skips any remaining code within the loop's body and *does not* re-evaluate the header's criteria.

If a loop skips parts of iterations or stops prematurely due to a `continue` or `break` statement, it returns the values and references from the *last iteration* where the script *evaluated* the return expression. If the script did not evaluate the return expression across *any* of the loop's iterations, the loop returns na results for all non-void types.

The example below selectively displays numbers from an array within a label on the last historical bar. It uses a for…in loop to iterate through the array's elements and build a "string" to use as the displayed text. The loop's body contains an if statement that controls the flow of specific iterations. If the `number` in the current iteration is 8, the script immediately *exits* the loop using the `break` keyword. Otherwise, if the `number` is even, it *skips* the rest of the current iteration and moves to the next one using the `continue` keyword.

If neither of the if statement's conditions occur, the script evaluates the *last expression* within the loop's body (i.e., the return expression), which converts the current `number` to a "string" and concatenates the result with the `tempString` value. The loop returns the *last evaluated result* from this expression after termination. The script assigns the returned value to the `finalLabelText` variable and uses that variable as the `text` argument in the label.new() call:

Figure 22: image

```
//@version=6
indicator("Loop keywords and variable assignment demo")

//@variable An `array` of arbitrary "int" values to selectively convert to "string" and display in a `label`.
var array<int> randomArray = array.from(1, 5, 2, -3, 14, 7, 9, 8, 15, 12)

// Label creation logic.
if barstate.islastconfirmedhistory
    //@variable A "string" containing representations of selected values from the `randomArray`.
    string tempString = ""
    //@variable The final text to display in the `label`. The `for..in` loop returns the result after it termi
    string finalLabelText = for number in randomArray
        // Stop the current iteration and exit the loop if the `number` from the `randomArray` is 8.
        if number == 8
            break
        // Skip the rest of the current iteration and proceed to the next iteration if the `number` is even.
        else if number % 2 == 0
            continue
        // Convert the `number` to a "string", append ", ", and concatenate the result with the current `tempS
        // This code represents the loop's return expression.
        tempString += str.tostring(number) + ", "
```

```
// Display the value of the `finalLabelText` within a `label` on the current bar.
label.new(bar_index, 0, finalLabelText, color = color.blue, textcolor = color.white, size = size.huge)
```

Note that:

- The label displays only *odd* numbers from the array because the script does not reassign the `tempString` when the loop iteration's `number` is even. However, it does not include the *last* odd number from the array (15) because the loop stops when `number == 8`, preventing iteration over the remaining `randomArray` elements.
- When the script exits the loop due to the `break` keyword, the loop's return value becomes the last evaluated result from the `tempString` reassignment expression. In this case, the last time that code executes is on the iteration where `number == 9`.

`for` loops

The for loop statement creates a *count-controlled* loop, which uses a *counter* variable to manage the iterative executions of its local code block. The counter starts at a predefined initial value, and the loop increments or decrements the counter by a fixed amount after each iteration. The loop stops its iterations after the counter reaches a specified final value.

Pine Script uses the following syntax to define a for loop:

```
[variables = | :=] for counter = from_num to to_num [by step_num]    statements | continue | break    return_e
```

Where the following parts define the *loop header*:

- `counter` represents the counter variable, which can be any valid identifier. The loop increments or decrements this variable's value from the initial value (`from_num`) to the final value (`to_num`) by a fixed amount (`step_num`) after each iteration. The last possible iteration occurs when the variable's value reaches the `to_num` value.
- `from_num` is the `counter` variable's initial value on the first iteration.
- `to_num` is the *final*`counter` value for which the loop's header allows a new iteration. The loop adjusts the `counter` value by the `step_num` amount until it reaches or passes this value. If the script modifies the `to_num` during a loop iteration, the loop header uses the new value to control the allowed subsequent iterations.
- `step_num` is a positive value representing the amount by which the `counter` value increases or decreases until it reaches or passes the `to_num` value. If the `from_num` value is greater than the *initial*`to_num` value, the loop *subtracts* this amount from the `counter` value after each iteration. Otherwise, the loop *adds* this amount after each iteration. The default is 1.

Refer to the Common characteristics section above for detailed information about the `variables`, `statements`, `continue`, `break`, and `return_expression` parts of the loop's syntax.

This simple script demonstrates a for loop that draws several labels at future bar indices during its execution on the last historical chart bar. The loop's counter starts at 0, then increases by 1 until it reaches a value of 10, at which point the final iteration occurs:

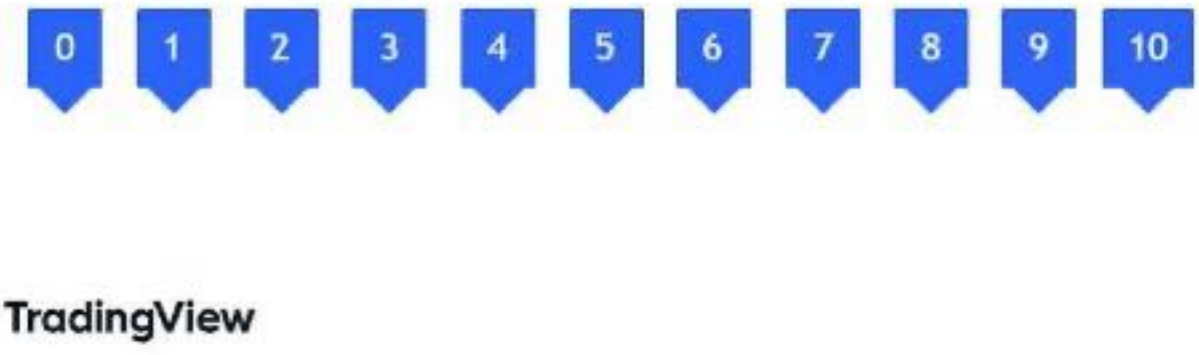

Figure 23: image

```
//@version=6
indicator("Simple `for` loop demo")

if barstate.islastconfirmedhistory
    // Define a `for` loop that iterates from `i == 0` to `i == 10` by 1 (11 total iterations).
    for i = 0 to 10
        // Draw a new label `i` bars ahead of the current bar.
        label.new(bar_index + i, 0, str.tostring(i), textcolor = color.white, size = size.large)
```

Note that:

- The `i` variable represents the loop's *counter*. This variable is local to the loop's scope, meaning no *outer scopes* can access it. The code uses the variable within the loop's body to determine the location and text of each label drawing.
- Programmers often use `i`, `j`, and `k` as loop counter identifiers. However, *any* valid variable name is allowed. For example, this code behaves the same if we name the counter `offset` instead of `i`.
- The for loop structure *automatically* manages the counter variable. We do not need to define code in the loop's body to increment its value.

The direction in which a for loop adjusts its counter depends on the *initial*`from_num` and `to_num` values in the loop's header, and the direction does not change across iterations. The loop counts *upward* after each iteration when the `to_num` value is *above* the `from_num` value, as shown in the previous example. If the `to_num` value is *below* the `from_num` value, the loop counts *downward* instead.

The script below calculates and plots the volume-weighted moving average (VWMA) of open prices across a specified number of bars. Then, it uses a downward-counting for loop to compare the last historical bar's value to the values from previous bars, starting with the oldest bar in the specified lookback window. On each loop iteration, the script retrieves a previous bar's `vwmaOpen` value, calculates the difference from the current bar's value, and displays the result in a label at the past bar's opening price:

Figure 24: image

```
//@version=6
indicator("`for` loop demo", "VWMA differences", true, max_labels_count = 500)

//@variable Display color for indicator visuals.
const color DISPLAY_COLOR = color.rgb(17, 127, 218)

//@variable The number of bars in the `vwmaOpen` calculation.
int maLengthInput = input.int(20, "VWMA length", 1)
//@variable The number of past bars to look back through and compare to the current bar.
int lookbackInput = input.int(15, "Lookback length", 1, 500)
```

```
//@variable The volume-weighted moving average of `open` values over `maLengthInput` bars.
float vwmaOpen = ta.vwma(open, maLengthInput)

if barstate.islastconfirmedhistory
    // Define a `for` loop that counts *downward* from `i == lookbackInput` to `i == 1`.
    for i = lookbackInput to 1
        //@variable The difference between the `vwmaOpen` from `i` bars ago and the current `vwmaOpen`.
        float vwmaDifference = vwmaOpen[i] - vwmaOpen
        //@variable A "string" representation of `vwmaDifference`, rounded to two fractional digits.
        string displayText = (vwmaDifference > 0 ? "+" : "") + str.tostring(vwmaDifference, "0.00")
        // Draw a label showing the `displayText` at the `open` of the bar from `i` bars back.
        label.new(
             bar_index - i, open[i], displayText, textcolor = color.white, color = DISPLAY_COLOR,
             style = label.style_label_lower_right, size = size.normal
         )

// Plot the `vwmaOpen` value.
plot(vwmaOpen, "VWMA", color = DISPLAY_COLOR, linewidth = 2)
```

Note that:

- The script uses the loop's counter (`i`) to within the history-referencing operator to retrieve past values of the `vwmaOpen` series. It also uses the counter to determine the location of each label drawing.
- The loop in this example *decreases* the counter by one on each iteration because the final counter value in the loop's header (`1`) is less than the starting value (`lookbackInput`).

Programmers can use for loops to iterate through collections, such as arrays and matrices. The loop's counter can serve as an *index* for retrieving or modifying a collection's contents. For example, this code block uses array.get() inside a for loop to successively retrieve elements from an array:

```
int lastIndex = array.size(myArray) - 1
for i = 0 to lastIndex
   element = array.get(i)
```

Note that:

- Array *indexing* starts from 0, but the array.size() function *counts* array elements starting from 1. Therefore, we must subtract 1 from the array's size to get the maximum index value. This way, the loop counter avoids representing an out-of-bounds index on the last loop iteration.
- The for…in loop statement is often the *preferred* way to loop through collections. However, programmers may prefer a for loop for some tasks, such as looping through stepped index values, iterating over a collection's contents in reverse or a nonlinear order, and more. See the Looping through arrays and Looping through matrices sections to learn more about the best practices for looping through these collection types.

The script below calculates the RSI and momentum of close prices over three different lengths (10, 20, and 50) and displays their values within a table on the last chart bar. It stores "string" values for the header title within arrays and the "float" values of the calculated indicators within a 2x3 matrix. The script uses a for loop to access the elements in the arrays and initialize the `displayTable` header cells. It then uses *nested*for loops to iterate over the *row* and *column* indices in the `taMatrix`, access elements, convert their values to strings, and populate the remaining table cells:

```
//@version=6
indicator("`for` loop with collections demo", "Table of TA Indexes", overlay = true)

// Calculate the RSI and momentum of `close` values with constant lengths of 10, 20, and 50.
float rsi10 = ta.rsi(close, 10)
float rsi20 = ta.rsi(close, 20)
float rsi50 = ta.rsi(close, 50)
float mom10 = ta.mom(close, 10)
float mom20 = ta.mom(close, 20)
float mom50 = ta.mom(close, 50)

if barstate.islast
    //@variable A `table` that displays indicator values in the top-right corner of the chart.
    var table displayTable = table.new(
```

Figure 25: image

```
     position.top_right, columns = 5, rows = 4, border_color = color.black, border_width = 1
 )
//@variable An array containing the "string" titles to display within the side header of each table row.
array<string> sideHeaderTitles = array.from("TA Index", "RSI", "Momentum")
//@variable An array containing the "string" titles to representing the length of each displayed indicator
array<string> topHeaderTitles = array.from("10", "20", "50")
//@variable A matrix containing the values to display within the table.
matrix<float> taMatrix = matrix.new<float>()
// Populate the `taMatrix` with indicator values. The first row contains RSI data and the second contains
taMatrix.add_row(0, array.from(rsi10, rsi20, rsi50, mom10, mom20, mom50))
taMatrix.reshape(2, 3)

// Initialize top header cells.
displayTable.cell(1, 0, "Bars Length", text_color = color.white, bgcolor = color.blue)
displayTable.merge_cells(1, 0, 3, 0)

// Initialize additional header cells within a `for` loop.
for i = 0 to 2
    displayTable.cell(0, i + 1, sideHeaderTitles.get(i), text_color = color.white, bgcolor = color.blue)
    displayTable.cell(i + 1, 1, topHeaderTitles.get(i), text_color = color.white, bgcolor = color.purple)

// Use nested `for` loops to iterate through the row and column indices of the `taMatrix`.
for i = 0 to taMatrix.rows() - 1
    for j = 0 to taMatrix.columns() - 1
        //@variable The value stored in the `taMatrix` at the `i` row and `j` column.
        float elementValue =  taMatrix.get(i, j)
        // Initialize a cell in the `displayTable` at the `i + 2` row and `j + 1` column showing a "string
        // representation of the `elementValue`.
        displayTable.cell(
             column = j + 1, row = i + 2, text = str.tostring(elementValue, "#.##"), text_color = chart.fg
         )
```

Note that:

- Both arrays of header names (`sideHeaderTitles` and `topHeaderTitles`) contain the same number of elements, which allows the script to iterate through their contents simultaneously using a single for loop.
- The nested for loops iterate over *all* the index values in the `taMatrix`. The *outer* loop iterates over each *row* index, and the *inner* loop iterates over every *column* index on each outer loop iteration.

- The script creates and displays the table only on the last historical bar and all realtime bars because the historical states of tables are *never* visible. See this section of the Profiling and optimization page for more information.

It's important to note that a for loop's header *dynamically* evaluates the `to_num` value at the start of *every iteration*. If the `to_num` argument is a variable and the script changes its value during an iteration, the loop uses the *new value* to update its stopping condition. Likewise, the stopping condition can change across iterations when the `to_num` argument is an expression or function call that depends on data modified in the loop's scope, such as a call to array.size() on a locally resized array or str.length() on an adjusted "string". Therefore, scripts can use for loops to perform iterative tasks where the exact number of required iterations is *not predictable* in advance, similar to while loops.

For example, the following script uses a dynamic for loop to determine the historical offset of the most recent bar whose close differs from the current bar's close by at least one standard deviation. The script declares a `barOffset` variable with an initial value of zero and uses that variable to define the loop counter's `to_num` boundary. Within the loop's scope, the script increments the `barOffset` by one if the referenced bar's `close` is not far enough from the current bar's value. Each time the `barOffset` value increases, the loop increases its final counter value, allowing an *extra iteration*. The script plots the `barOffset` and the corresponding bar's close for visual reference:

Figure 26: image

```
//@version=6
indicator("`for` loop with dynamic `to_num` demo")

//@variable The length of the standard deviation.
int lengthInput = input.int(20, "Length", 1, 4999)

//@variable The standard deviation of `close` prices over `lengthInput` bars.
float stdev = ta.stdev(close, lengthInput)

//@variable The minimum bars back where the past bar's `close` differs from the current `close` by at least `s
//          Used as the weight value in the weighted average.
int barOffset = 0

// Define a `for` loop that iterates from 0 to `barsBack`.
for i = 0 to barOffset
    // Add 1 for each bar where the distance from that bar's `close` to the current bar's `close` is less than
    // Each time `barsBack` increases, it changes the loop's `to_num` boundary, allowing another iteration.
    barOffset += math.abs(close - close[i]) < stdev ? 1 : 0

//@variable A gradient color for the `barOffset` plot.
```

```
color offsetColor = color.from_gradient(barOffset, 0, lengthInput, color.blue, color.orange)

// Plot the `barOffset` in a separate pane.
plot(barOffset, "Bar offset", offsetColor, 1, plot.style_columns)
// Plot the historical `close` price from `barOffset` bars back in the main chart pane.
plot(close[barOffset], "Historical bar's price", color.blue, 3, force_overlay = true)
```

Note that:

- Changing the `to_num` value on an iteration does not affect the established *direction* in which the loop adjusts its counter variable. For instance, if the loop in this example changed `barOffset` to -1 on any iteration, it would stop immediately after that iteration ends without reducing the `i` value.
- The script uses `force_overlay = true` in the second plot() call to display the historical closing price on the main chart pane.

`while` loops

The while loop statement creates a *condition-controlled* loop, which uses a *conditional expression* to control the executions of its local block. The loop continues its iterations as long as the specified condition remains `true`.

Pine Script uses the following syntax to define a while loop:

```
[variables = | :=] while condition    statements | continue | break    return_expression
```

Where the `condition` in the loop's *header* can be a literal, variable, expression, or function call that returns a "bool" value.

Refer to the Common characteristics section above for detailed information about the `variables`, `statements`, `continue`, `break`, and `return_expression` parts of the loop's syntax.

A while loop's header evaluates its `condition` before each iteration. Consequently, when the script modifies the condition within an iteration, the loop's header reflects those changes on the *next* iteration.

Depending on the specified condition in the loop header, a while loop can behave similarly to a for loop, continuing iteration until a *counter* variable reaches a specified limit. For example, the following script uses a for loop and while loop to perform the same task. Both loops draw a label displaying their respective counter value on each iteration:

Figure 27: image

```
//@version=6
indicator("`while` loop with a counter condition demo")

if barstate.islastconfirmedhistory
    // A `for` loop that creates blue labels displaying each `i` value.
    for i = 0 to 10
        label.new(
             bar_index + i, 0, str.tostring(i), color = color.blue, textcolor = color.white,
             size = size.large, style = label.style_label_down
```

```
     )

//@variable An "int" to use as a counter within a `while` loop.
int j = 0
// A `while` loop that creates orange labels displaying each `j` value.
while j <= 10
    label.new(
         bar_index + j, 0, str.tostring(j), color = color.orange, textcolor = color.white,
         size = size.large, style = label.style_label_up
     )
    // Update the `j` counter within the local block.
    j += 1
```

Note that:

- When a while loop uses count-based logic, it must explicitly manage the user-specified counter within the local block. In contrast, a for loop increments its counter automatically.
- The script declares the variable the while loop uses as a counter *outside* the loop's scope, meaning its value is usable in additional calculations after the loop terminates.
- If this code did not increment the `j` variable within the while loop's body, the value would *never* reach 10, meaning the loop would run *indefinitely* until causing a runtime error.

Because a while loop's execution depends on its condition remaining `true`, and the condition might not change on a specific iteration, the *precise* number of expected iterations might not be knowable *before* the loop begins. Therefore, while loops are often helpful in scenarios where the exact loop boundaries are *unknown*.

The script below tracks when the chart's close crosses outside Keltner Channels with a user-specified length and channel width. When the price crosses outside the current bar's channel, the script draws a box highlighting all the previous *consecutive* bars with close values within that price window. The script uses a while loop to analyze past bars' prices and incrementally adjust the left side of each new box until the drawing covers all the latest consecutive bars in the current range:

Figure 28: image

```
//@version=6
indicator("`while` loop demo", "Price window boxes", true)

//@variable The length of the channel.
int lengthInput = input.int(20, "Channel length", 1, 4999)
//@variable The width multiplier of the channel.
float widthInput = input.float(2.0, "Width multiplier", 0)

//@variable The `lengthInput`-bar EMA of `close` prices.
```

```
float ma = ta.ema(close, lengthInput)
//@variable The `lengthInput`-bar ATR, multiplied by the `widthInput`.
float atr = ta.atr(lengthInput) * widthInput
//@variable The lower bound of the channel.
float channelLow = ma - atr
//@variable The upper bound of the channel.
float channelHigh = ma + atr

//@variable Is `true` when the `close` price is outside the current channel range, `false` otherwise.
bool priceOutsideChannel = close < channelLow or close > channelHigh

// Check if the `close` crossed outside the channel range, then analyze the past bars within the current range
if priceOutsideChannel and not priceOutsideChannel[1]
    //@variable A box that highlights consecutive past bars within the current channel's price window.
    box windowBox = box.new(
         bar_index, channelHigh, bar_index, channelLow, border_width = 2, bgcolor = color.new(color.gray, 85)
     )
    //@variable The lookback index for box adjustment. The `while` loop increments this value on each iteratio
    int i = 1
    // Use a `while` loop to look backward through close` prices. The loop iterates as long as the past `close
    // from `i` bars ago is between the current bar's `channelLow` and `channelHigh`.
    while close[i] >= channelLow and close[i] <= channelHigh
        // Adjust the left side of the box.
        windowBox.set_left(bar_index - i)
        // Add 1 to the `i` value to check the `close` from the next bar back on the next iteration.
        i += 1

// Plot the `channelLow` and `channelHigh` for visual reference.
plot(channelLow, "Channel low")
plot(channelHigh, "Channel high")
```

Note that:

- The left and right edges of boxes sit within the horizontal *center* of their respective bars, meaning that each drawing spans from the middle of the first consecutive bar to the middle of the last bar within each window.
- This script uses the `i` variable as a history-referencing index within the *conditional expression* the while loop checks on each iteration. The variable **does not** behave as a loop counter, as the iteration boundaries are **unknown**. The loop executes its local block repeatedly until the condition becomes `false`.

`for…in` loops

The for…in loop statement creates a *collection-controlled* loop, which uses the *contents* of a collection to control its iterations. This loop structure is often the preferred approach for looping through arrays, matrices, and maps.

A for…in loop traverses a collection *in order*, retrieving one of its stored items on each iteration. Therefore, the loop's boundaries depend directly on the number of *items* (array*elements*, matrix*rows*, or map*key-value pairs*).

Pine Script features *two* general forms of the for…in loop statement. The *first form* uses the following syntax:

```
[variables = | :=] for item in collection_id    statements | continue | break    return_expression
```

Where `item` is a *variable* that holds sequential values or references from the specified `collection_id`. The variable starts with the collection's *first item* and takes on successive items in order after each iteration. This form is convenient when a script must access values from an array or matrix iteratively but does not require the item's *index* in its calculations.

The *second form* has a slightly different syntax that includes a tuple in its *header*:

```
[variables = | :=] for [index, item] in collection_id    statements | continue | break    return_expression
```

Where `index` is a variable that contains the *index* or *key* of the retrieved `item`. This form is convenient when a task requires using a collection's items *and* their indices in iterative calculations. This form of the for…in loop is *required* when directly iterating through the contents of a map. See this section below for more information.

Refer to the Common characteristics section above for detailed information about the `variables`, `statements`, `continue`, `break`, and `return_expression` parts of the loop's syntax.

The iterative behavior of a for…in loop depends on the *type* of collection the header specifies as the `collection_id`:

- When using an array in the header, the loop performs *element-wise* iteration, meaning the retrieved `item` on each iteration is one of the array's *elements*.
- When using a matrix in the header, the loop performs *row-wise* iteration, which means that each `item` represents a *row array*.
- When using a map in the header, the loop performs *pair-wise* iteration, which retrieves a *key* and corresponding *value* on each iteration.

Looping through arrays

Pine scripts can iterate over the elements of arrays using any loop structure. However, the for…in loop is typically the most convenient because it automatically verifies the size of an array when controlling iterations. With other loop structures, programmers must carefully set the header's boundaries or conditions to *prevent* the loop from attempting to access an element at a *nonexistent* index.

For example, a for loop can access an array's elements using the counter variable as the lookup index in functions such as array.get(). However, programmers must ensure the counter always represents a *valid index* to prevent out-of-bounds errors. Additionally, if an array might be *empty*, programmers must set conditions to prevent the loop's execution entirely.

The code below shows a for loop whose counter boundaries depend on the number of elements in an array. If the array is empty, containing zero elements, the header's final counter value is na, which *prevents* iteration. Otherwise, the final value is *one less* than the array's size (i.e., the index of the last element):

```
for index = 0 to (array.size(myArray) == 0 ? na : array.size(myArray) - 1)
    element = array.get(myArray, index)
```

In contrast, a for…in loop automatically validates an array's size and *directly* accesses its elements, providing a more convenient solution than a traditional for loop. The line below achieves the *same effect* as the code above without requiring the programmer to define boundaries explicitly or use the array.get() function to access each element:

```
for element in myArray
```

The following example examines bars on a lower timeframe to gauge the strength of *intrabar* trends within each chart bar. The script uses a request.security_lower_tf() call to retrieve an array of intrabar hl2 prices from a calculated `lowerTimeframe`. Then, it uses a for…in loop to access each `price` within the `intrabarPrices` array and compare the value to the current close to calculate the bar's `strength`. The script plots the `strength` as columns in a separate pane:

Figure 29: image

```
//@version=6
indicator("`for element in array` demo", "Intrabar strength")

//@variable A valid timeframe closest to one-tenth of the current chart's timeframe, "1" if the timeframe is t
```

```
var string lowerTimeframe = timeframe.from_seconds(math.max(int(timeframe.in_seconds() / 10), 60))
//@variable An array of intrabar `hl2` prices calculated from the `lowerTimeframe`.
array<float> intrabarPrices = request.security_lower_tf("", lowerTimeframe, hl2)

//@variable The excess trend strength of `intrabarPrices`.
float strength = 0.0

// Loop directly through the `intrabarPrices` array. Each iteration's `price` represents an array element.
for price in intrabarPrices
    // Subtract 1 from the `strength` if the retrieved `price` is above the current bar's `close` price.
    if price > close
        strength -= 1
    // Add 1 to the `strength` if the retrieved `price` is below the current bar's `close` price.
    else if price < close
        strength += 1

//@variable Is `color.teal` when the `strength` is positive, `color.maroon` otherwise.
color strengthColor = strength > 0 ? color.teal : color.maroon

// Plot the `strength` as columns colored by the `strengthColor`.
plot(strength, "Intrabar strength", strengthColor, 1, plot.style_columns)
```

The second form of the for…in loop is a convenient solution when a script's calculations require accessing each element *and* corresponding index within an array:

```
for [index, element] in myArray
```

For example, suppose we want to display a *numerated* list of array elements within a label while excluding values at specific indices. We can use the second form of the for…in loop structure to accomplish this task. The simple script below declares a `stringArray` variable that references an array of predefined "string" values. On the last historical bar, the script uses a for…in loop to access each `index` and `element` in the `stringArray` to construct the `labelText`, which it uses in a label.new() call after the loop ends:

Figure 30: image

```
//@version=6
indicator("`for [index, item] in array` demo", "Array numerated output")

//@variable An array of "string" values to display as a numerated list.
```

```
var array<string> stringArray = array.from("First", "Second", "Third", "Before Last", "Last")

if barstate.islastconfirmedhistory
    //@variable A "string" modified within a loop to display within the `label`.
    string labelText = "Array values: \n"
    // Loop through the `stringArray`, accessing each `index` and corresponding `element`.
    for [index, element] in stringArray
        // Skip the third `element` (at `index == 2`) in the `labelText`. Include an "ELEMENT SKIPPED" message
        if index == 2
            labelText += "-- ELEMENT SKIPPED -- \n"
            continue
        labelText += str.tostring(index + 1) + ": " + element + "\n"
    // Display the `labelText` within a `label`.
    label.new(
         bar_index, 0, labelText, textcolor = color.white, size = size.huge,
         style = label.style_label_center, textalign = text.align_left
     )
```

Note that:

- This example adds 1 to the `index` in the str.tostring() call to start the numerated list with a value of `"1"`, because array indices always begins at 0.
- On the *third* loop iteration, when `index == 2`, the script adds an `"-- ELEMENT SKIPPED --"` message to the `labelText` instead of the retrieved `element` and uses the `continue` keyword to skip the remainder of the iteration. See this section above to learn more about loop keywords.

Let's explore an advanced example demonstrating the utility of for…in loops. The following indicator draws a fixed number of horizontal lines at calculated pivot high levels, and it analyzes the lines within a loop to determine which ones represent active (*uncrossed*) pivots.

Each time the script detects a new pivot high point, it creates a new line, *inserts* that line at the beginning of the `pivotLines` array, then removes the oldest element and deletes its ID. The script accesses each line within the array using a for…in loop, analyzing and modifying the properties of the `pivotLine` retrieved on each iteration. When the current high crosses above the `pivotLine`, the script changes its style to signify that it is no longer an active level. Otherwise, it extends the line's `x2` coordinate and uses its price to calculate the average *active* pivot value. The script also plots each pivot high value and the average active pivot for visual reference:

Figure 31: image

```
//@version=6
indicator("`for...in` loop with arrays demo", "Active high pivots", true, max_lines_count = 500)
```

```
//@variable The number of bars required on the left and right to confirm a pivot point.
int pivotBarsInput = input.int(5, "Pivot leg length", 1)
//@variable The number of recent pivot lines to analyze. Controls the size of the `pivotLines` array.
int maxRecentLines = input.int(20, "Maximum recent lines", 1, 500)

//@variable An array that acts as a queue holding the most recent pivot high lines.
var array<line> pivotLines = array.new<line>(maxRecentLines)
//@variable The pivot high price, or `na` if no pivot is found.
float highPivotPrice = ta.pivothigh(pivotBarsInput, pivotBarsInput)

if not na(highPivotPrice)
    //@variable The `chart.point` for the start of the line. Does not contain `time` information.
    firstPoint = chart.point.from_index(bar_index - pivotBarsInput, highPivotPrice)
    //@variable The `chart.point` for the end of each line. Does not contain `time` information.
    secondPoint = chart.point.from_index(bar_index, highPivotPrice)
    //@variable A horizontal line at the new pivot level.
    line hiPivotLine = line.new(firstPoint, secondPoint, width = 2, color = color.green)
    // Insert the `hiPivotLine` at the beginning of the `pivotLines` array.
    pivotLines.unshift(hiPivotLine)
    // Remove the oldest line from the array and delete its ID.
    line.delete(pivotLines.pop())

//@variable The sum of active pivot prices.
float activePivotSum = 0.0
//@variable The number of active pivot high levels.
int numActivePivots = 0

// Loop through the `pivotLines` array, directly accessing each `pivotLine` element.
for pivotLine in pivotLines
    //@variable The `x2` coordinate of the `pivotline`.
    int lineEnd = pivotLine.get_x2()
    // Move to the next `pivotline` in the array if the current line is inactive.
    if pivotLine.get_x2() < bar_index - 1
        continue
    //@variable The price value of the `pivotLine`.
    float pivotPrice = pivotLine.get_price(bar_index)
    // Change the style of the `pivotLine` and stop extending its display if the `high` is above the `pivotPri
    if high > pivotPrice
        pivotLine.set_color(color.maroon)
        pivotLine.set_style(line.style_dotted)
        pivotLine.set_width(1)
        continue
    // Extend the `pivotLine` and add the `pivotPrice` to the `activePivotSum` when the loop allows a full ite
    pivotLine.set_x2(bar_index)
    activePivotSum  += pivotPrice
    numActivePivots += 1

//@variable The average active pivot high value.
float avgActivePivot = activePivotSum / numActivePivots

// Plot crosses at the `highPivotPrice`, offset backward by the `pivotBarsInput`.
plot(highPivotPrice, "High pivot marker", color.green, 3, plot.style_cross, offset = -pivotBarsInput)
// Plot the `avgActivePivot` as a line with breaks.
plot(avgActivePivot, "Avg. active pivot", color.orange, 3, plot.style_linebr)
```

Note that:

- The loop in this example executes on *every bar* because it has to compare each active `pivotLine`'s price with the current high value, and it uses the prices to calculate the `avgActivePivot` on each bar.
- Pine Script features several ways to calculate averages, many of which *do not* require a loop. However, a loop is necessary in this example because the script uses information only available on the **current bar** to determine which

prices contribute toward the average.
- The *first* form of the for…in loop is the most convenient option in this example because we need direct access to the lines within the `pivotLines` array, but we do not need the corresponding *index* values.

Looping through matrices

Pine scripts can iterate over the contents of a matrix in several different ways. Unlike arrays, matrices use *two* indices to reference their elements because they store data in a *rectangular* format. The first index refers to *rows*, and the second refers to *columns*. If a programmer opts to use for or while loops to iterate through matrices instead of using for…in, they must carefully define the loop boundaries or conditions to avoid out-of-bounds errors.

This code block shows a for loop that performs *row-wise* iteration, looping through each *row index* in a matrix and using the value in a matrix.row() call to retrieve a row array. If the matrix is empty, the loop statement uses a final loop counter value of na to *prevent* iteration. Otherwise, the final counter is *one less* than the row count, which represents the *last* row index:

```
for rowIndex = 0 to (myMatrix.rows() == 0 ? na : myMatrix.rows() - 1)
    rowArray = myMatrix.row(rowIndex)
```

Note that:

- If we replace the matrix.rows() and matrix.row() calls with matrix.columns() and matrix.col(), the loop performs *column-wise* iteration instead.

The for…in loop statement is the more convenient approach to loop over and access the rows of a matrix in order, as it automatically validates the number of rows and retrieves an array of the current row's elements on each iteration:

```
for rowArray in myMatrix
```

When a script's calculations require access to each row from a matrix and its corresponding *index*, programmers can use the second form of the for…in loop:

```
for [rowIndex, rowArray] in myMatrix
```

Note that:

- The for…in loop only performs **row-wise** iteration on matrices. To *emulate* column-wise iteration, programmers can use a for…in loop on a transposed copy.

The following example displays a custom "string" representing the rows of a matrix with extra information, which it displays within a label. When the script executes on the last historical bar, it creates a 3x3 `randomMatrix` populated with random values. Then, using the first form of the for…in loop, the script iterates through each `row` in the `randomMatrix` to create a "string" representing the row's contents, its average, and whether the average is above 0.5, and it concatenates that "string" with the `labelText`. After the loop ends, the script creates a label displaying the `labelText` value:

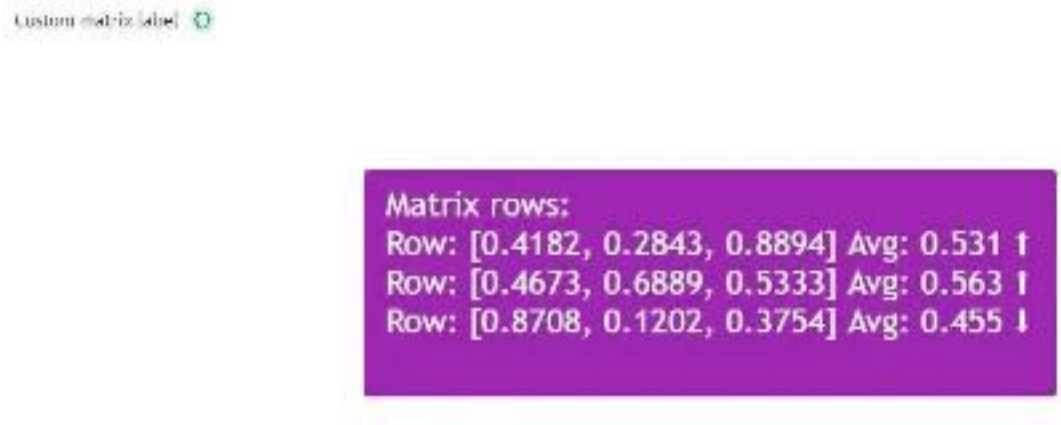

Figure 32: image

```
//@version=6
indicator("`for row in matrix` demo", "Custom matrix label")

//@variable Generates a random value between 0 and 1, rounded to 4 decimal places.
rand() =>
    math.round(math.random(), 4)

if barstate.islastconfirmedhistory
    //@variable A matrix of randomized values to format and display in a `label`.
    matrix<float> randomMatrix = matrix.new<float>()
    // Add a row of 9 randomized values and reshape the matrix to 3x3.
    randomMatrix.add_row(
         0, array.from(rand(), rand(), rand(), rand(), rand(), rand(), rand(), rand(), rand())
     )
    randomMatrix.reshape(3, 3)

    //@variable A custom "string" representation of `randomMatrix` information. Modified within a loop.
    string labelText = "Matrix rows: \n"

    // Loop through the rows in the `randomMatrix`.
    for row in randomMatrix
        //@variable The average element value within the `row`.
        float rowAvg = row.avg()
        //@variable An upward arrow when the `rowAvg` is above 0.5, a downward arrow otherwise.
        string directionChar = rowAvg > 0.5 ? " " : " "
        // Add a "string" representing the `row` array, its average, and the `directionChar` to the `labelText
        labelText += str.format("Row: {0} Avg: {1} {2}\n", row, rowAvg, directionChar)

    // Draw a `label` displaying the `labelText` on the current bar.
    label.new(
         bar_index, 0, labelText, color = color.purple, textcolor = color.white, size = size.huge,
         style = label.style_label_center, textalign = text.align_left
     )
```

Working with matrices often entails iteratively accessing their *elements*, not just their rows and columns, typically using *nested loops*. For example, this code block uses an outer for loop to iterate over row indices. The inner for loop iterates over column indices on *each* outer loop iteration and calls matrix.get() to access an element:

```
for rowIndex = 0 to (myMatrix.rows() == 0 ? na : myMatrix.rows() - 1)
    for columnIndex = 0 to myMatrix.columns() - 1
        element = myMatrix.get(rowIndex, columnIndex)
```

Alternatively, a more convenient approach for this type of task is to use nested for…in loops. The outer for…in loop in this code block retrieves each row array in a matrix, and the inner for…in statement loops through that array:

```
for rowArray in myMatrix
    for element in rowArray
```

The script below creates a 3x2 matrix, then accesses and modifies its elements within nested for…in loops. Both loops use the second form of the for…in statement to retrieve index values and corresponding items. The outer loop accesses a row index and row array from the matrix. The inner loop accesses each index and respective element from that array.

Within the nested loop's iterations, the script converts each `element` to a "string" and initializes a table cell at the `rowIndex` row and `colIndex` column. Then, it uses the loop header variables within matrix.set() to update the matrix element. After the outer loop terminates, the script displays a "string" representation of the *updated*matrix within a label:

```
//@version=6
indicator("Nested `for...in` loops on matrices demo")

if barstate.islastconfirmedhistory
    //@variable  A matrix containing numbers to display.
    matrix<float> displayNumbers = matrix.new<float>()
    // Populate the `displayNumbers` matrix and reshape to 3x2.
```

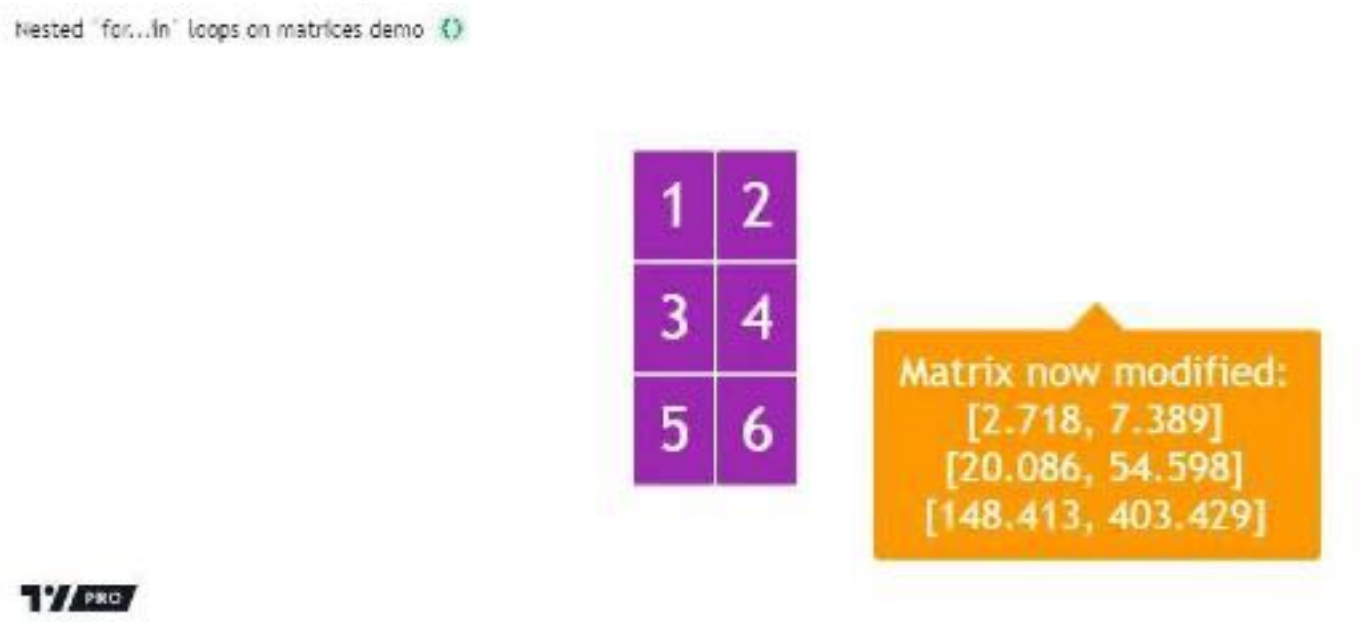

Figure 33: image

```
displayNumbers.add_row(0, array.from(1, 2, 3, 4, 5, 6))
displayNumbers.reshape(3, 2)

//@variable A table that displays the elements of the `displayNumbers` before modification.
table displayTable = table.new(
     position = position.middle_center, columns = displayNumbers.columns(), rows = displayNumbers.rows(),
     bgcolor = color.purple, border_color = color.white, border_width = 2
 )

// Loop through the `displayNumbers`, retrieving the `rowIndex` and the current `row`.
for [rowIndex, row] in displayNumbers
    // Loop through the current `row` on each outer loop iteration to retrieve the `colIndex` and `element
    for [colIndex, element] in row
        // Initialize a table cell at the `rowIndex` row and `colIndex` column displaying the current `ele
        displayTable.cell(column = colIndex, row = rowIndex, text = str.tostring(element),
             text_color = color.white, text_size = size.huge
         )
        // Update the `displayNumbers` value at the `rowIndex` and `colIndex`.
        displayNumbers.set(rowIndex, colIndex, math.round(math.exp(element), 3))

// Draw a `label` to display a "string" representation of the updated `displayNumbers` matrix.
label.new(
     x = bar_index, y = 0, text = "Matrix now modified: \n" + str.tostring(displayNumbers), color = color.
     textcolor = color.white, size = size.huge, style = label.style_label_up
 )
```

Looping through maps

The for…in loop statement is the primary, most convenient approach for iterating over the data within Pine Script maps.

Unlike arrays and matrices, maps are *unordered collections* that store data in *key-value pairs*. Rather than traversing an internal lookup index, a script references the *keys* from the pairs within a map to access its *values*. Therefore, when looping through a map, scripts must perform *pair-wise* iteration, which entails retrieving key-value pairs across iterations rather than indexed elements or rows.

Note that:

- Although maps are unordered collections, Pine Script internally tracks the *insertion order* of their key-value pairs.

One way to access the data from a map is to use the map.keys() function, which returns an array containing all the *keys* from the map, sorted in their insertion order. A script can use the for…in structure to loop through the array of keys and call map.get() to retrieve corresponding values:

```
for key in myMap.keys()
    value = myMap.get(key)
```

However, the more convenient, *recommended* approach is to loop through a map directly *without* creating new arrays. To loop through a map directly, use the second form of the for…in loop statement. Using this loop with a map creates a tuple containing a *key* and respective *value* on each iteration. As when looping through a map.keys() array, this *direct*for…in loop iterates through a map's contents in their insertion order:

```
for [key, value] in myMap
```

Note that:

- The second form of the for…in loop is the **only** way to iterate *directly* through a map. A script cannot directly loop through this collection type without retrieving a key and value on each iteration.

Let's consider a simple example demonstrating how a for…in loop works on a map. When the script below executes on the last historical bar, it declares a `simpleMap` variable with an assigned map of "string" keys and "float" values. The script puts the keys from the `newKeys` array into the collection with corresponding random values. It then uses a for…in loop to iterate through the key-value pairs from the `simpleMap` and construct the `displayText`. After the loop ends, the script shows the `displayText` within a label to visualize the result:

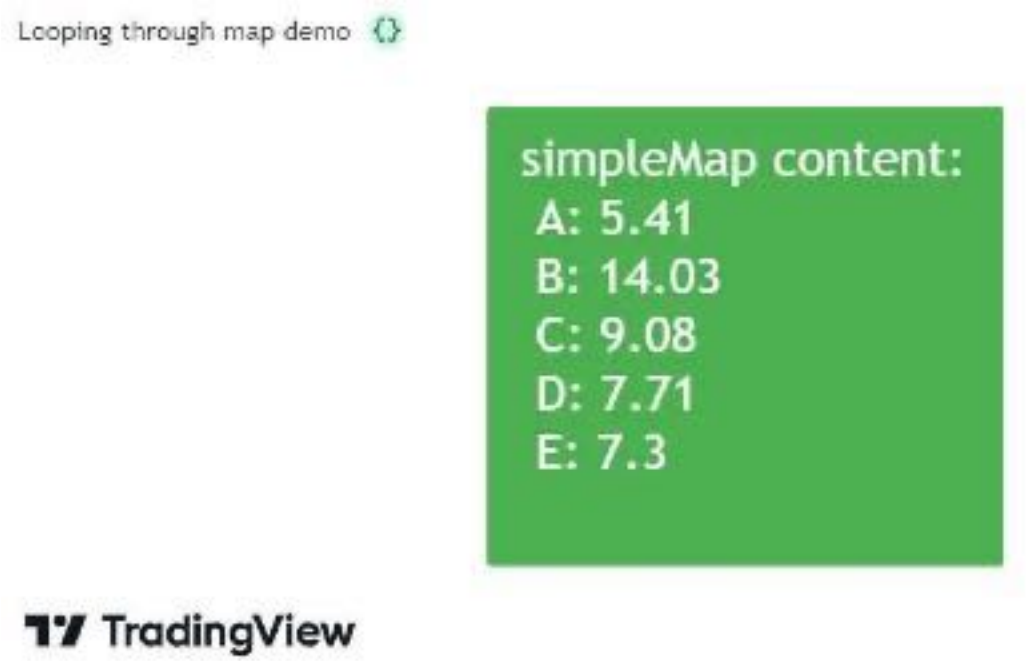

Figure 34: image

```
//@version=6
indicator("Looping through map demo")

if barstate.islastconfirmedhistory
    //@variable A map of "string" keys and "float" values to render within a `label`.
    map<string, float> simpleMap = map.new<string, float>()

    //@variable An array of "string" values representing the keys to put into the map.
    array<string> newKeys = array.from("A", "B", "C", "D", "E")
    // Put key-value pairs into the `simpleMap`.
    for key in newKeys
        simpleMap.put(key, math.random(1, 20))

    //@variable A "string" representation of the `simpleMap` contents. Modified within a loop.
```

```
string displayText = "simpleMap content: \n "

// Loop through each key-value pair within the `simpleMap`.
for [key, value] in simpleMap
    // Add a "string" representation of the pair to the `displayText`.
    displayText += key + ": " + str.tostring(value, "#.##") + "\n "

// Draw a `label` showing the `displayText` on the current bar.
label.new(
     x = bar_index, y = 0, text = displayText, color = color.green, textcolor = color.white,
     size = size.huge, textalign = text.align_left, style = label.style_label_center
 )
```

Note that:

- This script utilizes both forms of the for...in loop statement. The first loop iterates through the "string" elements of the `newKeys` array to put key-value pairs into the `simpleMap`, and the second iterates directly through the map's key-value pairs to construct the custom `displayText`.

[Previous

Conditional structures](#conditional-structures)[Next

Type system](#type-system) User Manual/Language/Type system

Type system

Introduction

The Pine Script™ type system determines the compatibility of a script's values with various functions and operations. While it's possible to write simple scripts without knowing anything about the type system, a reasonable understanding of it is necessary to achieve any degree of proficiency with the language, and an in-depth knowledge of its subtleties allows programmers to harness its full potential.

Pine Script™ uses types to classify all values, and it uses qualifiers to determine whether values and references are constant, established on the first script execution, or dynamic across executions. This system applies to all Pine values and references, including literals, variables, expressions, function returns, and function arguments.

The type system closely intertwines with Pine's execution model and time series concepts. Understanding all three is essential for making the most of the power of Pine Script™.

Qualifiers

Pine Script™ *qualifiers* identify when values are accessible to a script:

- Values and references qualified as const are established at compile time (i.e., when saving the script in the Pine Editor or adding it to the chart).
- Values qualified as input are established at input time (i.e., when confirming values based on user input, primarily from the "Settings/Inputs" tab).
- Values qualified as simple are established at bar zero (i.e., the first script execution).
- Values qualified as series can change throughout the script's executions.

Pine Script™ bases the dominance of type qualifiers on the following hierarchy: **const < input < simple < series**, where "const" is the *weakest* qualifier and "series" is the *strongest*. The qualifier hierarchy translates to this rule: whenever a variable, function, or operation is compatible with a specific qualified type, values with *weaker* qualifiers are also allowed.

Scripts always qualify their expressions' returned types based on the *dominant qualifier* in their calculations. For example, evaluating an expression that involves "input" and "series" values will return a value qualified as "series". Furthermore, scripts **cannot** change a value's qualifier to one that's *lower* on the hierarchy. If a value acquires a *stronger* qualifier (e.g., a value initially inferred as "simple" becomes "series" later in the script's executions), that state is irreversible.

It's important to note that "series" values are the **only** ones that can change across script executions, including those from various built-ins, such as close and volume, as well as the results of expressions involving "series" values. All values qualified as "const", "input", or "simple" remain consistent across all script executions.

const

Values or references qualified as "const" are established at *compile time*, before the script starts its executions. Compilation initially occurs when saving a script in the Pine Editor, which does not require it to run on a chart. Values or references with the "const" qualifier *never change* between script executions, not even on the first execution.

All *literal* values and the results returned by expressions involving only values qualified as "const" automatically adopt the "const" qualifier.

These are some examples of literal values:

- *literal int*: `1`, `-1`, `42`
- *literal float*: `1.`, `1.0`, `3.14`, `6.02E-23`, `3e8`
- *literal bool*: `true`, `false`
- *literal color*: `#FF55C6`, `#FF55C6ff`
- *literal string*: `"A text literal"`, `"Embedded single quotes 'text'"`, `'Embedded double quotes "text"'`

Our Style guide recommends using uppercase SNAKE_CASE to name "const" variables for readability. While not a requirement, one can also use the var keyword when declaring "const" variables so the script only initializes them on the *first bar* of the dataset. See this section of our User Manual for more information.

Below is an example that uses "const" values within the indicator() and plot() functions, which both require a value of the "const string" qualified type as their `title` argument:

```
//@version=6

// The following global variables are all of the "const string" qualified type:

//@variable The title of the indicator.
INDICATOR_TITLE = "const demo"
//@variable The title of the first plot.
var PLOT1_TITLE = "High"
//@variable The title of the second plot.
const string PLOT2_TITLE = "Low"
//@variable The title of the third plot.
PLOT3_TITLE = "Midpoint between " + PLOT1_TITLE + " and " + PLOT2_TITLE

indicator(INDICATOR_TITLE, overlay = true)

plot(high, PLOT1_TITLE)
plot(low, PLOT2_TITLE)
plot(hl2, PLOT3_TITLE)
```

The following example will raise a compilation error since it uses syminfo.ticker, which returns a "simple" value because it depends on chart information that's only accessible after the script's first execution:

```
//@version=6

//@variable The title in the `indicator()` call.
var NAME = "My indicator for " + syminfo.ticker

indicator(NAME, "", true) // Causes an error because `NAME` is qualified as a "simple string".
plot(close)
```

The const keyword allows the declaration of variables and parameters with constant *value assignments*. Declaring a variable with this keyword instructs the script to forbid using *reassignment* and *compound assignment* operations on it. For example, this script declares the `myVar` variable with the keyword, then attempts to assign a new "float" value to the variable with the addition assignment operator (+=), resulting in a compilation error:

```
//@version=6
indicator("Cannot reassign const demo")

//@variable A "float" variable declared as `const`, preventing reassignment.
const float myVar = 0.0
```

```
myVar += 1.0 // Causes an error. Reassignment and compound assignments are not allowed on `const` variables.

plot(myVar)
```

It's crucial to note that declaring a variable with the const keyword forces it to maintain a constant reference to the value returned by a specific expression, but that *does not* necessarily define the nature of the assigned value. For example, a script can declare a const variable that maintains a constant reference to an expression returning the *ID* of a *special type*. Although the script cannot *reassign* the variable, the assigned ID is a "series" value:

```
//@version=6
indicator("Constant reference to 'series' ID demo")

//@variable A `label` variable declared as `const`, preventing reassignment.
//          Although the reference is constant, the ID of the `label` is a "series" value.
const label myVar = label.new(bar_index, close)
```

input

Most values qualified as "input" are established after initialization via the `input.*()` functions. These functions produce values that users can modify within the "Inputs" tab of the script's settings. When one changes any of the values in this tab, the script *restarts* from the beginning of the chart's history to ensure its inputs are consistent throughout its executions. Some of Pine's built-in variables, such as chart.bg_color also use the "input" qualifier, even though `input.*()` functions do not return them, since the script receives their values at *input time*.

The following script plots the value of a `sourceInput` from the `symbolInput` and `timeframeInput` context. The request.security() call is valid in this script since its `symbol` and `timeframe` parameters allow "series string" arguments by default, meaning they can also accept "input string" values because the "input" qualifier is *lower* on the hierarchy:

```
//@version=6
indicator("input demo", overlay = true)

//@variable The symbol to request data from. Qualified as "input string".
symbolInput = input.symbol("AAPL", "Symbol")
//@variable The timeframe of the data request. Qualified as "input string".
timeframeInput = input.timeframe("D", "Timeframe")
//@variable The source of the calculation. Qualified as "series float".
sourceInput = input.source(close, "Source")

//@variable The `sourceInput` value from the requested context. Qualified as "series float".
requestedSource = request.security(symbolInput, timeframeInput, sourceInput)

plot(requestedSource)
```

simple

Values qualified as "simple" are available on the first script execution, and they remain consistent across subsequent executions.

Users can explicitly define variables and parameters that accept "simple" values by including the `simple` keyword in their declaration.

Many built-in variables return "simple" qualified values because they depend on information that a script can only obtain once it starts running on the chart. Additionally, many built-in functions require "simple" arguments that do not change over time. Wherever a script allows "simple" values, it can also accept values qualified as "input" or "const".

This script highlights the background to warn users that they're using a non-standard chart type. It uses the value of chart.is_standard to calculate the `isNonStandard` variable, then uses that variable's value to calculate a `warningColor` that also references a "simple" value. The `color` parameter of bgcolor() allows a "series color" argument, meaning it can also accept a "simple color" value since "simple" is lower on the hierarchy:

```
//@version=6
indicator("simple demo", overlay = true)

//@variable Is `true` when the current chart is non-standard. Qualified as "simple bool".
isNonStandard = not chart.is_standard
//@variable Is orange when the the current chart is non-standard. Qualified as "simple color".
```

```
simple color warningColor = isNonStandard ? color.new(color.orange, 70) : na

// Colors the chart's background to warn that it's a non-standard chart type.
bgcolor(warningColor, title = "Non-standard chart color")
```

series

Values qualified as "series" provide the most flexibility in scripts since they can change across executions.

Users can explicitly define variables and parameters that accept "series" values by including the `series` keyword in their declarations.

Built-in variables such as open, high, low, close, volume, time, and bar_index, and the result from any expression using such built-ins, are qualified as "series". The result of any function or operation that returns a dynamic value will always be a "series", as will the results from using the history-referencing operator [] to access historical values. Wherever a script allows "series" values, it will also accept values with any other qualifier, as "series" is the *highest* qualifier on the hierarchy.

This script displays the highest and lowest value of a `sourceInput` over `lengthInput` bars. The values assigned to the `highest` and `lowest` variables are of the "series float" qualified type, as they can change throughout the script's execution:

```
//@version=6
indicator("series demo", overlay = true)

//@variable The source value to calculate on. Qualified as "series float".
series float sourceInput = input.source(close, "Source")
//@variable The number of bars in the calculation. Qualified as "input int".
lengthInput = input.int(20, "Length")

//@variable The highest `sourceInput` value over `lengthInput` bars. Qualified as "series float".
series float highest = ta.highest(sourceInput, lengthInput)
//@variable The lowest `sourceInput` value over `lengthInput` bars. Qualified as "series float".
lowest = ta.lowest(sourceInput, lengthInput)

plot(highest, "Highest source", color.green)
plot(lowest, "Lowest source", color.red)
```

Types

Pine Script™ *types* classify values and determine the functions and operations they're compatible with. They include:

- The fundamental types: int, float, bool, color, and string
- The special types: plot, hline, line, linefill, box, polyline, label, table, chart.point, array, matrix, and map
- User-defined types (UDTs)
- Enums
- void

Fundamental types refer to the underlying nature of a value, e.g., a value of 1 is of the "int" type, 1.0 is of the "float" type, "AAPL" is of the "string" type, etc. Special types and user-defined types utilize *IDs* that refer to objects of a specific type. For example, a value of the "label" type contains an ID that acts as a *pointer* referring to a "label" object. The "void" type refers to the output from a function or method that does not return a usable value.

Pine Script™ can automatically convert values from some types into others. The auto-casting rules are: **int → float → bool**. See the Type casting section of this page for more information.

In most cases, Pine Script™ can automatically determine a value's type. However, we can also use type keywords to *explicitly* specify types for readability and for code that requires explicit definitions (e.g., declaring a variable assigned to na). For example:

```
//@version=6
indicator("Types demo", overlay = true)

//@variable A value of the "const string" type for the `ma` plot's title.
string MA_TITLE = "MA"

//@variable A value of the "input int" type. Controls the length of the average.
```

```
int lengthInput = input.int(100, "Length", minval = 2)

//@variable A "series float" value representing the last `close` that crossed over the `ma`.
var float crossValue = na

//@variable A "series float" value representing the moving average of `close`.
float ma = ta.sma(close, lengthInput)
//@variable A "series bool" value that's `true` when the `close` crosses over the `ma`.
bool crossUp = ta.crossover(close, ma)
//@variable A "series color" value based on whether `close` is above or below its `ma`.
color maColor = close > ma ? color.lime : color.fuchsia

// Update the `crossValue`.
if crossUp
    crossValue := close

plot(ma, MA_TITLE, maColor)
plot(crossValue, "Cross value", style = plot.style_circles)
plotchar(crossUp, "Cross Up", " ", location.belowbar, size = size.small)
```

int

Values of the "int" type represent integers, i.e., whole numbers without any fractional quantities.

Integer literals are numeric values written in *decimal* notation. For example:

```
1
-1
750
```

Built-in variables such as bar_index, time, timenow, dayofmonth, and strategy.wintrades all return values of the "int" type.

float

Values of the "float" type represent floating-point numbers, i.e., numbers that can contain whole and fractional quantities.

Floating-point literals are numeric values written with a `.` delimiter. They may also contain the symbol `e` or `E` (which means "10 raised to the power of X", where X is the number after the `e` or `E` symbol). For example:

```
3.14159    // Rounded value of Pi ( )
- 3.0
6.02e23    // 6.02 * 10^23 (a very large value)
1.6e-19    // 1.6 * 10^-19 (a very small value)
```

The internal precision of "float" values in Pine Script™ is 1e-16.

Built-in variables such as close, hlcc4, volume, ta.vwap, and strategy.position_size all return values of the "float" type.

bool

Values of the "bool" type represent the truth value of a comparison or condition, which scripts can use in conditional structures and other expressions.

There are only two literals that represent boolean values:

```
true    // true value
false   // false value
```

A `bool` variable can never be na, and any conditional structure that can return `na` will return `false` instead. For example, an if condition returns `bool` values, when the condition is not met and the `else` block is not specified, it will returns `false`.

Built-in variables such as barstate.isfirst, chart.is_heikinashi, session.ismarket, and timeframe.isdaily all return values of the "bool" type.

color

Color literals have the following format: `#RRGGBB` or `#RRGGBBAA`. The letter pairs represent *hexadecimal* values between `00` and `FF` (0 to 255 in decimal) where:

- `RR`, `GG` and `BB` pairs respectively represent the values for the color's red, green and blue components.
- `AA` is an optional value for the color's opacity (or *alpha* component) where `00` is invisible and `FF` opaque. When the literal does not include an `AA` pair, the script treats it as fully opaque (the same as using `FF`).
- The hexadecimal letters in the literals can be uppercase or lowercase.

These are examples of "color" literals:

```
#000000      // black color
#FF0000      // red color
#00FF00      // green color
#0000FF      // blue color
#FFFFFF      // white color
#808080      // gray color
#3ff7a0      // some custom color
#FF000080    // 50% transparent red color
#FF0000ff    // same as #FF0000, fully opaque red color
#FF000000    // completely transparent red color
```

Pine Script™ also has built-in color constants, including color.green, color.red, color.orange, color.blue (the default color in `plot*()` functions and many of the default color-related properties in drawing types), etc.

When using built-in color constants, it is possible to add transparency information to them via the color.new() function.

Note that when specifying red, green or blue components in `color.*()` functions, we use "int" or "float" arguments with values between 0 and 255. When specifying transparency, we use a value between 0 and 100, where 0 means fully opaque and 100 means completely transparent. For example:

```
//@version=6
indicator("Shading the chart's background", overlay = true)

//@variable A "const color" value representing the base for each day's color.
color BASE_COLOR = color.rgb(0, 99, 165)

//@variable A "series int" value that modifies the transparency of the `BASE_COLOR` in `color.new()`.
int transparency = 50 + int(40 * dayofweek / 7)

// Color the background using the modified `BASE_COLOR`.
bgcolor(color.new(BASE_COLOR, transparency))
```

See the User Manual's page on colors for more information on using colors in scripts.

string

Values of the "string" type represent sequences of letters, numbers, symbols, spaces, and other characters.

String literals in Pine are characters enclosed in single or double quotation marks. For example:

```
"This is a string literal using double quotes."
'This is a string literal using single quotes.'
```

Single and double quotation marks are functionally equivalent in Pine Script™. A "string" enclosed within double quotation marks can contain any number of single quotation marks and vice versa:

```
"It's an example"
'The "Star" indicator'
```

Scripts can *escape* the enclosing delimiter in a "string" using the backslash character (\). For example:

```
'It\'s an example'
"The \"Star\" indicator"
```

We can create "string" values containing the new line escape character (`\n`) for displaying multi-line text with `plot*()` and `log.*()` functions and objects of drawing types. For example:

```
"This\nString\nHas\nOne\nWord\nPer\nLine"
```

We can use the + operator to concatenate "string" values:

```
"This is a " + "concatenated string."
```

The built-ins in the `str.*()` namespace create "string" values using specialized operations. For instance, this script creates a *formatted string* to represent "float" price values and displays the result using a label:

```
//@version=6
indicator("Formatted string demo", overlay = true)

//@variable A "series string" value representing the bar's OHLC data.
string ohlcString = str.format("Open: {0}\nHigh: {1}\nLow: {2}\nClose: {3}", open, high, low, close)

// Draw a label containing the `ohlcString`.
label.new(bar_index, high, ohlcString, textcolor = color.white)
```

See our User Manual's page on Text and shapes for more information about displaying "string" values from a script.

Built-in variables such as syminfo.tickerid, syminfo.currency, and timeframe.period return values of the "string" type.

plot and hline

Pine Script™'s plot() and hline() functions return IDs that respectively reference instances of the "plot" and "hline" types. These types display calculated values and horizontal levels on the chart, and one can assign their IDs to variables for use with the built-in fill() function.

For example, this script plots two EMAs on the chart and fills the space between them using a fill() call:

```
//@version=6
indicator("plot fill demo", overlay = true)

//@variable A "series float" value representing a 10-bar EMA of `close`.
float emaFast = ta.ema(close, 10)
//@variable A "series float" value representing a 20-bar EMA of `close`.
float emaSlow = ta.ema(close, 20)

//@variable The plot of the `emaFast` value.
emaFastPlot = plot(emaFast, "Fast EMA", color.orange, 3)
//@variable The plot of the `emaSlow` value.
emaSlowPlot = plot(emaSlow, "Slow EMA", color.gray, 3)

// Fill the space between the `emaFastPlot` and `emaSlowPlot`.
fill(emaFastPlot, emaSlowPlot, color.new(color.purple, 50), "EMA Fill")
```

It's important to note that unlike other special types, there is no `plot` or `hline` keyword in Pine to explicitly declare a variable's type as "plot" or "hline".

Users can control where their scripts' plots display via the variables in the `display.*` namespace and a `plot*()` function's `force_overlay` parameter. Additionally, one script can use the values from another script's plots as *external inputs* via the input.source() function (see our User Manual's section on source inputs).

Drawing types

Pine Script™ drawing types allow scripts to create custom drawings on charts. They include the following: line, linefill, box, polyline, label, and table.

Each type also has a namespace containing all the built-ins that create and manage drawing instances. For example, the following `*.new()` constructors create new objects of these types in a script: line.new(), linefill.new(), box.new(), polyline.new(), label.new(), and table.new().

Each of these functions returns an *ID* which is a reference that uniquely identifies a drawing object. IDs are always qualified as "series", meaning their qualified types are "series line", "series label", etc. Drawing IDs act like pointers, as each ID references a specific instance of a drawing in all the functions from that drawing's namespace. For instance, the ID of a line returned by a line.new() call is used later to refer to that specific object once it's time to delete it with line.delete().

Chart points

Chart points are special types that represent coordinates on the chart. Scripts use the information from chart.point objects to determine the chart locations of lines, boxes, polylines, and labels.

Objects of this type contain three *fields*: `time`, `index`, and `price`. Whether a drawing instance uses the `time` or `price` field from a chart.point as an x-coordinate depends on the drawing's `xloc` property.

We can use any of the following functions to create chart points in a script:

- chart.point.new() - Creates a new chart.point with a specified `time`, `index`, and `price`.
- chart.point.now() - Creates a new chart.point with a specified `price` y-coordinate. The `time` and `index` fields contain the time and bar_index of the bar the function executes on.
- chart.point_from_index() - Creates a new chart.point with an `index` x-coordinate and `price` y-coordinate. The `time` field of the resulting instance is na, meaning it will not work with drawing objects that use an `xloc` value of xloc.bar_time.
- chart.point.from_time() - Creates a new chart.point with a `time` x-coordinate and `price` y-coordinate. The `index` field of the resulting instance is na, meaning it will not work with drawing objects that use an `xloc` value of xloc.bar_index.
- chart.point.copy() - Creates a new chart.point containing the same `time`, `index`, and `price` information as the `id` in the function call.

This example draws lines connecting the previous bar's high to the current bar's low on each chart bar. It also displays labels at both points of each line. The line and labels get their information from the `firstPoint` and `secondPoint` variables, which reference chart points created using chart.point_from_index() and chart.point.now():

```
//@version=6
indicator("Chart points demo", overlay = true)

//@variable A new `chart.point` at the previous `bar_index` and `high`.
firstPoint = chart.point.from_index(bar_index - 1, high[1])
//@variable A new `chart.point` at the current bar's `low`.
secondPoint = chart.point.now(low)

// Draw a new line connecting coordinates from the `firstPoint` and `secondPoint`.
// This line uses the `index` fields from the points as x-coordinates.
line.new(firstPoint, secondPoint, color = color.purple, width = 3)
// Draw a label at the `firstPoint`. Uses the point's `index` field as its x-coordinate.
label.new(
     firstPoint, str.tostring(firstPoint.price), color = color.green,
     style = label.style_label_down, textcolor = color.white
 )
// Draw a label at the `secondPoint`. Uses the point's `index` field as its x-coordinate.
label.new(
     secondPoint, str.tostring(secondPoint.price), color = color.red,
     style = label.style_label_up, textcolor = color.white
 )
```

Collections

Collections in Pine Script™ (arrays, matrices, and maps) utilize reference IDs, much like other special types (e.g., labels). The type of the ID defines the type of *elements* the collection will contain. In Pine, we specify array, matrix, and map types by appending a type template to the array, matrix, or map keywords:

- `array<int>` defines an array containing "int" elements.
- `array<label>` defines an array containing "label" IDs.
- `array<UDT>` defines an array containing IDs referencing objects of a user-defined type (UDT).
- `matrix<float>` defines a matrix containing "float" elements.
- `matrix<UDT>` defines a matrix containing IDs referencing objects of a user-defined type (UDT).
- `map<string, float>` defines a map containing "string" keys and "float" values.
- `map<int, UDT>` defines a map containing "int" keys and IDs of user-defined type (UDT) instances as values.

For example, one can declare an "int" array with a single element value of 10 in any of the following, equivalent ways:

```
a1 = array.new<int>(1, 10)
array<int> a2 = array.new<int>(1, 10)
```

```
a3 = array.from(10)
array<int> a4 = array.from(10)
```

Note that:

- The `int[]` syntax can also specify an array of "int" elements, but its use is discouraged. No equivalent exists to specify the types of matrices or maps in that way.
- Type-specific built-ins exist for arrays, such as array.new_int(), but the more generic array.new form is preferred, which would be `array.new<int>()` to create an array of "int" elements.

User-defined types

The type keyword allows the creation of *user-defined types* (UDTs) from which scripts can create objects. UDTs are composite types; they contain an arbitrary number of *fields* that can be of any type, including other user-defined types.

The syntax to declare a user-defined type is:

```
[export ]type <UDT_identifier>    <field_type> <field_name>[ = <value>]    ...
```

where:

- export is the keyword that a library script uses to export the user-defined type. To learn more about exporting UDTs, see our User Manual's Libraries page.
- `<UDT_identifier>` is the name of the user-defined type.
- `<field_type>` is the type of the field.
- `<field_name>` is the name of the field.
- `<value>` is an optional default value for the field, which the script will assign to it when creating new objects of that UDT. If one does not provide a value, the field's default is na. The same rules as those governing the default values of parameters in function signatures apply to the default values of fields. For example, a UDT's default values cannot use results from the history-referencing operator [] or expressions.

This example declares a `pivotPoint` UDT with an "int" `pivotTime` field and a "float" `priceLevel` field that will respectively hold time and price information about a calculated pivot:

```
//@type             A user-defined type containing pivot information.
    //@field pivotTime  Contains time information about the pivot.
    //@field priceLevel Contains price information about the pivot.
    type pivotPoint
        int   pivotTime
        float priceLevel
```

User-defined types support *type recursion*, i.e., the fields of a UDT can reference objects of the same UDT. Here, we've added a `nextPivot` field to our previous `pivotPoint` type that references another `pivotPoint` instance:

```
//@type             A user-defined type containing pivot information.
    //@field pivotTime  Contains time information about the pivot.
    //@field priceLevel Contains price information about the pivot.
    //@field nextPivot  A `pivotPoint` instance containing additional pivot information.
    type pivotPoint
        int        pivotTime
        float      priceLevel
        pivotPoint nextPivot
```

Scripts can use two built-in methods to create and copy UDTs: `new()` and `copy()`. See our User Manual's page on Objects to learn more about working with UDTs.

Enum types

The enum keyword allows the creation of an *enum*, otherwise known as an *enumeration*, *enumerated type*, or *enum type*. An enum is a unique type construct containing distinct, named fields representing *members* (i.e., possible values) of the type. Enums allow programmers to control the values accepted by variables, conditional expressions, and collections, and they facilitate convenient dropdown input creation with the input.enum() function.

The syntax to declare an enum is as follows:

```
[export ]enum <enumName>    <field_1>[ = <title_1>]    <field_2>[ = <title_2>]    ...    <field_N>[ = <title_N
```

where:

- export is the optional keyword allowing a library to export the enum for use in other scripts. See this section to learn more about exporting enum types.
- `<enumName>` is the name of the enum type. Scripts can use the enum's name as the type keyword in variable declarations and type templates.
- `<field_*>` is the name of an enum field, representing a named member (value) of the `enumName` type. Each field must have a unique name that does not match the name or title of any other field in the enum. To retrieve an enum member, reference its field name using dot notation syntax (i.e., `enumName.fieldName`).
- `<title_*>` is a "const string" title assigned to a field. If one does not specify a title, the field's title is the "string" representation of its name. The input.enum() function displays field titles within its dropdown in the script's "Settings/Inputs" tab. Users can also retrieve a field's title with the str.tostring() function. As with field names, each field's title must not match the name or title of any other field in the enum.

This example declares an `maChoice` enum. Each field within this declaration represents a distinct member of the `maChoice` enum type:

```
//@enum       An enumeration of named values for moving average selection.
//@field sma  Selects a Simple Moving Average.
//@field ema  Selects an Exponential Moving Average.
//@field wma  Selects a Weighted Moving Average.
//@field hma  Selects a Hull Moving Average.
enum maChoice
    sma  = "Simple Moving Average"
    ema  = "Exponential Moving Average"
    wma  = "Weighted Moving Average"
    hma  = "Hull Moving Average"
```

Note that:

- All the enum's possible values are available upon the *first* script execution and do not change across subsequent executions. Hence, they automatically adopt the simple qualifier.

The script below uses the `maChoice` enum within an input.enum() call to create a *dropdown* input in the "Settings/Inputs" tab that displays all the field titles. The `maInput` value represents the member of the enum that corresponds to the user-selected title. The script uses the selected member within a switch structure to determine the built-in moving average it calculates:

```
//@version=6
indicator("Enum types demo", overlay = true)

//@enum       An enumeration of named values for moving average selection.
//@field sma  Selects a Simple Moving Average.
//@field ema  Selects an Exponential Moving Average.
//@field wma  Selects a Weighted Moving Average.
//@field hma  Selects a Hull Moving Average.
enum maChoice
    sma  = "Simple Moving Average"
    ema  = "Exponential Moving Average"
    wma  = "Weighted Moving Average"
    hma  = "Hull Moving Average"

//@variable The `maChoice` member representing a selected moving average name.
maChoice maInput = input.enum(maChoice.sma, "Moving average type")
//@variable The length of the moving average.
int lengthInput = input.int(20, "Length", 1, 4999)

//@variable The moving average selected by the `maInput`.
float selectedMA = switch maInput
    maChoice.sma => ta.sma(close, lengthInput)
    maChoice.ema => ta.ema(close, lengthInput)
    maChoice.wma => ta.wma(close, lengthInput)
    maChoice.hma => ta.hma(close, lengthInput)

// Plot the `selectedMA`.
plot(selectedMA, "Selected moving average", color.teal, 3)
```

See the Enums page and the Enum input section of the Inputs page to learn more about using enums and enum inputs.

void

There is a "void" type in Pine Script™. Functions having only side-effects and returning no usable result return the "void" type. An example of such a function is alert(); it does something (triggers an alert event), but it returns no usable value.

Scripts cannot use "void" results in expressions or assign them to variables. No `void` keyword exists in Pine Script™ since one cannot declare a variable of the "void" type.

na value

There is a special value in Pine Script™ called na, which is an acronym for *not available*. We use na to represent an undefined value from a variable or expression. It is similar to `null` in Java and `None` in Python.

Scripts can automatically cast na values to almost any type. However, in some cases, the compiler cannot infer the type associated with an na value because more than one type-casting rule may apply. For example:

```
// Compilation error!
myVar = na
```

The above line of code causes a compilation error because the compiler cannot determine the nature of the `myVar` variable, i.e., whether the variable will reference numeric values for plotting, string values for setting text in a label, or other values for some other purpose later in the script's execution.

To resolve such errors, we must explicitly declare the type associated with the variable. Suppose the `myVar` variable will reference "float" values in subsequent script iterations. We can resolve the error by declaring the variable with the float keyword:

```
float myVar = na
```

or by explicitly casting the na value to the "float" type via the float() function:

```
myVar = float(na)
```

To test if the value from a variable or expression is na, we call the na() function, which returns `true` if the value is undefined. For example:

```
//@variable Is 0 if the `myVar` is `na`, `close` otherwise.
float myClose = na(myVar) ? 0 : close
```

Do not use the == comparison operator to test for na values, as scripts cannot determine the equality of an undefined value:

```
//@variable Returns the `close` value. The script cannot compare the equality of `na` values, as they're undef
float myClose = myVar == na ? 0 : close
```

Best coding practices often involve handling na values to prevent undefined values in calculations.

We can ensure the expression also returns an actionable value on the first bar by replacing the undefined past value with a value from the current bar. This line of code uses the nz() function to replace the past bar's close with the current bar's open when the value is na:

```
//@variable Is `true` when the `close` exceeds the last bar's `close` (or the current `open` if the value is `
bool risingClose = close > nz(close[1], open)
```

Protecting scripts against na instances helps to prevent undefined values from propagating in a calculation's results. For example, this script declares an `allTimeHigh` variable on the first bar. It then uses the math.max() between the `allTimeHigh` and the bar's high to update the `allTimeHigh` throughout its execution:

```
//@version=6
indicator("na protection demo", overlay = true)

//@variable The result of calculating the all-time high price with an initial value of `na`.
var float allTimeHigh = na

// Reassign the value of the `allTimeHigh`.
// Returns `na` on all bars because `math.max()` can't compare the `high` to an undefined value.
allTimeHigh := math.max(allTimeHigh, high)
```

```
plot(allTimeHigh) // Plots `na` on all bars.
```

This script plots a value of na on all bars, as we have not included any na protection in the code. To fix the behavior and plot the intended result (i.e., the all-time high of the chart's prices), we can use nz() to replace na values in the `allTimeHigh` series:

```
//@version=6
indicator("na protection demo", overlay = true)

//@variable The result of calculating the all-time high price with an initial value of `na`.
var float allTimeHigh = na

// Reassign the value of the `allTimeHigh`.
// We've used `nz()` to prevent the initial `na` value from persisting throughout the calculation.
allTimeHigh := math.max(nz(allTimeHigh), high)

plot(allTimeHigh)
```

Type templates

Type templates specify the data types that collections (arrays, matrices, and maps) can contain.

Templates for arrays and matrices consist of a single type identifier surrounded by angle brackets, e.g., `<int>`, `<label>`, and `<PivotPoint>` (where `PivotPoint` is a user-defined type (UDT)).

Templates for maps consist of two type identifiers enclosed in angle brackets, where the first specifies the type of *keys* in each key-value pair, and the second specifies the *value* type. For example, `<string, float>` is a type template for a map that holds `string` keys and `float` values.

Users can construct type templates from:

- Fundamental types: int, float, bool, color, and string
- The following special types: line, linefill, box, polyline, label, table, and chart.point
- User-defined types (UDTs)
- Enum types

Note that:

- Maps can use any of these types as *values*, but they can only accept fundamental types or enum types as *keys*.

Scripts use type templates to declare variables that reference collections, and when creating new collection instances. For example:

```
//@version=6
indicator("Type templates demo")

//@variable A variable initially assigned to `na` that accepts arrays of "int" values.
array<int> intArray = na
//@variable An empty matrix that holds "float" values.
floatMatrix = matrix.new<float>()
//@variable An empty map that holds "string" keys and "color" values.
stringColorMap = map.new<string, color>()
```

Type casting

Pine Script™ includes an automatic type-casting mechanism that *casts* (converts) **"int"** values to **"float"** when necessary. Variables or expressions requiring "float" values can also use "int" values because any integer can be represented as a floating point number with its fractional part equal to 0.

It's sometimes necessary to cast one type to another when auto-casting rules do not suffice. For such cases, the following type-casting functions are available: int(), float(), bool(), color(), string(), line(), linefill(), label(), box(), and table().

The example below shows a code that tries to use a "const float" value as the `length` argument in the ta.sma() function call. The script will fail to compile, as it cannot automatically convert the "float" value to the required "int" type:

```
//@version=6
indicator("Explicit casting demo", overlay = true)
```

```
//@variable The length of the SMA calculation. Qualified as "const float".
float LENGTH = 10.0

float sma = ta.sma(close, LENGTH) // Compilation error. The `length` parameter requires an "int" value.

plot(sma)
```

The code raises the following error: *"Cannot call 'ta.sma' with argument 'length'='LENGTH'. An argument of 'const float' type was used but a 'series int' is expected."*

The compiler is telling us that the code is using a "float" value where an "int" is required. There is no auto-casting rule to cast a "float" to an "int", so we must do the job ourselves. In this version of the code, we've used the int() function to explicitly convert our "float" `LENGTH` value to the "int" type within the ta.sma() call:

```
//@version=6
indicator("explicit casting demo")

//@variable The length of the SMA calculation. Qualified as "const float".
float LENGTH = 10.0

float sma = ta.sma(close, int(LENGTH)) // Compiles successfully since we've converted the `LENGTH` to "int".

plot(sma)
```

Explicit type casting is also handy when declaring variables assigned to na, as explained in the previous section.

For example, once could explicitly declare a variable with a value of na as a "label" type in either of the following, equivalent ways:

```
// Explicitly specify that the variable references "label" objects:
label myLabel = na

// Explicitly cast the `na` value to the "label" type:
myLabel = label(na)
```

Tuples

A *tuple* is a comma-separated set of expressions enclosed in brackets. When a function, method, or other local block returns more than one value, scripts return those values in the form of a tuple.

For example, the following user-defined function returns the sum and product of two "float" values:

```
//@function Calculates the sum and product of two values.
calcSumAndProduct(float a, float b) =>
    //@variable The sum of `a` and `b`.
    float sum = a + b
    //@variable The product of `a` and `b`.
    float product = a * b
    // Return a tuple containing the `sum` and `product`.
    [sum, product]
```

When we call this function later in the script, we use a *tuple declaration* to declare multiple variables corresponding to the values returned by the function call:

```
// Declare a tuple containing the sum and product of the `high` and `low`, respectively.
[hlSum, hlProduct] = calcSumAndProduct(high, low)
```

Keep in mind that unlike declaring single variables, we cannot explicitly define the types the tuple's variables (`hlSum` and `hlProduct` in this case), will contain. The compiler automatically infers the types associated with the variables in a tuple.

In the above example, the resulting tuple contains values of the same type ("float"). However, it's important to note that tuples can contain values of *multiple types*. For example, the `chartInfo()` function below returns a tuple containing "int", "float", "bool", "color", and "string" values:

```
//@function Returns information about the current chart.
chartInfo() =>
```

```
    //@variable The first visible bar's UNIX time value.
    int firstVisibleTime = chart.left_visible_bar_time
    //@variable The `close` value at the `firstVisibleTime`.
    float firstVisibleClose = ta.valuewhen(ta.cross(time, firstVisibleTime), close, 0)
    //@variable Is `true` when using a standard chart type, `false` otherwise.
    bool isStandard = chart.is_standard
    //@variable The foreground color of the chart.
    color fgColor = chart.fg_color
    //@variable The ticker ID of the current chart.
    string symbol = syminfo.tickerid
    // Return a tuple containing the values.
    [firstVisibleTime, firstVisibleClose, isStandard, fgColor, symbol]
```

Tuples are especially handy for requesting multiple values in one request.security() call.

For instance, this `roundedOHLC()` function returns a tuple containing OHLC values rounded to the nearest prices that are divisible by the symbol's minimum tick value. We call this function as the `expression` argument in request.security() to request a tuple containing daily OHLC values:

```
//@function Returns a tuple of OHLC values, rounded to the nearest tick.
roundedOHLC() =>
    [math.round_to_mintick(open), math.round_to_mintick(high), math.round_to_mintick(low), math.round_to_minti

[op, hi, lo, cl] = request.security(syminfo.tickerid, "D", roundedOHLC())
```

We can also achieve the same result by directly passing a tuple of rounded values as the `expression` in the request.security() call:

```
[op, hi, lo, cl] = request.security(
     syminfo.tickerid, "D",
     [math.round_to_mintick(open), math.round_to_mintick(high), math.round_to_mintick(low), math.round_to_mint
 )
```

Local blocks of conditional structures, including if and switch statements, can return tuples. For example:

```
[v1, v2] = if close > open
    [high, close]
else
    [close, low]
```

and:

```
[v1, v2] = switch
close > open => [high, close]
=>              [close, low]
```

However, ternaries cannot contain tuples, as the return values in a ternary statement are not considered local blocks:

```
// Not allowed.
[v1, v2] = close > open ? [high, close] : [close, low]
```

Note that all items within a tuple returned from a function are qualified as "simple" or "series", depending on its contents. If a tuple contains a "series" value, all other elements within the tuple will also adopt the "series" qualifier. For example:

```
//@version=6
indicator("Qualified types in tuples demo")

getParameters(float source, simple int length) =>
    // The second item in the tuple becomes a "series" if `source` is a "series"
    // because all returned tuple elements adopt the strongest qualifier.
    [source, length]

// Although the `length` argument is a "const int", the `len` variable is a "series" because `src` is a "serie
[src, len] = getParameters(close, 20)

// Causes a compilation error because `ta.ema()` requires a "simple int" `length` argument.
```

```
plot(ta.ema(src, len))
```

[Previous

Loops](#loops)[Next

Built-ins](#built-ins) User Manual/Language/Built-ins

Built-ins

Introduction

Pine Script™ has hundreds of *built-in* variables and functions. They provide your scripts with valuable information and make calculations for you, dispensing you from coding them. The better you know the built-ins, the more you will be able to do with your Pine scripts.

On this page, we present an overview of some of Pine's built-in variables and functions. They will be covered in more detail in the pages of this manual covering specific themes.

All built-in variables and functions are defined in the Pine Script™ v6 Reference Manual. It is called a "Reference Manual" because it is the definitive reference on the Pine Script™ language. It is an essential tool that will accompany you anytime you code in Pine, whether you are a beginner or an expert. If you are learning your first programming language, make the Reference Manual your friend. Ignoring it will make your programming experience with Pine Script™ difficult and frustrating — as it would with any other programming language.

Variables and functions in the same family share the same *namespace*, which is a prefix to the function's name. The ta.sma() function, for example, is in the `ta` namespace, which stands for "technical analysis". A namespace can contain both variables and functions.

Some variables have function versions as well, e.g.:

- The ta.tr variable returns the "True Range" of the current bar. The ta.tr(true) function call also returns the "True Range", but when the previous close value which is normally needed to calculate it is na, it calculates using `high - low` instead.
- The time variable gives the time at the open of the current bar. The time(timeframe) function returns the time of the bar's open from the `timeframe` specified, even if the chart's timeframe is different. The time(timeframe, session) function returns the time of the bar's open from the `timeframe` specified, but only if it is within the `session` time. The time(timeframe, session, timezone) function returns the time of the bar's open from the `timeframe` specified, but only if it is within the `session` time in the specified `timezone`.

Built-in variables

Built-in variables exist for different purposes. These are a few examples:

- Price- and volume-related variables: open, high, low, close, hl2, hlc3, ohlc4, and volume.
- Symbol-related information in the `syminfo` namespace: syminfo.basecurrency, syminfo.currency, syminfo.description, syminfo.main_tickerid, syminfo.mincontract, syminfo.mintick, syminfo.pointvalue, syminfo.prefix, syminfo.root, syminfo.session, syminfo.ticker, syminfo.tickerid, syminfo.timezone, and syminfo.type.
- Timeframe (a.k.a. "interval" or "resolution", e.g., 15sec, 30min, 60min, 1D, 3M) variables in the `timeframe` namespace: timeframe.isseconds, timeframe.isminutes, timeframe.isintraday, timeframe.isdaily, timeframe.isweekly, timeframe.ismonthly, timeframe.isdwm, timeframe.multiplier, timeframe.main_period, and timeframe.period.
- Bar states in the `barstate` namespace (see the Bar states page): barstate.isconfirmed, barstate.isfirst, barstate.ishistory, barstate.islast, barstate.islastconfirmedhistory, barstate.isnew, and barstate.isrealtime.
- Strategy-related information in the `strategy` namespace: strategy.equity, strategy.initial_capital, strategy.grossloss, strategy.grossprofit, strategy.wintrades, strategy.losstrades, strategy.position_size, strategy.position_avg_price, strategy.wintrades, etc.

Built-in functions

Many functions are used for the result(s) they return. These are a few examples:

- Math-related functions in the `math` namespace: math.abs(), math.log(), math.max(), math.random(), math.round_to_mintick() etc.
- Technical indicators in the `ta` namespace: ta.sma(), ta.ema(), ta.macd(), ta.rsi(), ta.supertrend(), etc.

- Support functions often used to calculate technical indicators in the `ta` namespace: ta.barssince(), ta.crossover(), ta.highest(), etc.
- Functions to request data from other symbols or timeframes in the `request` namespace: request.dividends(), request.earnings(), request.financial(), request.quandl(), request.security(), request.splits().
- Functions to manipulate strings in the `str` namespace: str.format(), str.length(), str.tonumber(), str.tostring(), etc.
- Functions used to define the input values that script users can modify in the script's "Settings/Inputs" tab, in the `input` namespace: input(), input.color(), input.int(), input.session(), input.symbol(), etc.
- Functions used to manipulate colors in the `color` namespace: color.from_gradient(), color.new(), color.rgb(), etc.

Some functions do not return a result but are used for their side effects, which means they do something, even if they don't return a result:

- Functions used as a declaration statement defining one of three types of Pine scripts, and its properties. Each script must begin with a call to one of these functions: indicator(), strategy() or library().
- Plotting or coloring functions: bgcolor(), plotbar(), plotcandle(), plotchar(), plotshape(), fill().
- Strategy functions placing orders, in the `strategy` namespace: strategy.cancel(), strategy.close(), strategy.entry(), strategy.exit(), strategy.order(), etc.
- Strategy functions returning information on indivdual past trades, in the `strategy` namespace: strategy.closedtrades.entry_bar_ strategy.closedtrades.entry_price(), strategy.closedtrades.entry_time(), strategy.closedtrades.exit_bar_index(), strategy.closedtrades.max_drawdown(), strategy.closedtrades.max_runup(), strategy.closedtrades.profit(), etc.
- Functions to generate alert events: alert() and alertcondition().

Other functions return a result, but we don't always use it, e.g.: hline(), plot(), array.pop(), label.new(), etc.

All built-in functions are defined in the Pine Script™ v6 Reference Manual. You can click on any of the function names listed here to go to its entry in the Reference Manual, which documents the function's signature, i.e., the list of *parameters* it accepts and the qualified type of the value(s) it returns (a function can return more than one result). The Reference Manual entry will also list, for each parameter:

- Its name.
- The qualified type of the value it requires (we use *argument* to name the values passed to a function when calling it).
- If the parameter is required or not.

All built-in functions have one or more parameters defined in their signature. Not all parameters are required for every function.

Let's look at the ta.vwma() function, which returns the volume-weighted moving average of a source value. This is its entry in the Reference Manual:

The entry gives us the information we need to use it:

- What the function does.
- Its signature (or definition):

```
ta.vwma(source, length) → series float
```

- The parameters it includes: `source` and `length`
- The qualified type of the result it returns: "series float".
- An example showing it in use: `plot(ta.vwma(close, 15))`.
- An example showing what it does, but in long form, so you can better understand its calculations. Note that this is meant to explain — not as usable code, because it is more complicated and takes longer to execute. There are only disadvantages to using the long form.
- The "RETURNS" section explains exacty what value the function returns.
- The "ARGUMENTS" section lists each parameter and gives the critical information concerning what qualified type is required for arguments used when calling the function.
- The "SEE ALSO" section refers you to related Reference Manual entries.

This is a call to the function in a line of code that declares a `myVwma` variable and assigns the result of `ta.vwma(close, 20)` to it:

```
myVwma = ta.vwma(close, 20)
```

Note that:

- We use the built-in variable close as the argument for the `source` parameter.
- We use `20` as the argument for the `length` parameter.

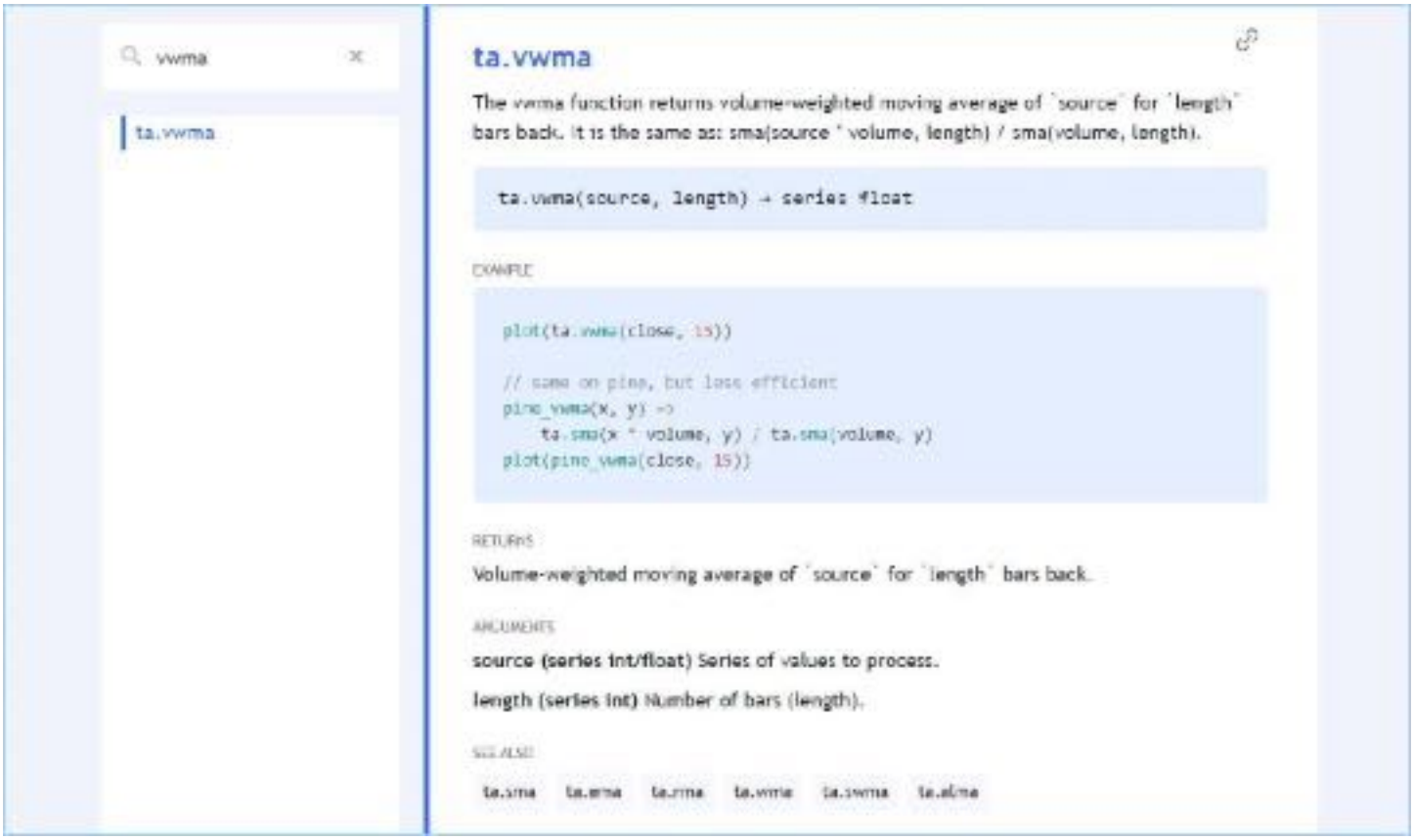

Figure 35: image

- If placed in the global scope (i.e., starting in a line's first position), it will be executed by the Pine Script™ runtime on each bar of the chart.

We can also use the parameter names when calling the function. Parameter names are called *keyword arguments* when used in a function call:

```
myVwma = ta.vwma(source = close, length = 20)
```

You can change the position of arguments when using keyword arguments, but only if you use them for all your arguments. When calling functions with many parameters such as indicator(), you can also forego keyword arguments for the first arguments, as long as you don't skip any. If you skip some, you must then use keyword arguments so the Pine Script™ compiler can figure out which parameter they correspond to, e.g.:

```
indicator("Example", "Ex", true, max_bars_back = 100)
```

Mixing things up this way is not allowed:

```
indicator(precision = 3, "Example") // Compilation error!
```

When calling built-ins, it is critical to ensure that the arguments you use are of the required qualified type, which will vary for each parameter.

To learn how to do this, one needs to understand Pine Script™'s type system. The Reference Manual entry for each built-in function includes an "ARGUMENTS" section which lists the qualified type required for the argument supplied to each of the function's parameters.

[Previous

Type system](#type-system)[Next

User-defined functions](#user-defined-functions) User Manual/Language/User-defined functions

User-defined functions

Introduction

User-defined functions are functions that you write, as opposed to the built-in functions in Pine Script™. They are useful to define calculations that you must do repetitevely, or that you want to isolate from your script's main section of calculations. Think of user-defined functions as a way to extend the capabilities of Pine Script™, when no built-in function will do what you need.

You can write your functions in two ways:

- In a single line, when they are simple, or
- On multiple lines

Functions can be located in two places:

- If a function is only used in one script, you can include it in the script where it is used. See our Style guide for recommendations on where to place functions in your script.
- You can create a Pine Script™ library to include your functions, which makes them reusable in other scripts without having to copy their code. Distinct requirements exist for library functions. They are explained in the page on libraries.

Whether they use one line or multiple lines, user-defined functions have the following characteristics:

- They cannot be embedded. All functions are defined in the script's global scope.
- They do not support recursion. It is **not allowed** for a function to call itself from within its own code.
- The type of the value returned by a function is determined automatically and depends on the type of arguments used in each particular function call.
- A function's returned value is that of the last value in the function's body.
- Each instance of a function call in a script maintains its own, independent history.

Single-line functions

Simple functions can often be written in one line. This is the formal definition of single-line functions:

```
<function_declaration>    <identifier>(<parameter_list>) => <return_value>
<parameter_list>    {<parameter_definition>{, <parameter_definition>}}
<parameter_definition>    [<identifier> = <default_value>]
<return_value>    <statement> | <expression> | <tuple>
```

Here is an example:

```
f(x, y) => x + y
```

After the function `f()` has been declared, it's possible to call it using different types of arguments:

```
a = f(open, close)
b = f(2, 2)
c = f(open, 2)
```

In the example above, the type of variable `a` is *series* because the arguments are both *series*. The type of variable `b` is *integer* because arguments are both *literal integers*. The type of variable `c` is *series* because the addition of a *series* and *literal integer* produces a *series* result.

Multi-line functions

Pine Script™ also supports multi-line functions with the following syntax:

```
<identifier>(<parameter_list>) =>    <local_block>
<identifier>(<list of parameters>) =>    <variable declaration>    ...    <variable declaration or expression>
```

where:

```
<parameter_list>    {<parameter_definition>{, <parameter_definition>}}
<parameter_definition>    [<identifier> = <default_value>]
```

The body of a multi-line function consists of several statements. Each statement is placed on a separate line and must be preceded by 1 indentation (4 spaces or 1 tab). The indentation before the statement indicates that it is a part of the body of the function and not part of the script's global scope. After the function's code, the first statement without an indent indicates the body of the function has ended.

Either an expression or a declared variable should be the last statement of the function's body. The result of this expression (or variable) will be the result of the function's call. For example:

```
geom_average(x, y) =>
    a = x*x
    b = y*y
    math.sqrt(a + b)
```

The function `geom_average` has two arguments and creates two variables in the body: `a` and `b`. The last statement calls the function `math.sqrt` (an extraction of the square root). The `geom_average` call will return the value of the last expression: `(math.sqrt(a + b))`.

Scopes in the script

Variables declared outside the body of a function or of other local blocks belong to the *global* scope. User-declared and built-in functions, as well as built-in variables also belong to the global scope.

Each function has its own *local* scope. All the variables declared within the function, as well as the function's arguments, belong to the scope of that function, meaning that it is impossible to reference them from outside — e.g., from the global scope or the local scope of another function.

On the other hand, since it is possible to refer to any variable or function declared in the global scope from the scope of a function (except for self-referencing recursive calls), one can say that the local scope is embedded into the global scope.

In Pine Script™, nested functions are not allowed, i.e., one cannot declare a function inside another one. All user functions are declared in the global scope. Local scopes cannot intersect with each other.

Functions that return multiple results

In most cases a function returns only one result, but it is possible to return a list of results (a *tuple*-like result):

```
fun(x, y) =>
    a = x+y
    b = x-y
    [a, b]
```

Special syntax is required for calling such functions:

```
[res0, res1] = fun(open, close)
plot(res0)
plot(res1)
```

Limitations

User-defined functions can contain calls to most built-in functions within their local scopes. However, calls to any of the following functions are **not** allowed in a function's scope: barcolor(), bgcolor(), plot(), plotshape(), plotchar(), plotarrow(), plotcandle(), plotbar(), hline(), fill(), alertcondition(), indicator(), strategy(), or library().

[Previous

Built-ins](#built-ins)[Next

Objects](#objects) User Manual/Language/Objects

ADVANCED

Objects

Introduction

Pine Script™ objects are instances of *user-defined types* (UDTs). They are the equivalent of variables containing parts called *fields*, each able to hold independent values that can be of various types.

Experienced programmers can think of UDTs as methodless classes. They allow users to create custom types that organize different values under one logical entity.

Creating objects

Before an object can be created, its type must be defined. The User-defined types section of the Type system page explains how to do so.

Let's define a `pivotPoint` type to hold pivot information:

```
type pivotPoint
    int x
    float y
    string xloc = xloc.bar_time
```

Note that:

- We use the type keyword to declare the creation of a UDT.
- We name our new UDT `pivotPoint`.
- After the first line, we create a local block containing the type and name of each field.
- The `x` field will hold the x-coordinate of the pivot. It is declared as an "int" because it will hold either a timestamp or a bar index of "int" type.
- `y` is a "float" because it will hold the pivot's price.
- `xloc` is a field that will specify the units of `x`: xloc.bar_index or xloc.bar_time. We set its default value to xloc.bar_time by using the `=` operator. When an object is created from that UDT, its `xloc` field will thus be set to that value.

Now that our `pivotPoint` UDT is defined, we can proceed to create objects from it. We create objects using the UDT's `new()` built-in method. To create a new `foundPoint` object from our `pivotPoint` UDT, we use:

```
foundPoint = pivotPoint.new()
```

We can also specify field values for the created object using the following:

```
foundPoint = pivotPoint.new(time, high)
```

Or the equivalent:

```
foundPoint = pivotPoint.new(x = time, y = high)
```

At this point, the `foundPoint` object's `x` field will contain the value of the time built-in when it is created, `y` will contain the value of high and the `xloc` field will contain its default value of xloc.bar_time because no value was defined for it when creating the object.

Object placeholders can also be created by declaring na object names using the following:

```
pivotPoint foundPoint = na
```

This example displays a label where high pivots are detected. The pivots are detected `legsInput` bars after they occur, so we must plot the label in the past for it to appear on the pivot:

```
//@version=6
indicator("Pivot labels", overlay = true)
int legsInput = input(10)

// Define the `pivotPoint` UDT.
type pivotPoint
    int x
    float y
    string xloc = xloc.bar_time

// Detect high pivots.
pivotHighPrice = ta.pivothigh(legsInput, legsInput)
if not na(pivotHighPrice)
    // A new high pivot was found; display a label where it occurred `legsInput` bars back.
    foundPoint = pivotPoint.new(time[legsInput], pivotHighPrice)
    label.new(
      foundPoint.x,
      foundPoint.y,
      str.tostring(foundPoint.y, format.mintick),
      foundPoint.xloc,
      textcolor = color.white)
```

Take note of this line from the above example:

```
foundPoint = pivotPoint.new(time[legsInput], pivotHighPrice)
```

This could also be written using the following:

```
pivotPoint foundPoint = na
foundPoint := pivotPoint.new(time[legsInput], pivotHighPrice)
```

When using the var keyword while declaring a variable assigned to an object of a user-defined type, the keyword automatically applies to all the object's fields:

```
//@version=6
indicator("Objects using `var` demo")

//@type A custom type to hold index, price, and volume information.
type BarInfo
    int   index = bar_index
    float price = close
    float vol   = volume

//@variable A `BarInfo` instance whose fields persist through all iterations, starting from the first bar.
var BarInfo firstBar = BarInfo.new()
//@variable A `BarInfo` instance declared on every bar.
BarInfo currentBar = BarInfo.new()

// Plot the `index` fields of both instances to compare the difference.
plot(firstBar.index)
plot(currentBar.index)
```

It's important to note that assigning an object to a variable that uses the varip keyword does *not* automatically allow the object's fields to persist without rolling back on each *intrabar* update. One must apply the keyword to each desired field in the type declaration to achieve this behavior. For example:

```
//@version=6
indicator("Objects using `varip` fields demo")

//@type A custom type that counts the bars and ticks in the script's execution.
type Counter
    int       bars  = 0
    varip int ticks = 0

//@variable A `Counter` object whose reference persists throughout all bars.
var Counter counter = Counter.new()

// Add 1 to the `bars` and `ticks` fields. The `ticks` field is not subject to rollback on unconfirmed bars.
counter.bars  += 1
counter.ticks += 1

// Plot both fields for comparison.
plot(counter.bars, "Bar counter", color.blue, 3)
plot(counter.ticks, "Tick counter", color.purple, 3)
```

Note that:

- We used the var keyword to specify that the `Counter` object assigned to the `counter` variable persists throughout the script's execution.
- The `bars` field rolls back on realtime bars, whereas the `ticks` field does not since we included varip in its declaration.

Changing field values

The value of an object's fields can be changed using the := reassignment operator.

This line of our previous example:

```
foundPoint = pivotPoint.new(time[legsInput], pivotHighPrice)
```

Could be written using the following:

```
foundPoint = pivotPoint.new()
foundPoint.x := time[legsInput]
foundPoint.y := pivotHighPrice
```

Collecting objects

Pine Script™ collections (arrays, matrices, and maps) can contain objects, allowing users to add virtual dimensions to their data structures. To declare a collection of objects, pass a UDT name into its type template.

This example declares an empty array that will hold objects of a `pivotPoint` user-defined type:

```
pivotHighArray = array.new<pivotPoint>()
```

To explicitly declare the type of a variable as an array, matrix, or map of a user-defined type, use the collection's type keyword followed by its type template. For example:

```
var array<pivotPoint> pivotHighArray = na
pivotHighArray := array.new<pivotPoint>()
```

Let's use what we have learned to create a script that detects high pivot points. The script first collects historical pivot information in an array. It then loops through the array on the last historical bar, creating a label for each pivot and connecting the pivots with lines:

Figure 36: image

```
//@version=6
indicator("Pivot Points High", overlay = true)

int legsInput = input(10)

// Define the `pivotPoint` UDT containing the time and price of pivots.
type pivotPoint
    int openTime
    float level

// Create an empty `pivotPoint` array.
var pivotHighArray = array.new<pivotPoint>()

// Detect new pivots (`na` is returned when no pivot is found).
```